

操作系统概述

2024年11月13日 17:56

一、概念：

操作系统是控制和**管理**整个计算机系统的**硬件与软件资源**的，合理的组织调度计算机的工作与分配，进而**为用户和其他软件提供方便接口和环境**的程序的集合。操作系统是计算机系统中最基本的**系统软件**。

1、**操作系统是资源的管理者**：从下到上管理硬件（处理机、存储器、文件、设备）、操作系统、应用程序

2、**提供接口和环境**：起到承上启下的作用，屏蔽了硬件的复杂性，用户不用直接操作硬件也能使用计算机资源，操作系统也不允许用户直接操作硬件资源，用户只能通过系统调用来请求操作系统来为其服务，间接的使用系统资源，提供了命令接口（联机命令接口cmd+脱机命令接口bat）和给应用软件提供程序接口（系统调用）和给用户图形化界面

3、**系统软件**：操作系统是最接近硬件的软件，实现了对计算机资源的扩充，把裸机变成了功能强+使用方便的机器

二、操作系统的特征

1、并发：两个或者多个活动在同一给定的时间间隔中**交替进行**（注：并发在**宏观上并行**的但是**微观上是交替进行**的，只是间隔太小人眼看不出，**并行是同时运行**，同时完成两个或者多个操作，只有多核CPU才能实现并行，例如4核处理器可以实现4个进程同时操作）

2、共享：计算机系统内的**资源可以被多个进程共用**，其中分为互斥共享方式和同时共享方式

互斥共享：当A访问完后B才能访问，例如QQ视频聊天占用了摄像头，此时微信视频聊天就打不开摄像头了

同时共享：A和B可以同时访问一个资源，例如QQ和微信可以同时发送文件包（一个时间段内间隔发送）

3、异步：由于资源有限以及等待访问等原因，进程以走走停停**不可预知**的方式向前推进

4、虚拟：把**一个物理上的实体变成好几个逻辑上的对应物**，可以分为时分复用技术（虚拟处理器）和空分复用技术（虚拟存储器）

时分复用就是1个CPU虚拟出多个逻辑上的CPU来为多个用户服务，并且每个用户都感觉计算机是专门为自己服务的

空分复用就是1个内存虚拟出多个逻辑上的内存空间，从逻辑上扩充了内存容量

5、**最基本的特征：并发和共享**

三、操作系统的历程

1、手工操作阶段：此时没有操作系统，程序的装入运行输出都是人工，用户独占全机。此时资源利用率低，人机速度不匹配产生矛盾。

2、单道批处理阶段：将写好的一堆作业以**脱机的方式输入磁带**，并且配上监督程序使得作业能**一批一批的连续处理**，特点是自动性（一批作业自动逐个执行）、顺序性（先放的先处理）、单道性（内存中只能有一道程序运行）。但是系统利用率低因为IO速度和CPU的速度不匹配，例如作业运行道一半要调用打印机然后慢慢打，后面的程序就只能等着。

3、多道批处理阶段：用户提交的作业都放在外存上排成一个队列，按照一定的算法从后备队列中选择几个调入内存，相互**穿插运行**，**共享系统的资源**，某个程序因为IO暂停的时候CPU转头去处理另一个（中断机制实现）。特点是多道（内存中同时放多个程序）、宏观上并行微观上并发、制约性（多个进程之间竞争资源，相互制约）、间断性（考虑到公平竞争，一个程序的执行是间断的）、失去了顺序性。提高了资源利用率，能让CPU保持忙碌状态，但是进程放进去就不能更改了，没有提供人机交互的功能。

4、分时操作系统：主要用于交互式作业，利用**分时技术**（处理器运行时间分成很多时间片，将时间片轮流的分给各联机作业使用）的操作系统，**提供了人机交互**的功能。特点是：多路性（多个用户终端使用一台主机）、交互性（用户可以通过终端的方式直接控制程序运行，实现人机交互）、独立性（用户独立操作，互不干扰）、及时性（系统响应时间快）。缺点是在一定的场合下不能得到非常及时的处理（规定处理时间短于时间片）

5、实时操作系统：**能优先处理一些紧急任务**的操作系统，分为硬实时系统（必须及时响应，例如导弹拦截）和软实时系统（可以松一些例如订票系统），从可靠性来看实时操作系统更好，从交互性来看分时操作系统更好。

6、网络操作系统 和 分布式操作系统：

7、个人操作系统

四、运行环境

1、处理器运行模式：计算机系统中CPU处理两种不同性质的程序，有**内核程序**和**应用程序**，前者管理后者，因此前者需要一些**特权指令**，这些特权指令后者无法执行。

特权指令：不允许用户程序只允许操作系统使用的指令

非特权指令：普通的运算指令，只能访问地址空间不能访问系统的软硬件资源

内核程序：系统的管理者，可以执行一切指令，运行在核心态（管态、核心态）

应用程序：普通的用户程序，只能执行非特权指令，运行在用户态（目态），用户请求系统服务的时候使用**访管指令**，这个是非特权指令

2、内核是计算机上配置的底层软件，它管理着各种系统资源，是链接应用程序和硬件的桥梁，由硬件紧密关联的模块和运行频率较高的模块组成，大多数操作系统的内核包括以下4个方面：

时钟管理：时钟的作用第一是计时，提供标准系统时间，然后通过时钟中断来实现进程之间的切换。例如分时系统要提供时间片，时钟来给时间片计时；批处理系统要用时钟来衡量每一个作业的运行进度等等。

中断机制：引入中断机制是为了提高CPU的利用率，在该机制中只有一小部分属于内核来保护和恢复中断现场并移交控制权。用户态到核心态就是通过中断机制来实现的。

原语：原子语，不可再分的程序，操作只能一气呵成，处于操作系统的最底层，最接近硬件的一部分，运行时间较短且调用频繁。

系统控制处理：为了实现有效的管理，系统需要一些基本操作，包括：进程管理、存储器管理、设备管理

3、中断和异常

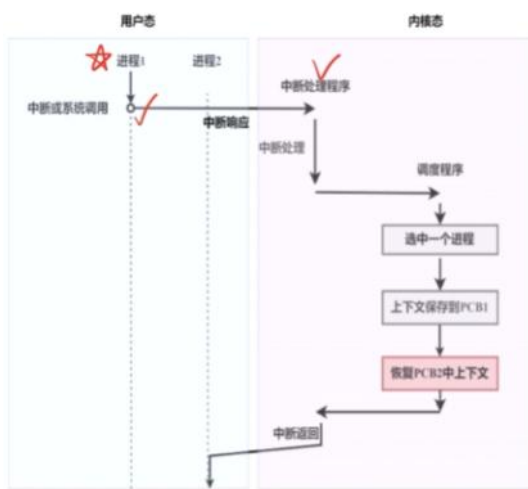
系统不允许用户程序实现内核态的功能，但是它们有时候又需要这些功能，那么就需要建立一个入口来让运行用户态的CPU转移到内核态，这是**通过硬件来实现的**（0为内核态1为用户态），这个入口只能通过中断或者异常来实现。

内中断又称异常：就是来自内部的中断信号，例如程序的非法操作码、地址越界、运算溢出等事件，这些事件都不可屏蔽，必须立刻处理

外中断又称中断：就是来自外部的中断信号，例如IO设备发出的中断信号表示IO操作已经完成，人工干预等。分为可屏蔽和不可屏蔽的中断。

4、系统调用

系统调用就是系统给程序员（应用程序）提供的唯一接口，可以获得操作系统的服务，在用户态中发生核心态处理（注意是用户跟系统说，然后系统替用户干，不是用户取得了权限），系统调用引起的中断叫访管中断。



进程与线程

2025年5月21日星期三 下午5:04

一、进程的概念和组成

1、进程的概念：

程序是静态的，是存放在磁盘里的可执行文件，是一系列指令的集合（exe文件）

进程是动态的，是程序的一次执行过程，同一个程序多次执行会对应多个进程

进程 性能 应用历史记录 启动 用户 详细信息 服务

名称	状态	7% CPU	45% 内存	1% 磁盘	0% 网络
> Microsoft Edge (8)		1.6%	724.1 MB	0 MB/秒	0 Mbps
桌面窗口管理器		1.6%	37.0 MB	0 MB/秒	0 Mbps
System		1.0%	0.1 MB	0.1 MB/秒	0 Mbps
> 任务管理器		1.0%	46.6 MB	0 MB/秒	0 Mbps
> OneNote for Windows 10 (2)		0.7%	75.6 MB	0.1 MB/秒	0 Mbps
Windows 音频设备图形隔离		0.5%	9.0 MB	0 MB/秒	0 Mbps
系统中断		0.3%	0 MB	0 MB/秒	0 Mbps
> 服务主机: Windows Manage...		0%	13.0 MB	0 MB/秒	0 Mbps
WMI Provider Host		0%	15.9 MB	0 MB/秒	0 Mbps
NVIDIA Web Helper Service (...)		0%	7.0 MB	0 MB/秒	0 Mbps
> DAX API		0%	3.4 MB	0.1 MB/秒	0 Mbps
Microsoft Edge		0%	15.3 MB	0.1 MB/秒	0.1 Mbps
> 服务主机: 连接设备平台用户服...		0%	10.0 MB	0 MB/秒	0 Mbps
Windows 资源管理器		0%	73.2 MB	0 MB/秒	0 Mbps

2、进程的特征：

相比于单个程序的顺序执行，进程具有

动态性：进程具有生命周期，是动态产生、变化、消亡的，是最基本的特征。

并发性：多个进程同时存在于内存中，能在一个时间段内同时运行，微观内交替执行。

独立性：进程是一个独立的个体，能独立获得资源和独立的接收调度。

异步性：由于进程之间的相互制约，进程的执行以一种不可预知的方式向前执行。

结构性：每个进程都配置一个PCB，结构上看都由程序段、PCB、数据段组成

二、进程的组成

进程是一个独立的个体，由以下三部分组成，最核心的就是PCB（进程控制块）

1、进程控制块PCB

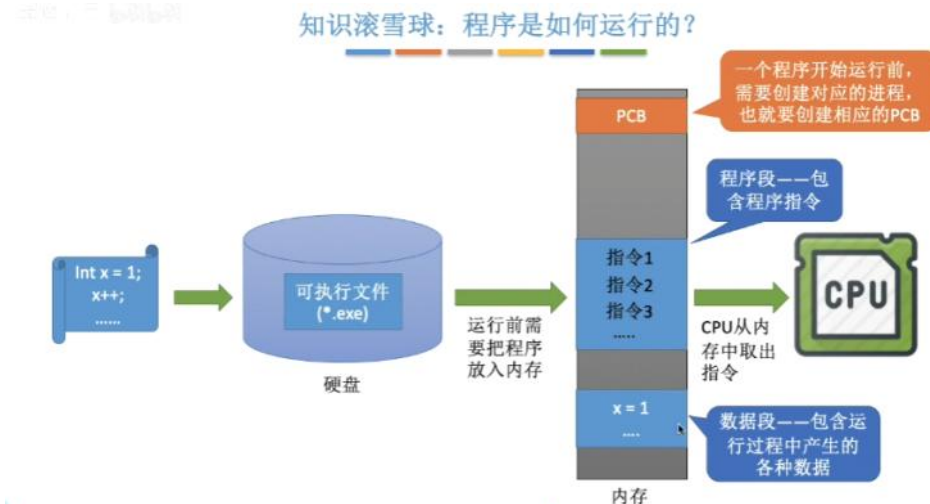
PCB是进程存在的唯一标志，当进程被创建时，操作系统会为其创建一个PCB，当进程结束时操作系统会回收相应的PCB。PCB里面主要包含以下信息：

名称	PID	状态	用户名	CPU	内存(活动...)	UAC 虚拟化
AdobePCBroker.exe	16848	正在运行	LENOVO	00	1,012 K	已禁用
AggregatorHost.exe	10336	正在运行	SYSTEM	00	1,736 K	不允许
ApplicationFrame...	13680	正在运行	LENOVO	00	15,824 K	已禁用
ApplicationWebSe...	9232	正在运行	SYSTEM	00	4,788 K	不允许
audiodg.exe	1796	正在运行	LOCAL SE...	00	9,116 K	不允许
CCXProcess.exe	16392	正在运行	LENOVO	00	260 K	已禁用
clash-verge-servic...	5412	正在运行	SYSTEM	00	944 K	不允许
CompPkgSrv.exe	16772	正在运行	LENOVO	00	988 K	已禁用
conhost.exe	4908	正在运行	SYSTEM	00	5,444 K	不允许
conhost.exe	7288	正在运行	SYSTEM	00	5,616 K	不允许
conhost.exe	8876	正在运行	SYSTEM	00	5,516 K	不允许
conhost.exe	11644	正在运行	LENOVO	00	376 K	已禁用
conhost.exe	16440	正在运行	LENOVO	00	5,492 K	已禁用
conhost.exe	16528	正在运行	LENOVO	00	5,476 K	已禁用
conhost.exe	15904	正在运行	SYSTEM	00	5,676 K	不允许
csrss.exe	820	正在运行	SYSTEM	00	1,300 K	不允许
csrss.exe	580	正在运行	SYSTEM	00	1,236 K	不允许
ctfmon.exe	9804	正在运行	LENOVO	00	8,872 K	已禁用
DAX3API.exe	5336	正在运行	SYSTEM	00	3,516 K	不允许
DAX3API.exe	6828	正在运行	SYSTEM	00	1,500 K	不允许
dllhost.exe	7548	正在运行	LENOVO	00	1,816 K	已禁用
dwm.exe	1768	正在运行	DWM-1	00	38,900 K	已禁用
esif.uf.exe	5728	正在运行	SYSTEM	00	648 K	不允许

如上图：PID是用于让操作系统来区分各个进程的，相当于每一个进程的“身份证号”，当进程创建时，操作系统会为该进程分配一个唯一的PID。除了PID还有UID进程所属用户等信息，这些信息集合起来保存在PCB中。

PCB是给操作系统用的，程序段和数据段是进程给自己用的。

2、程序段和数据段



如上图：程序段是给操作系统执行的指令的集合，数据段是自带的或者运行期间产生的数据。假设同时挂3个QQ号，就产生了3个进程，他们的程序段都是一样的，PCB和数据段各不相同。

3、C语言中的详细分段

C语言编写的程序在内存中分为正文段、数据堆段、数据栈段；全局变量、常量、二进制代码存放在正文段；动态分配malloc分配的存储区在数据堆段；临时使用的变量存放在数据栈段。

可能考法：根据C语言程序回答存储区域

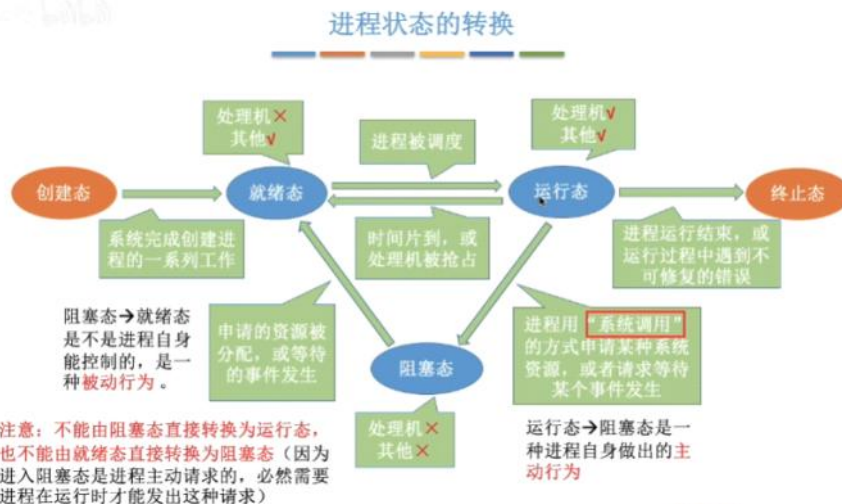


三、进程的状态

进程的状态是不断的变化的，通常情况下，进程的状态由以下5种，其中运行、就绪、阻塞是基本状态。

- 1、创建态：进程正在被创建，系统为其分配资源，初始化PCB
- 2、就绪态：进程 完成创建，具备运行条件，但是CPU此时比较忙，暂时不能为其服务
- 3、运行态：当CPU空闲时，操作系统会在就绪队列中选择一个就绪进程来运行，在CPU运行的进程处于运行态，CPU会执行对应程序的指令。
- 4、阻塞态：进程运行的时候可能会等待系统资源的分配（如等待打印机、摄像头空闲下来）等事件，这段时间进程无法往下执行了，操作系统会让这个进程下CPU，进入阻塞态。当进程等待的事件完成后，操作系统会由阻塞态转为就绪态。
- 5、终止态：当进程执行到exit()调用或者其他情况的时候，会请求操作系统终止这些进程，此时这些进程会进入“终止态”，下CPU，回收内存空间，最后回收PCB，然后进程彻底消失了。

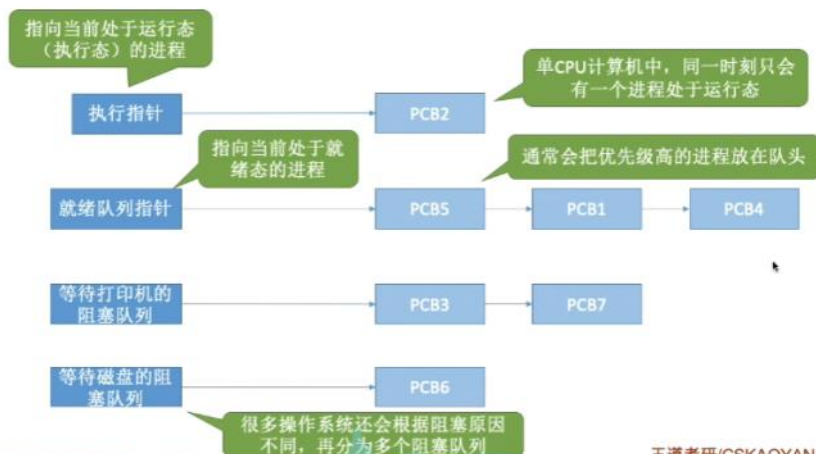
注意：运行到阻塞是进程主动进行的，阻塞到就绪态是被动进行的，不是自己能控制的。当进程处于运行态的时候，进程运行时间超出时间片的限制，就会从运行态转移到就绪态。



四、进程的组织：

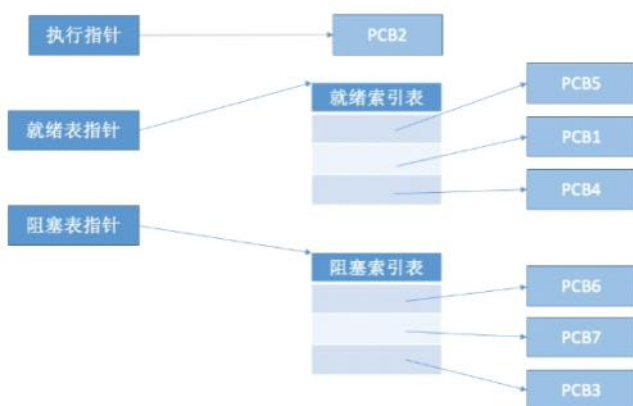
- 1、链接方式

进程的组织——链接方式



2、索引方式

进程的组织——索引方式



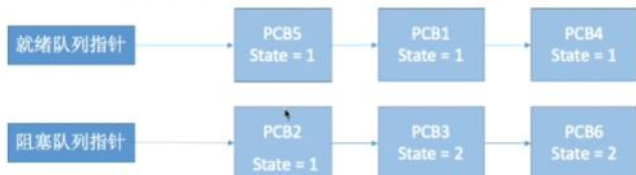
五、进程控制：进程控制就是要实现进程状态的转换

1、进程的控制是通过“原语”来实现的



如上图：原语处于计算机软件的最底层，离硬件非常接近，具有不可分割的原子性，运行起来一气呵成。

Eg: 假设PCB中的变量 state 表示进程当前所处状态，1表示就绪态，2表示阻塞态...



假设此时进程2等待的事件发生，则操作系统中，负责进程控制的内核程序至少需要做这样两件事：

①将PCB2的 state 设为 1

②将PCB2从阻塞队列放到就绪队列

完成了第一步后收到中断信号，那么PCB2的state=1，但是它却被放在阻塞队列里

王道考研/CSKAQYAN.C

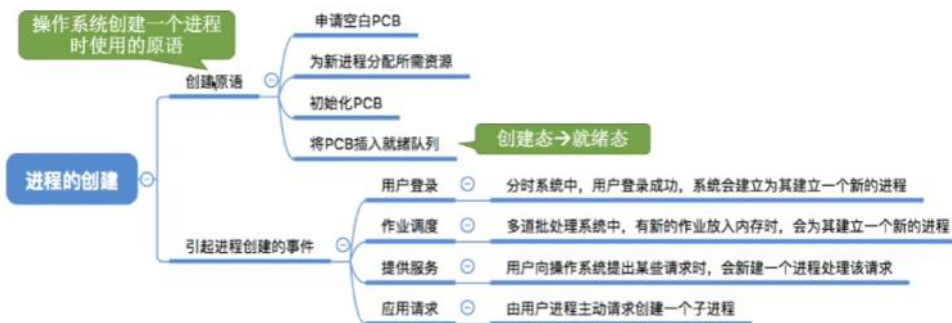
如果不能“一气呵成”，就有可能导致操作系统中某些关键数据结构信息不统一，从而影响操作系统进行别的工作。

2、原语的原子性是如何实现的

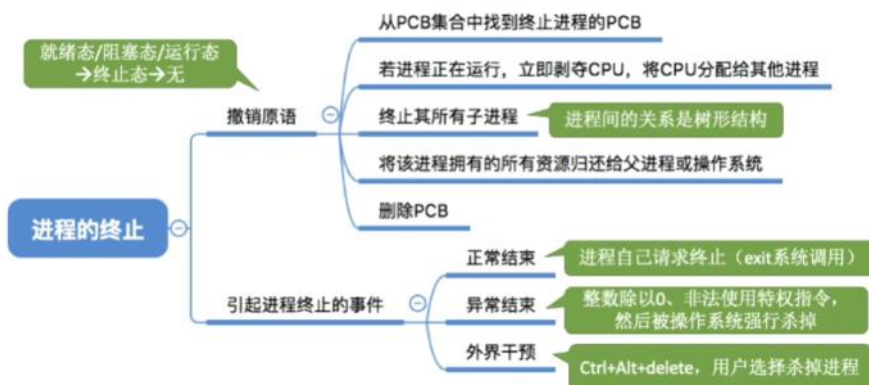
原语的执行一气呵成不能被中断，可以用“关中断指令”和“开中断指令”这两个特权指令来实现。CPU在执行完每一个指令后都会检查一下有没有中断指令，所谓关中断指令就是让CPU不再例行检查中断信号了，直到执行开中断指令之后才会恢复检查。这样在关中断和开中断之间的指令序列是不可以被中断的，就实现了原子性。



3、进程的创建：创建原语



4、进程的终止：撤销原语



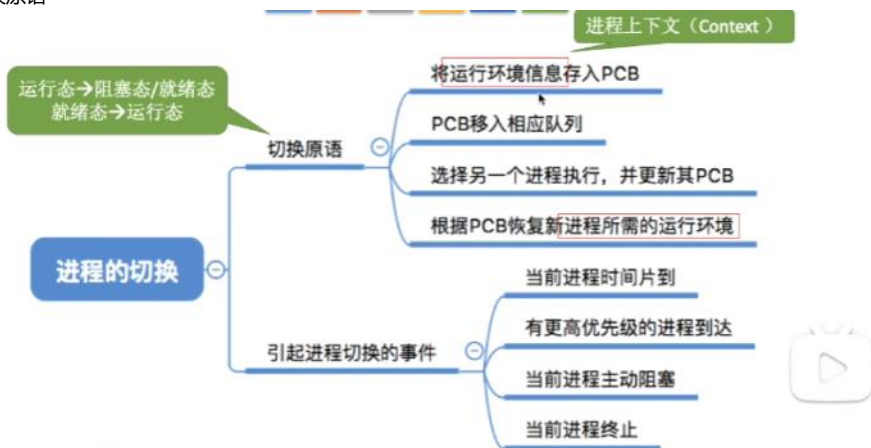
注意：子进程就是由进程创建的，为自己服务的进程，进程在创建子进程后，会将自己手里的内存资源分配给子进程，子进程运行完毕后会资源返回给进程。

Microsoft Edge (9)	4.8%	713.4 MB	0.1 MB/秒	0 Mbps
标签页: 2.1.4_进程控制_哔哩哔哩...	3.1%	415.9 MB	0 MB/秒	0 Mbps
GPU 进程	1.7%	122.9 MB	0 MB/秒	0 Mbps
浏览器	0%	133.2 MB	0.1 MB/秒	0 Mbps
实用工具: Audio Service	0%	2.6 MB	0 MB/秒	0 Mbps
实用工具: Storage Service	0%	5.3 MB	0 MB/秒	0 Mbps
备用呈现器	0%	9.7 MB	0 MB/秒	0 Mbps
实用工具: Media Foundatio...	0%	4.5 MB	0 MB/秒	0 Mbps
子帧: https://hdslib.com/	0%	17.0 MB	0 MB/秒	0 Mbps
实用工具: Wallet Service	0%	2.5 MB	0 MB/秒	0 Mbps

5、进程的阻塞与唤醒：阻塞原语和唤醒原语



6、切换原语



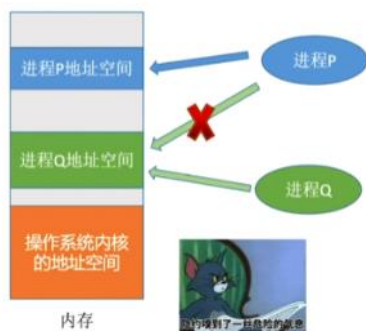
注意：运行环境指的是进程在寄存器中存放的中间结果，当寄存器运行其他进程的时候会覆盖掉原来的结果，因此需要将进程的运行环境存入PCB中，在下次回到寄存器的时候恢复现场。

六、进程之间的通信

进程之间的通信就是指两个进程之间产生数据交互，例如QQ文件分享到微信好友，需要QQ和微信这2个进程来配合。

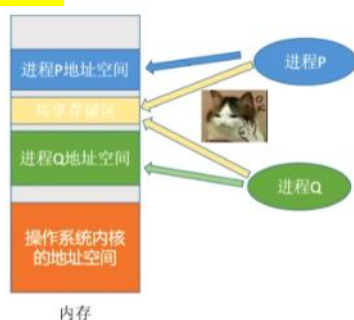
1、为什么进程之间的通信需要操作系统的支持

进程是分配系统资源的单位，因此各个进程拥有的内存地址空间是相互独立的，一个进程无法直接访问另一个进程的地址空间。如果进程P和进程Q需要进行通信，必须要有操作系统的支持才能完成进程的通信。



2、进程通信的3种方式

共享存储：在进程P和进程Q外申请一个共享存储区，并且映射到进程的虚拟地址空间，这个存储区可以被多个进程共享，因此数据交换可以通过共享区来实现。



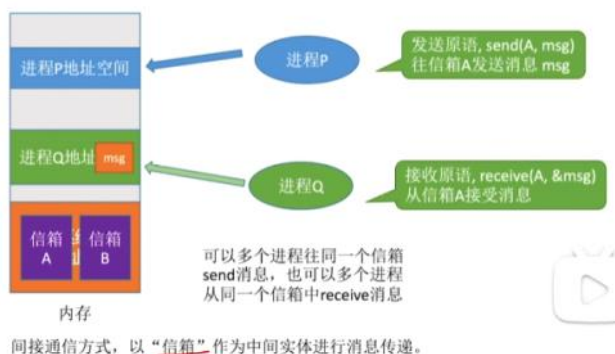
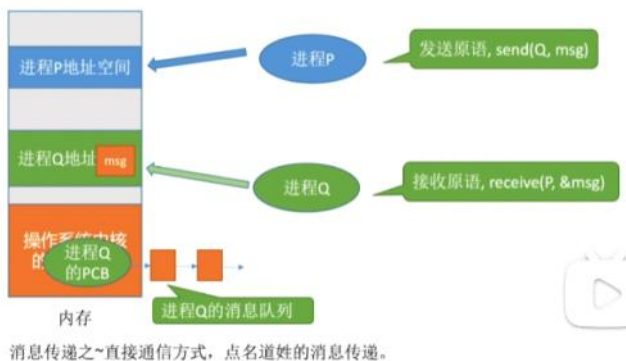
注意：只需要简单的增加一个页表项或者段表项，就可以将同一个共享区域映射到各个进程的地址空间中。为了避免访问冲突，各个进程对共享空间的访问是**互斥**的，可以使用操作系统内核提供的互斥工具来实现（例如PV操作）。

共享存储的实现有**基于数据结构的共享**，例如空间里只能放一个长度为10的数组，这种方式速度慢限制多，是一种**低级通信**的方式。还有**基于存储区的共享**，就是在内存中画出一块共享存储区，什么数据什么位置都是进程来决定的不是操作系统决定的，这种方式灵活性高，速度快，是**高级通信**方式。

消息传递：进程之间的数据交换以**格式化消息**为单位，进程之间通过操作系统提供的“发送消息”、“接收消息”两个原语进行数据交换。



消息传递又可以分为：直接通信方式，就是发送进程P指明接收进程的ID,通过原语send(Q,msg)来将消息复制到进程Q的消息队列中（位于操作系统内核中进程Q的PCB中），进程Q通过接收原语receive(P,&msg)来接收消息，把消息复制到进程Q里面；间接通信方式就是通过“信箱”来实现间接的通信，进程P向操作系统申请邮箱A，通过发送原语send(A,msg)来往A信箱发送消息，进程Q使用原语receive(A,&msg)来接收到消息。



管道通信：在内存中开辟一个大小固定的内存缓冲区称为管道，在管道内是先进先出的循环队列。和共享存储相比，共享存储开辟的空间很自由，想怎么填怎么取都行，然而管道通信只能是先进先出的读写。数据的传输只能是单向的（半双工通信），如果要实现全双工通信，需要设置2个管道。管道在各进程也是互斥的访问。管道写满就是写进程阻塞，管道读空则读进程阻塞。管道中数据读出就彻底消失，因此多个进程访问同一管道时，可能发生错乱，解决方案就是允许多个写进程，一个读进程？



3、信号：用于通知进程某一个特定的事件已经发生，进程收到一个信号后就对该信号进行处理。

序号	信号名	信号对应的事件	默认处理
1	SIGHUP	...	...
2	SIGINT	来自键盘的中断（Control-C）	终止
3	SIGQUIT	...	...
...	...	...	...
8	SIGFPE	浮点异常（如：除以0）	终止并转储内存
9	SIGKILL	杀死进程（如：父进程杀子进程）	终止
...	...	...	...
28	SIGWINCH	窗口大小变化	忽略
...	...	...	...
30	SIGPWR	...	...

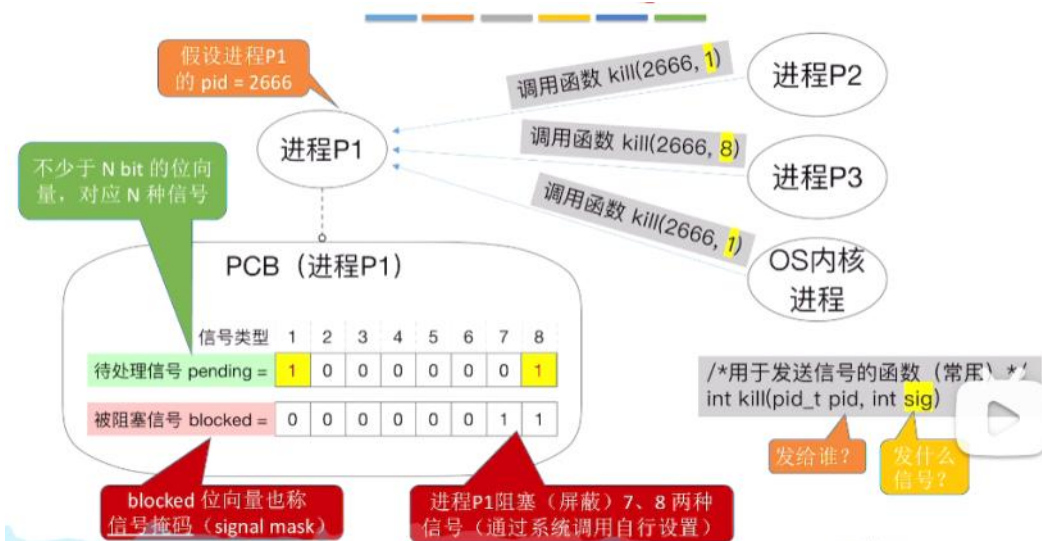
以上是linux定义的信号表，通常用信号名来表示序号，例如#define SIGFPE 8 当触发除以0的情况时，会向正在运行它的进程发送信号8，进程收到后就默认处理终止并转储内存，终止当前进程并将除以0的数据存到内存中。

信号的保存：信号保存于进程的PCB中，包含待处理信号和被阻塞信号，待处理信号就是正在等待进程处理的信号，被阻塞信号就是被进程屏蔽掉的信号（可以通过系统调用自行设置）。



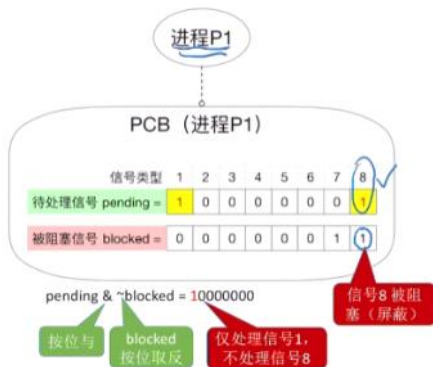
如上图，进程P1没有正在等待被处理的信号，但是信号7和信号8被屏蔽了。

信号的发送：信号的发送通过系统调用函数来实现，用户进程可以发送信号，操作系统和进程自身也可以发送信号。如下图进程P2和P3和Q5分别调用kill函数来向进程P1发送信号8和信号1，那么进程P1接收到信号后把信号1和信号8设置为1，准备进行处理。



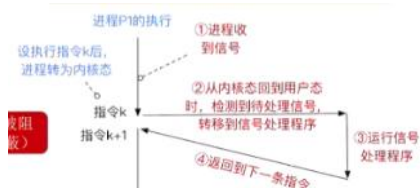
如上图：进程P1接收到2次信号1，会将第二次接收到的信号1扔掉。

什么时候处理信号：当进程从内核态转为用户态时，进程会例行检查有没有待处理的信号，先将被阻塞序号取反（1变成0）然后将待处理信号与被阻塞信号进行与运算，来最终处理结果为1的信号。



如上图，进程P1在经过运算后，最终处理信号1。

信号是怎么处理的：每个进程都会为每一个信号设置一个默认信号处理的程序（有些没有设置默认忽略，有些是用户自己设置怎么处理，用户设置的处理程序只作用于自身程序），在检测到信号后就执行该程序，在执行完后将待处理信号设置为0，然后执行程序原本要执行的下一条语句。以下是执行的具体过程。信号处理的优先级：序号小的优先。



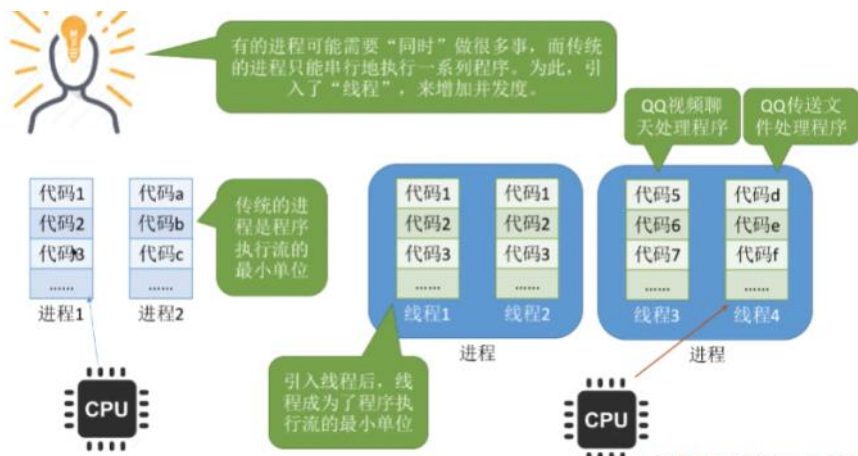
信号与异常的关系：信号可以作为异常的配套机制，让进程对操作系统的异常进行补充。

例如写了个计算器程序，当有个大聪明除以0后，系统异常的默认处理就是终止程序闪退了，这样的体验不是很好，开发者可以自定义SIGFPE信号的处理程序，不让他终止程序而是显示一个“笨蛋”。

七、线程

1、线程可以理解成一个轻量级的进程，以前进程是CPU的基本调度单位，现在引入线程后，线程成为了CPU的基本调度单位和程序执行流的最小单位，一个进程中可以包含多个线程。进程是资源分配的最小单位。

引入了线程后不仅进程之间可以并发的执行，进程里的线程之间也可以并发的执行，进一步的提升了系统的并发性，使得一个进程内也可以并发的处理各种任务（如qq的聊天和传文件并发执行）。也可以降低系统开销，因为CPU并发执行时只来回切换小的线程，大的进程不需要切换，减少了系统开销。



2、线程的属性

是CPU调度的基本单位，也是最小单位。

多CPU的计算机上各个线程可以在不同的CPU

每个线程有线程控制块TCB，有一个线程ID，和进程一样

线程和进程一样，有就绪、阻塞、运行等状态

线程几乎不拥有资源，资源都在上面的进程里面，同一个进程里的所有线程都能共享进程里的资源

同一进程的不同线程之间的通信不受限制，无需系统干预

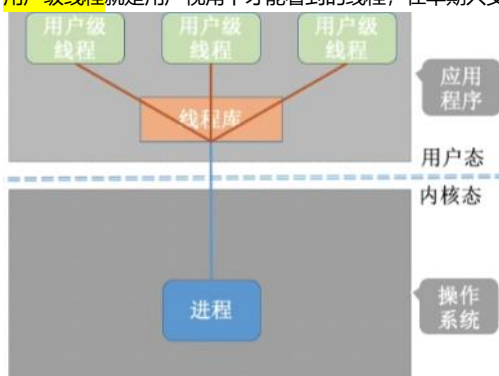
同一进程的线程切换不会引起进程切换，不同进程的线程切换会引起进程切换

切换同进程内的线程，系统开销很小，切换进程的系统开销很大

3、线程的实现方式

线程的实现分为2种，有用户级线程和内核级线程

用户级线程就是用户视角下才能看到的线程，在早期只支持进程的操作系统中创立，是由线程库实现的。



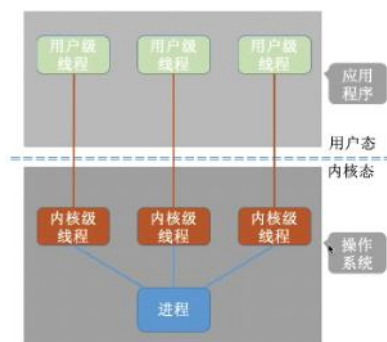
从代码角度上来看，线程就是一段代码逻辑，下图中的3个if就可以在逻辑上看成最简单的线程，由上到下顺序执行。这个时候while就成为了最简单的线程库，线程库的作用就是完成对线程的管理和调度。当然很多编程语言也提供了很多强大的线程库，可以实现线程的创建、销毁、调度等功能。

```
int main() {
    int i = 0;
    while (true) {
        if (i==0){处理视频聊天的代码;}
        if (i==1){处理文字聊天的代码;}
        if (i==2){处理文件传输的代码;}
        i = (i+1)%3; //i的值为 0,1,2,0,1,2...
    }
}
```

QQ进程

注意：用户级线程的管理工作是由应用程序本身在用户态管理的，无需切换内核态，与操作系统无关，操作系统也感知不到用户级线程的存在。因此优点就是不用CPU变态，节省很多开销成本，效率比较高；但是缺点就是有一个线程被阻塞，while循环就无法进行下去，其他的线程都会停止，并发度不高且不可以多核CPU上并行。

内核级线程就是操作系统视角也可以看到的线程，现代支持线程的操作系统大多都是内核级线程。

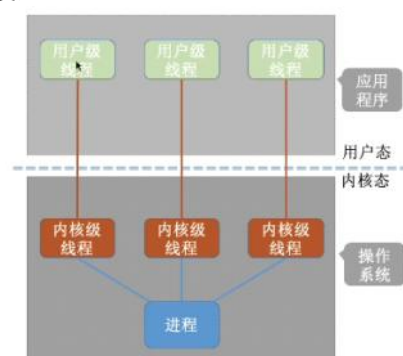


内核级线程的管理工作涉及到操作系统，线程的切换也需要CPU变态。这种实现方式优点是一个线程被阻塞，其他线程还可以继续执行，并发性强；但是一个进程有可能对应多个线程，在进行线程切换的时候涉及到CPU变态，系统的开销成本大。

注意：操作系统只能看见内核级线程，内核级线程才是处理机分配的基本单位

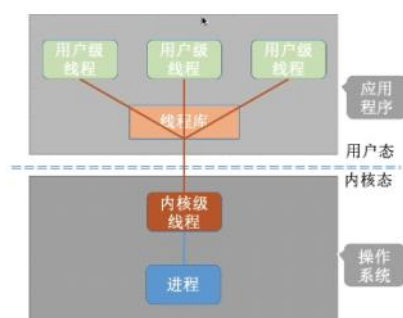
4、多线程模型

当用户级线程和内核级线程相结合的时候，就产生了多线程模型，能结合两者的优点。多线程模型包括：**一对一、多对一、多对多**。两个线程可以这么理解：用户级线程就是代码的载体，例如一个用户线程对应视频聊天另一个线程对应文件传输；内核级线程就是运行机会的载体，操作系统对每一个内核级线程进行处理机调度。



一对一模型：一个用户级线程映射到一个内核级线程。每个用户进程有与用户级线程同数量的内核级线程。

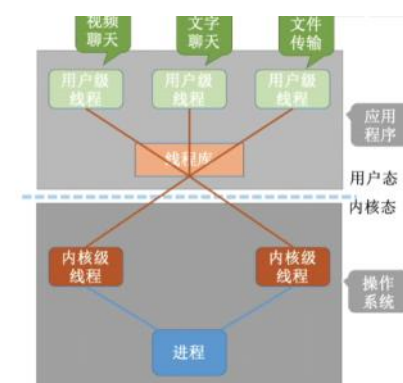
优点：当一个线程被阻塞后，别的线程还可以继续执行，并发能力强。多线程可在多核处理机上并行执行。



多对一模型：多个用户级线程映射到一个内核级线程。且一个进程只被分配一个内核级线程。

优点：用户级线程的切换在用户空间即可完成，不需要切换到核心态，线程管理的系统开销小，效率高

缺点：当一个用户级线程被阻塞后，整个进程都会被阻塞，并发度不高。多个线程不可在多核处理机上并行运行



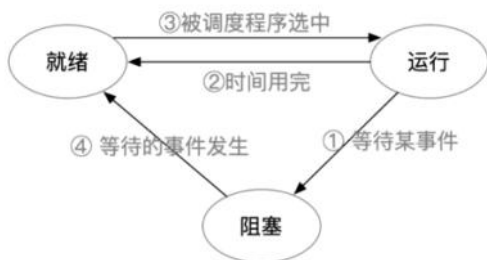
多对多模型：n 用户及线程映射到 m 个内核级线程 ($n \geq m$)。每个用户进程对应 m 个内核级线程。

克服了对一模型并发度不高的缺点（一个阻塞全体阻塞），又克服了一对一模型中一个用户进程占用太多内核级线程，开销太大的缺点。

注意：一个用户级线程对应多个内核级线程，当所有内核级线程被阻塞后，这个用户级线程才算被阻塞了。

5、线程的状态与转换

线程的状态与转换与进程完全一样，有就绪、运行、阻塞3种基本状态。



6、线程的组织与控制

线程的组织和控制与进程的PCB也完全一样

八、调度

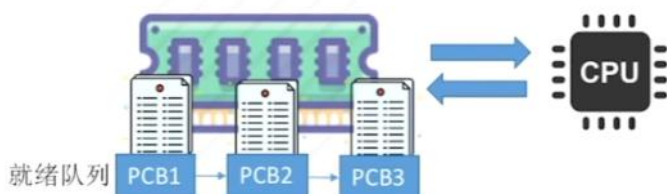
当有一堆东西要处理的时候，由于资源有限，这些东西无法同时处理，就需要调度来**确定某种规则来产生一个先后顺序**。

1、调度的三个层次：

高级调度（作业调度）：按照规则**从外存的后备队列里挑一个调入内存**，并创建进程。每一个作业只调入一次调出一次，调入创建PCB，调出撤销PCB。



低级调度（进程调度/处理机调度）：按照规则从**就绪队列里选一个进程放入CPU**。是最基本的一种调度，不可缺少。并且调度的频率很高，几十毫秒一次。



中级调度：当内存不够时，会将有些进程调出内存放到外存，等内存有空间的时候再放回来，暂存外存的进程状态是**挂起状态**，挂起的PCB会被组织成挂起队列。因此中级调度就是**决定哪个挂起队列的进程先回到内存**。

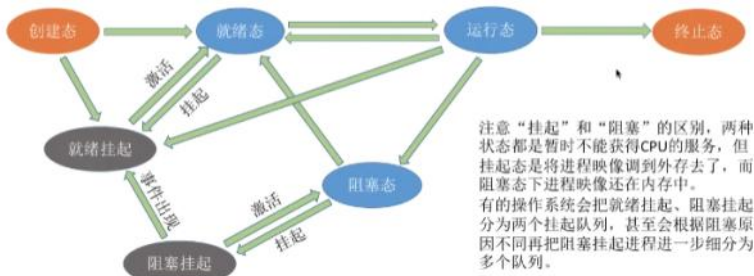


三种调度的对比：

	要做什么	调度发生在..	发生频率	对进程状态的影响
高级调度 (作业调度)	按照某种规则，从后备队列中选择合适的作业将其调入内存，并为其创建进程	外存→内存 (面向作业)	最低	无→创建态→就绪态
中级调度 (内存调度)	按照某种规则，从挂起队列中选择合适的进程将其数据调回内存	外存→内存 (面向进程)	中等	挂起态→就绪态 (阻塞挂起→阻塞态)
低级调度 (进程调度)	按照某种规则，从就绪队列中选择一个进程为其分配处理机	内存→CPU	最高	就绪态→运行态

2、挂起状态与七状态模型

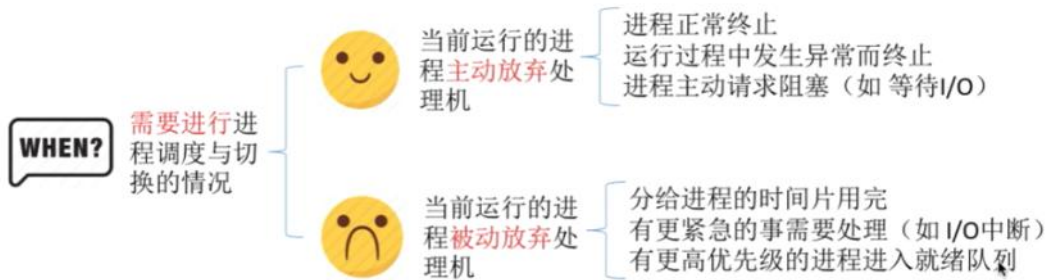
进程的挂起态又可以进一步分为就绪挂起和阻塞挂起，当进程处于创建态、就绪态、阻塞态时，由于内存不够，这两种状态的进程就有可能被放到外存成为挂起态。



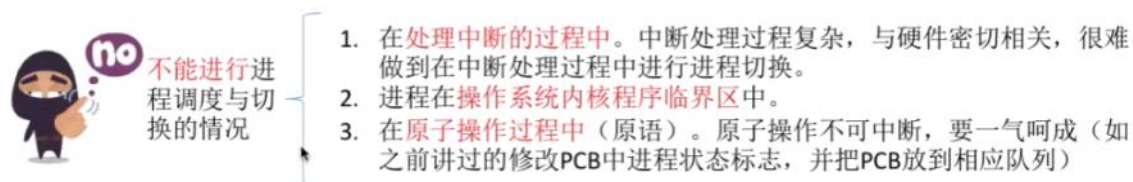
九、进程调度（低级调度）

进程调度就是将就绪队列里的进程选择一个放入CPU。

1、需要进程调度的情况



2、不能进行进程调度的情况



注意：进程在操作系统内核临界区是不能调度的，不尽快释放的话可能会影响到操作系统内核的其他管理工作。但是普通临界区资源不会影响到操作系统的管理工作（如打印机），是可以进行调度与切换的。

3、进程调度的方式

进程调度有抢占式和非抢占式（又称剥夺式和非剥夺式）两种方式，**抢占式**是当进程在CPU内执行时来了个优先级更高的进程进入就绪队列，就把当前CPU内的进程踢回就绪队列；**非抢占式**就是即便有优先级高的进入就绪队列，当前进程依旧在CPU内，直到该进程终止或者主动进入阻塞态。

抢占式实现简单，系统开销小但是无法及时处理紧急任务；非抢占式可以优先处理更紧急的进程，适合分时实时这些现代操作系统。

4、进程调度与进程切换的区别

进程切换就是一个进程让出CPU，另一个进程占用CPU的过程。

进程调度分为狭义的进程调度和广义的进程调度，狭义的进程调度就是就绪队列选一个要运行的进程，如果选择的是另一个进程，就需要进行进程切换；广义的进程调度包含选择进程和进程切换。

5、进程切换的过程

先对原来进程的各种数据进行保存，后对新的进程各种数据进行恢复（如PCB里面的程序计数器、状态字、各种数据寄存器等处理机现场信息），因此**进程切换是有代价的**，如果频繁的进行进程切换，必然降低系统效率和并发度。

6、调度器（调度程序）

调度程序说白了，就是决定让谁运行（调度算法），运行多长的时间（时间片大小），当创建新进程、进程退出、运行进程阻塞、IO中断发生等事件发生时，会触发调度器来进行进程调度。除此之外，非抢占式调度器就只有运行的程序阻塞或者退出的时候才触发；抢占式调度器当每个时钟中断或K个时钟中断才会触发调度程序工作。

7、闲逛进程（idle）

当就绪队列没有其他进程的时候就会运行闲逛进程，类似于人没事干就发呆闲逛。该进程优先级最低，可以是0地址指令（不需要访问CPU的任何寄存器），占用一个完整的指令周期，能耗也低。

十、调度算法的评价指标

1、CPU利用率

由于CPU造价昂贵，人们希望让CPU尽可能更多的工作，因此**CPU利用率就是指CPU忙碌时间占用的比例**。

$$\text{利用率} = \frac{\text{忙碌的时间}}{\text{总时间}}$$

有的题目还会要求计算某种设备的利用率

Eg: 某计算机只支持单道程序, 某个作业刚开始需要在CPU上运行5秒, 再用打印机打印输出5秒, 之后再执行5秒, 才能结束。在此过程中, CPU利用率、打印机利用率分别是多少?

$$\text{CPU利用率} = \frac{5+5}{5+5+5} = 66.66\%$$

$$\text{打印机利用率} = \frac{5}{15} = 33.33\%$$

通常会考察多道程序并发执行的情况, 可以用“甘特图”来辅助计算

2、系统吞吐量

对于计算机来说, 希望用最小的时间处理完最多的作业, 因此**系统吞吐量表示平均每秒完成多少作业**。

$$\text{系统吞吐量} = \frac{\text{总共完成了多少道作业}}{\text{总共花了多少时间}}$$

$$\text{系统吞吐量} = \frac{\text{总共完成了多少道作业}}{\text{总共花了多少时间}}$$

Eg: 某计算机系统处理完10道作业, 共花费100秒, 则系统吞吐量为?

$$10/100 = 0.1 \text{ 道/秒}$$

3、周转时间

对于计算机来说, 很关心自己的**作业从提交到完成发生了多少时间**。

$$(\text{作业}) \text{ 周转时间} = \text{作业完成时间} - \text{作业提交时间}$$

对于用户来说, 更关心自己的单个作业的周转时间

$$\text{平均周转时间} = \frac{\text{各作业周转时间之和}}{\text{作业数}}$$

对于操作系统来说, 更关心系统的整体表现, 因此更关心所有作业周转时间的平均值

$$\text{带权周转时间} = \frac{\text{作业周转时间}}{\text{作业实际运行的时间}} = \frac{\text{作业完成时间} - \text{作业提交时间}}{\text{作业实际运行的时间}}$$

带权周转时间必然 ≥ 1

$$\text{平均带权周转时间} = \frac{\text{各作业带权周转时间之和}}{\text{作业数}}$$

带权周转时间与周转时间都是越小越好



听我说

对于周转时间相同的两个作业, 实际运行时间长的作业在相同时间内被服务的时间更多, 带权周转时间更小, 用户满意度更高。

对于实际运行时间相同的两个作业, 周转时间短的带权周转时间更小, 用户满意度更高。



4、等待时间

等待时间就是作业处于**等待CPU的时间之和**, 等待时间越长用户体验越糟糕, 注意该进程在等待IO的时间不计入等待时间。平均等待时间用来衡量系统进程整体的等待时间。

5、响应时间

用户从提交请求到首次响应的时间称为响应时间, 响应时间越长用户体验越糟糕。

十一、调度算法

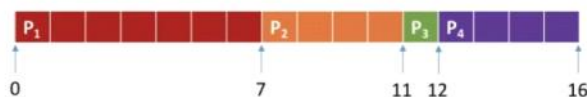
1、先来先服务FCFS

核心思想就是从就绪队列中**选择等待时间最长的作业放入CPU**。

例题: 各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**先来先服务**调度算法, 计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

先来先服务调度算法: 按照到达的先后顺序调度, 事实上就是等待时间越久的越优先得到服务。
因此, **调度顺序**为: P1 → P2 → P3 → P4



$$\text{周转时间} = \text{完成时间} - \text{到达时间}$$

$$P1=7-0=7; P2=11-2=9; P3=12-4=8; P4=16-5=11$$

$$\text{带权周转时间} = \text{周转时间} / \text{运行时间}$$

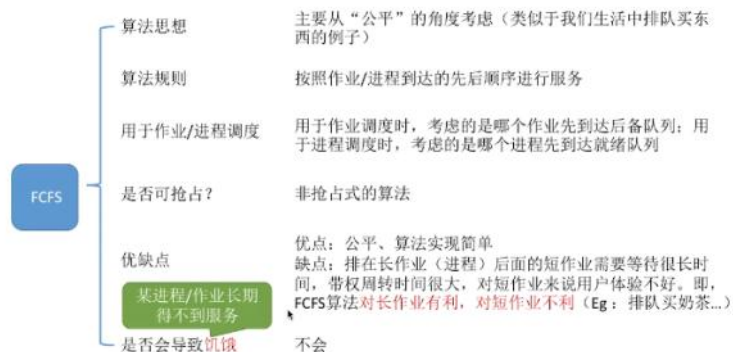
$$P1=7/7=1; P2=9/4=2.25; P3=8/1=8; P4=11/4=2.75$$

$$\text{等待时间} = \text{周转时间} - \text{运行时间}$$

$$P1=7-7=0; P2=9-4=5; P3=8-1=7; P4=11-4=7$$



注意: 上面例子只是理想的进程, 进程到达后要么在等待要么在运行, 如果又有计算又有IO操作, 那么**等待时间就是周转时间-运行时间-IO操作时间**。



2、短作业优先SJF和SPF

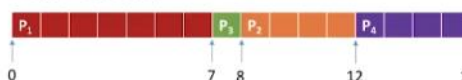
核心思想就是从就绪队列中选择运行时间最短的那一个。

非抢占式短作业优先算法:

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用非抢占式的短作业优先调度算法，计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

短作业/进程优先调度算法：每次调度时选择当前已到达且运行时间最短的作业/进程。
因此，调度顺序为：P1 → P3 → P2 → P4



周转时间 = 完成时间 - 到达时间

P1=7-0=7; P3=8-4=4; P2=12-2=10; P4=16-5=11

带权周转时间 = 周转时间/运行时间

P1=7/7=1; P3=4/1=4; P2=10/4=2.5; P4=11/4=2.75

等待时间 = 周转时间 - 运行时间

P1=7-7=0; P3=4-1=3; P2=10-4=6; P4=11-4=7

平均周转时间 = (7+4+10+11)/4 = 8

平均带权周转时间 = (1+4+2.5+2.75)/4 = 2.56

平均等待时间 = (0+3+6+7)/4 = 4

8.75
3.5
4.75

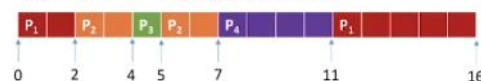
对比FCFS算法的结果，显然SPF算法的平均等待/周转/带权周转时间都要更低。

抢占式短作业优先算法（最短剩余时间优先算法）SRTN:

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用抢占式的短作业优先调度算法，计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

最短剩余时间优先算法：每当有进程加入就绪队列改变时就需要调度，如果新到达的进程剩余时间比当前运行的进程剩余时间更短，则由新进程抢占处理器，当前运行进程重新回到就绪队列。另外，当一个进程完成时也需要调度。



周转时间 = 完成时间 - 到达时间

P1=16-0=16; P2=7-2=5; P3=5-4=1; P4=11-5=6

带权周转时间 = 周转时间/运行时间

P1=16/7=2.28; P2=5/4=1.25; P3=1/1=1; P4=6/4=1.5

等待时间 = 周转时间 - 运行时间

P1=16-7=9; P2=5-4=1; P3=1-1=0; P4=6-4=2

平均周转时间 = (16+5+1+6)/4 = 7

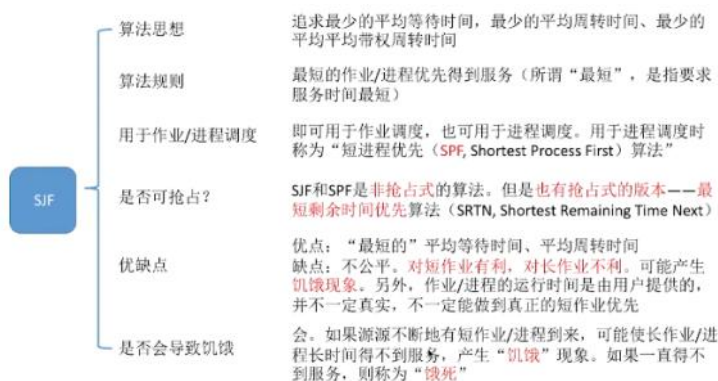
平均带权周转时间 = (2.28+1.25+1+1.5)/4 = 1.50

平均等待时间 = (9+1+0+2)/4 = 3

8
2.56
4

对比非抢占式的短作业优先算法，显然抢占式的这几个指标又要更低。

注意：如果没有提到抢占式，那就是默认非抢占式



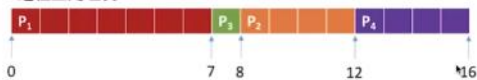
3、高响应比优先HRRN

前两个算法分别对短作业和长作业不友好，高响应比就总和考虑了作业的等待时间和运行时间，等待时间越长的优先，运行时间越短的优先。因此出现了响应比这个指标。响应比=1+等待时间/运行时间

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**高响应比优先**调度算法，计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

高响应比优先算法：非抢占式的调度算法，只有当前运行的进程**主动放弃CPU**时（正常/异常完成，或主动阻塞），才需要进行调度，调度时**计算所有就绪进程的响应比**，**选响应比最高的进程**上处理机。



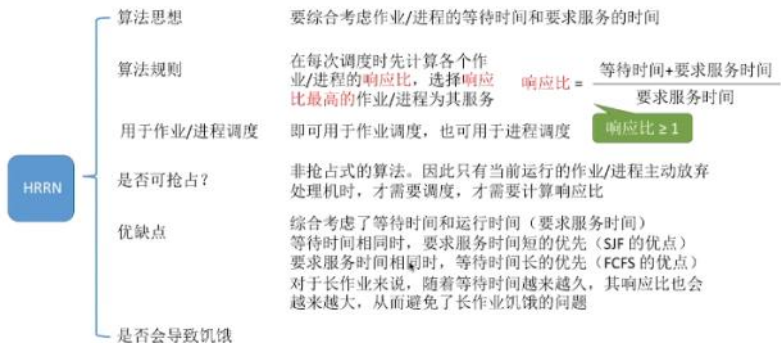
$$\text{响应比} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$$

0时刻：只有 P₁ 到达就绪队列，P₁ 上处理机

7时刻（P₁主动放弃CPU）：就绪队列中有 P₂（响应比=(5+4)/4=2.25）、P₃((3+1)/1=4)、P₄((2+4)/4=1.5)。

8时刻（P₃完成）：P₂(2.5)、P₄(1.75)

12时刻（P₂完成）：就绪队列中只剩下 P₄



4、三者对比

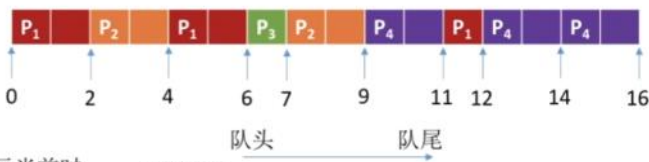
算法	思想&规则	可抢占?	优点	缺点	考虑到等待时间&运行时间?	会导致饥饿?
FCFS	自己回忆	非抢占式	公平；实现简单	对短作业不利	等待时间√ 运行时间×	不会
SJF/S PF	自己回忆	默认为非抢占式，也有SJF的抢占式版本最短剩余时间优先算法（SRTN）	“最短的”平均等待/周转时间；	对长作业不利，可能导致饥饿；难以做到真正的短作业优先	等待时间× 运行时间√	会
HRRN	自己回忆	非抢占式	上述两种算法的权衡折中，综合考虑的等待时间和运行时间		等待时间√ 运行时间√	不会

5、时间片轮转调度算法RR

该算法伴随着分时操作系统的诞生而产生，就是**公平的轮流**的为**每一个进程分配一个时间片**，进程在时间片规定的范围内执行，超过时间片规定时长就剥夺处理机然后下一个。是**抢占式**算法，利用**时钟中断**来实现。

进程	到达时间	运行时间
P1	0	5
P2	2	4
P3	4	1
P4	5	6

时间片轮转调度算法：轮流让就绪队列中的进程依次执行一个时间片（每次选择的都是排在就绪队列队头的进程）



时间片大小为 2（注：以下括号内表示当前时刻就绪队列中的进程、进程的剩余运行时间）

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**时间片轮转**调度算法，分析时间片大小分别是2、5时的进程运行情况。

进程	到达时间	运行时间
P1	0	5
P2	2	4
P3	4	1
P4	5	6

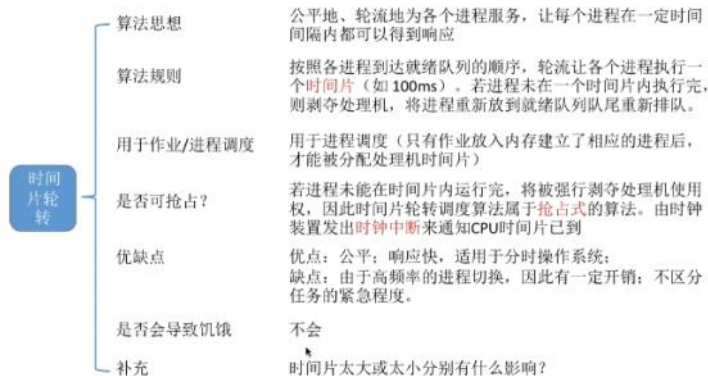
时间片轮转调度算法：轮流让就绪队列中的进程依次执行一个时间片（每次选择的都是排在就绪队列队头的进程）



时间片大小为 5

有3种情况：时间片用完就T出CPU；时间片没用完但是作业完成，就主动退出CPU；时间片用完但是就绪队列没有其他进程，就继续执行下一个时间片。

如果时间片过大就会变成先来先服务了，会增大响应时间，因此时间片不能太大。如果时间片太小又会造成切换过于频繁，让切换的时间占用大部分CPU的运行，从而挤占实际进程执行的时间，因此也不能太小。



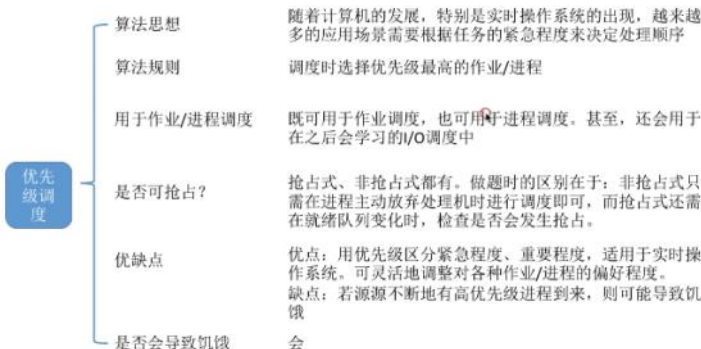
6、优先级调度算法

随着实时操作系统的出现，越来越多的应用场景需要根据任务的紧急程度来决定处理顺序。该算法就是每个作业**自带优先级**，调度时**选择优先级最高的作业**放入CPU。该算法也分为抢占式和非抢占式。



注意就绪队列未必只有1个，可以按照不同的优先级来组织，另外可以使用优先队列把优先级高的进程放在对头的位置。根据优先级是否可以改变也可以将其分为静态优先级和动态优先级两种，静态优先级是创建进程的时候确定的，运行过程中不会改变，而动态优先级相反。

通常系统进程的优先级大于用户进程、前台高于后台、偏好IO级进程（可以让IO更早的投入工作）、不偏好计算型进程（CPU繁忙型进程）。



7、多级反馈队列调度算法

上面的算法都有各自的优缺点，综合以上所有优点，集百家之长，就产生了多级反馈队列调度算法。进程运行的过程。



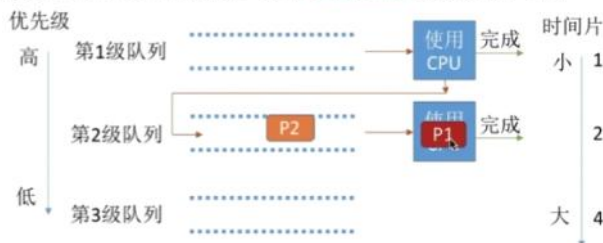
例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**多级反馈队列**调度算法，分析进程运行的过程。



例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

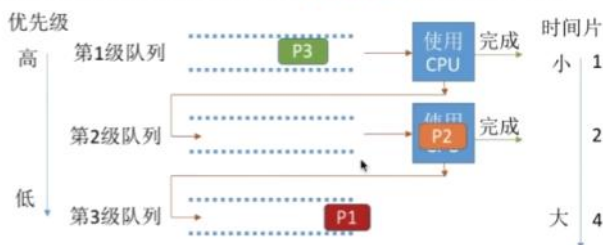
P1(1) → P2(1)



例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

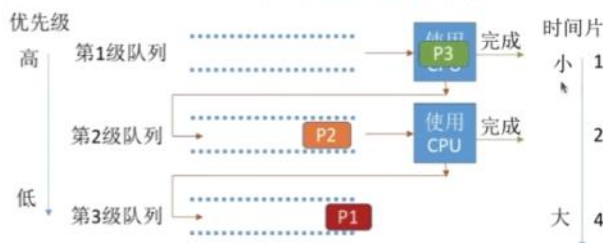
P1(1) → P2(1) → P1(2)
→ P2(1)



例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

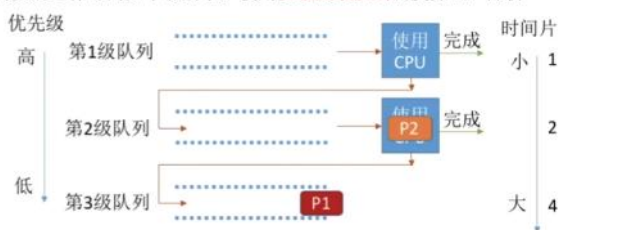
P1(1) → P2(1) → P1(2)
→ P2(1)



例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

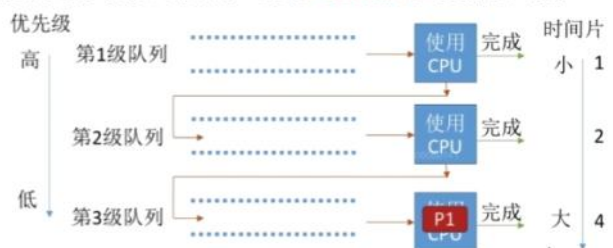
P1(1) → P2(1) → P1(2)
→ P2(1) → P3(1)



例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

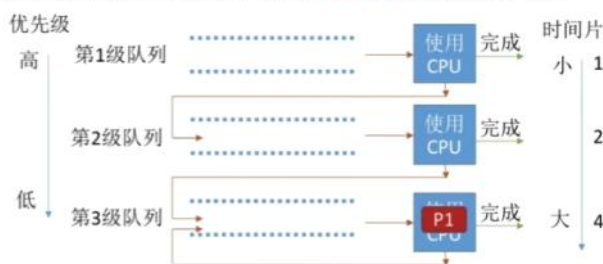
P1(1) → P2(1) → P1(2)
→ P2(1) → P3(1) → P2(2)



例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

P1(1) → P2(1) → P1(2)
→ P2(1) → P3(1) → P2(2)
→ P1(4)



如下图设置多级就绪队列，各队列优先级从高到低，时间片从小到大。

新进程到达时先进入第一级队列，按照先进先出的原则等待时间片，如果用完时间片还没有结束，就把作业放到下一级队列队尾，如果已经在最下级的队列了，就重新放到该队列队尾；只有上一级队列为空的时候，才会为下一级的队列分配时间片；如果进程运行中被抢占，则把进程放到原队列的队尾。

注意：如果有进程掉到了最低级队列，并且有源源不断的新队列，就可能会产生饥饿的情况。

多级反馈队列	算法思想	对其他调度算法的折中权衡				
	算法规则	1. 设置多级就绪队列，各级队列优先级从高到低，时间片从小到大 2. 新进程到达时先进入第1级队列。按FCFS原则排队等待被分配时间片，若用完时间片进程还未结束，则进程进入下一级队列队尾。如果此时已经是在最下级的队列，则重新放回该队列队尾 3. 只有第k级队列为空时，才会为k+1级队头的进程分配时间片				
	用于作业/进程调度	用于进程调度				
	是否可抢占?	抢占式的算法。在k级队列的进程运行过程中，若更上级的队列（1~k-1级）中进入了一个新进程，则由于新进程处于优先级更高的队列中，因此新进程会抢占处理机，原来运行的进程放回k级队列队尾。				
	优缺点	对各类型进程相对公平（FCFS的优点）；每个新到达的进程都可以很快得到响应（RR的优点）；短进程只用较少的时间就可完成（SPF的优点）；不必实现估计进程的运行时间（避免用户作假）；可灵活地调整对各类进程的偏好程度，比如CPU密集型进程、I/O密集型进程（拓展：可以将因I/O而阻塞的进程重新放回原队列，这样I/O型进程就可以保持较高优先级）				
	是否会导致饥饿	会				

算法	思想&规则	可抢占?	优点	缺点	会导致饥饿?	补充
时间片轮转		抢占式	公平，适用于分时系统	频繁切换有开销，不区分优先级	不会	时间片太大或太小有何影响?
优先级调度		有抢占式的，也有非抢占式的。注意做题时的区别	区分优先级，适用于实时系统	可能导致饥饿	会	动态/静态优先级。各类型进程如何设置优先级? 如何调整优先级?
多级反馈队列	较复杂，注意理解	抢占式	平衡优秀666	一般不说它有缺点，不过可能导致饥饿	会	

8、多处理机调度

前面讲的都是单CPU的情况下的调度，只需要决定让哪个就绪进程优先上CPU即可。当面对多CPU时，不仅要考虑这些，还要考虑进程上哪一个CPU。

在多处理机调度下，需要追求以下目标：

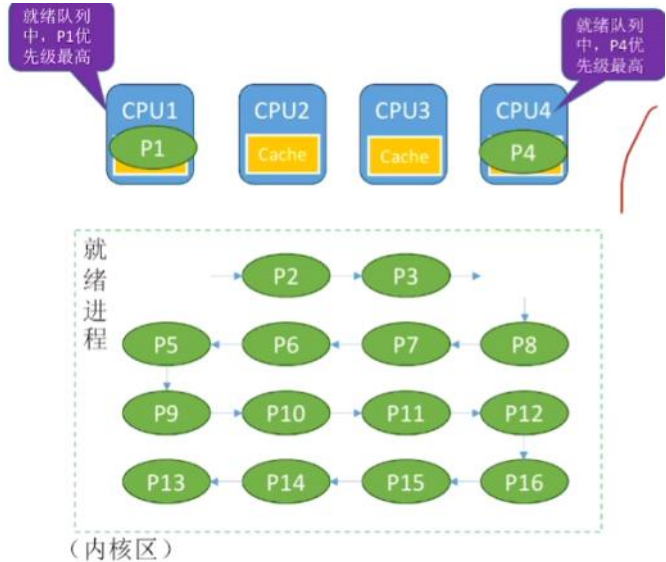
负载均衡——CPU都同等忙碌，不能忙的忙死闲的闲死

处理机亲和性——尽量让一个进程调度到同一个CPU上运行，来更好的发挥CPU中缓存作用

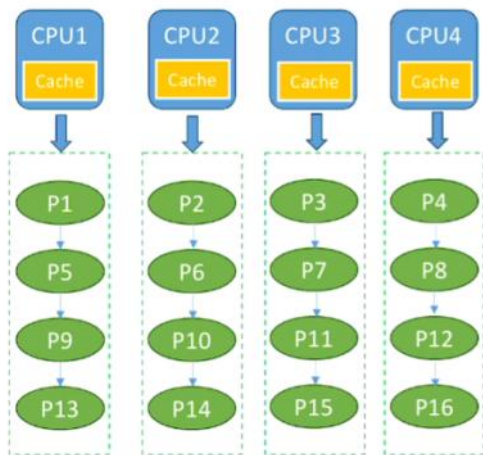
为了实现这两个目标：有2个解决方案

方案一公共就绪队列：设置一个公共的就绪进程队列，每个进程的PCB在队列中，所有CPU都可以访问这个队列（位于内核区），每个CPU运行调度程序，从就绪队列中选择优先级最高的运行，当一个CPU访问就绪队列的时候，就给队列上锁确保互斥性。

这种方案优点是负载均衡，缺点是亲和性不好，为了解决这个问题，有软亲和和硬亲和两种方法，软亲和就是进程调度尽量保证亲和性，硬亲和就是用户进程通过系统调用主动要求操作系统分配固定的CPU。



方案二私有就绪队列：每个CPU都有自己专有的就绪队列，CPU空闲的时候从自己的私有队列中选择一个运行。这样保证了亲和性，但是还需要一些手段来保证负载均衡。



迁移移(Push)策略就是一个特定的系统程序周期性检查每个CPU的负载情况，如果发现负载不平衡，就把忙碌CPU就绪队列的一些进程推到另一个空闲的CPU的就绪队列。

迁移移(Pull)策略：每个CPU运行调度程序的时候会周期性检查自己的负载情况和其他CPU的负载，如果CPU负载很低，就从其他高负载的CPU就绪队列中拉一些进自己的就绪队列。

迁移移类似于一个包工头分配工作，拉迁移类似于一群互帮互助的同事分担任务。

十一：进程的同步和互斥

1、进程同步的概念

众所周知，进程具有异步性的特征。异步就是指各进程以各自独立的、不可预知的速度向前推进。但是有时候需要进程以预制的顺序来，比如管道通信必须按照先“写数据”再“读数据”的顺序来执行的。如何解决异步问题，就是进程同步所讨论的内容。因此**同步是直接的制约关系**，进程因为需要再某些位置上**协调他们的工作次序**而产生的制约关系，制约关系**源于他们之间的相互合作**。

2、进程互斥的概念

操作系统具有共享性，共享方式有**互斥共享**和**同时共享**这两种，**互斥共享**就是一个时间段内只能有一个进程进行资源访问，同时共享是允许一段时间内由多个进程并发的进行访问。我们把一段时间内只允许一个进程使用的系统资源称为**临界资源**（例如打印机、摄像头等），对于临界资源的访问，必须互斥的进行。**进程互斥**就是指当一个进程访问某临界资源时，另一个想要访问的进程必须等待，等上一个访问结束后才能访问。

3、进程互斥的实现方式



如上图：**临界区就是访问临界资源的代码段**，进入区和退出区是实现进程互斥的代码段，临界区可以成为临界段。

为了实现对临界资源的互斥访问同时保证系统整体性能，需要遵循以下4个原则：

空闲让进：临界区空闲就进。

忙则等待：临界区已有进程，其他试图进入临界区的进程必须等待。

有限等待：对于请求访问的进程，应保证能在有限时间内进入临界区（不饥饿）

让权等待：进程不能进入临界区，应当立即释放CPU，防止忙等

4、进程互斥的软件实现方法

单标志法：两个进程在访问完临界区资源后会使用临界区的权限转交给另一个进程，每个进程进入临界区的**权限只能来源于另一个进程授予**。

`int turn = 0; //turn 表示当前允许进入临界区的进程号`

P0 进程:		P1进程:	
<code>while (turn != 0);</code>	①	<code>while (turn != 1);</code>	⑤ //进入区
<code>critical section;</code>	②	<code>critical section;</code>	⑥ //临界区
<code>turn = 1;</code>	③	<code>turn = 0;</code>	⑦ //退出区
<code>remainder section;</code>	④	<code>remainder section;</code>	⑧ //剩余区

如上图：turn初始值是0，如果p1先进入，就会在代码5这里一直转圈，直到给的时间片用完（有限等待）让p0进入，p0进入时不满足转圈条件就进去访问临界资源了，当访问完的时候将turn值设置为1，把访问权限授予P1。



但是这种算法存在一个问题，当进入临界区的P0一直不访问资源但又占着临界区资源，就会卡住P1，违背了“空闲让进”的原则。

双标志先检查法： 设置一个布尔型数组用来表示每个进程想进入邻接区的意愿，flag[0]=false表示进程P0不想进入临界区，flag[1]=true表示P1进程想进入临界区。每个进程在进入临界区之前先检查当前有没有别的进程想进入临界区，有的话就谦让给另一个进程，没有的话就把自己的flag[i]设置为true，然后开始访问。

```
bool flag[2]; //表示进入临界区意愿的数组 理解背后的含义：“表达意愿”
flag[0] = false; //刚开始设置为两个进程都不想进入临界区
flag[1] = false;

P0 进程:
while (flag[1]); ①
flag[0] = true; ②
critical section; ③
flag[0] = false; ④
remainder section;

P1 进程:
while (flag[0]); ⑤
flag[1] = true; ⑥
critical section; ⑦
flag[1] = false; ⑧
remainder section;
```

如上图，最初2个进程都是false，那就先来的先访问资源。当P1进程先进来但是P0的flag是true，就一直转圈等时间片用完然后让P0访问。



但是有个问题，这代码不是原语，检查和上锁之间没有一气呵成的特性，如果按照152637访问的话P0和P1会同时访问临界区，违反了“忙则等待”的原则。

双标志后检查法： 前面那个是先检查后上锁，但是这两个操作无法一气呵成，又导致了2个进程同时进入临界区的问题，因此产生了先上锁后检查的方法来解决。

```
bool flag[2]; //表示进入临界区意愿的数组 理解背后的含义：“表达意愿”
flag[0] = false; //刚开始设置为两个进程都不想进入临界区
flag[1] = false;

P0 进程:
flag[0] = true; ①
while (flag[1]); ②
critical section; ③
flag[0] = false; ④
remainder section;

P1 进程:
flag[1] = true; ⑤
while (flag[0]); ⑥
critical section; ⑦
flag[1] = false; ⑧
remainder section;
```

但这个更是依托，如果按照1526这个顺序，P0和P1都进不了临界区，两人互相谦让最后都憋死了。因此这种检查法虽然解决“忙则等待”的原则，又违背了“空闲让进”和“有限等待”的原则。

Peterson算法： 经典折中算法，结合双标志和单标志的思想，如果双方都争着进入临界区，就尝试着谦让，做一个有礼貌的进程。

```
bool flag[2]; //表示进入临界区意愿的数组，初始值都是false 背后的含义：“表达意愿”
int turn = 0; //turn 表示优先让哪个进程进入临界区 表达“谦让”

P0 进程:
flag[0] = true; ①
turn = 1; ②
while (flag[1] && turn==1); ③
critical section; ④
flag[0] = false; ⑤
remainder section;

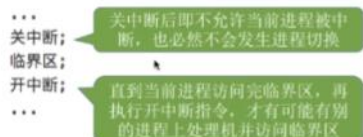
P1 进程:
flag[1] = true; ⑥
turn = 0; ⑦
while (flag[0] && turn==0); ⑧
critical section; ⑨
flag[1] = false; ⑩
remainder section;
```

如上图，设置flag表示该进程想用，turn表示一种谦让，愿意让对方先进入进程。因为最后turn只能是1种状态，谁最后谦让，谁就失去了优先权。满足了空闲让进、忙则等待、有限等待这三个原则，但是这种算法违背了“让权等待”，因为在while转圈的时候是不能及时的释放CPU资源的。

注意：在软件层面上实现进程互斥来来回回都离不开“谦让”和“表达意愿”这2个核心问题。

5、进程互斥的硬件实现方法

中断屏蔽方法： 和原语的实现思想一样，利用开/关中断指令实现。



这种方法优点是简单高效，缺点是不适用于多核CPU，只适合内核进程，不适合用户进程，因为开关中断指令在内核态运行，用户随意使用的话很危险。

TestAndSet指令： 简称TS指令或者TSL指令，

简称 TS 指令，也有地方称为 TestAndSetLock 指令，或 TSL 指令
TSL 指令是用硬件实现的，执行的过程不允许被中断，只能一气呵成。以下是用C语言描述的逻辑

```
// 布尔型共享变量 lock 表示当前临界区是否被加锁
// true 表示已加锁, false 表示未加锁
bool TestAndSet (bool *lock){
    bool old;
    old = *lock; //old用来存放lock 原来的值
    *lock = true; //无论之前是否已加锁, 都将lock设为true
    return old; //返回lock原来的值
}

// 以下是使用 TSL 指令实现互斥的算法逻辑
while (TestAndSet (&lock)); //“上锁”并“检查”
临界区代码段...
lock = false; //“解锁”
剩余区代码段...
```

若刚开始 lock 是 false，则 TSL 返回的 old 值为 false，while 循环条件不满足，直接跳过循环，进入临界区。若刚开始 lock 是 true，则执行 TSL 后 old 返回的值为 true，while 循环条件满足，会一直循环，直到当前访问临界区的进程在退出区进行“解锁”。
相比软件实现方法，TSL 指令把“上锁”和“检查”操作作用硬件的方式变成了一气呵成的原子操作。
优点：实现简单，无需像软件实现方法那样严格检查是否有逻辑漏洞；适用于多处理机环境
缺点：不满足“让权等待”原则，暂时无法进入临界区的进程会占用CPU并循环执行TSL指令，从而导致“忙等”。

Swap指令：

有的地方也叫 Exchange 指令，或简称 XCHG 指令。
Swap 指令是用硬件实现的，执行的过程不允许被中断，只能一气呵成。以下是用C语言描述的逻辑

```
// Swap 指令的作用是交换两个变量的值
Swap (bool *a, bool *b) {
    bool temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

// 以下是用 Swap 指令实现互斥的算法逻辑
// lock 表示当前临界区是否被加锁
bool old = true;
while (old == true)
    Swap (&lock, &old);
临界区代码段...
lock = false;
剩余区代码段...
```

逻辑上来看 Swap 和 TSL 并无太大区别，都是先记录下此时临界区是否已经被上锁（记录在 old 变量上），再将上锁标记 lock 设置为 true，最后检查 old，如果 old 为 false 则说明之前没有别的进程对临界区上锁，则可跳出循环，进入临界区。
优点：实现简单，无需像软件实现方法那样严格检查是否有逻辑漏洞；适用于多处理机环境
缺点：不满足“让权等待”原则，暂时无法进入临界区的进程会占用CPU并循环执行TSL指令，从而导致“忙等”。

6、互斥锁

互斥锁是解决临界区最简单的工具，一个进程在**进入临界区时应获得锁**，在**退出时释放锁**，这是通过调用函数acquire()和release()实现的。每个互斥锁有一个布尔变量available表示锁是否可以用，false表示已经上锁了不能用，就会在while一直绕圈直到时间片用完，true表示没上锁可以用，进程就会获得锁，直到访问完临界资源调用release释放锁。

互斥锁的**缺点就是忙等**，进程在临界区的时候，其他进程在进入临界区的时候只能转圈，占用CPU资源但是不干活，因此在多个进程共享同一CPU的时候会浪费CPU的周期，**只适合多处理器系统**，这样一个线程在CPU上忙等就不会影响其他线程的执行。优点是等待期间不用切换进程的上下文。

需要连续循环忙等的互斥锁都可以称为自旋锁，如TSL、swap、单标志法等。

十二、信号量

之前的进程互斥解决方案全部都无法实现让权等待并且双标志先检查中“检查”和“上锁”不能一气呵成，从而导致多个进程可能同时进入临界区，因此**提出了信号量机制**。

1、概念：**信号量本质上就是一个表示系统中某种资源数量的变量**，例如系统中只有一台打印机就可以设置一个初值为1的信号量。信号量可以是一个整数也可以是更复杂的形式。原语是执行可以一气呵成不可中断，如果能把进入区和退出区的操作原语实现，让这些操作不可中断就能避免问题。

一对原语:wait(S)原语和signal(S)原语，S表示信号量。其中wait和signal简称为**P操作和V操作**，因此做题可以用P(S)和V(S)表示。

2、整形信号量：用**int型变量作为信号量**，用来表示系统中某种资源的数量。

```
int S = 1; //初始化整形信号量s，表示当前系统中可用的打印机资源数
```

```
void wait (int S) { //wait 原语，相当于“进入区”
    while (S <= 0); //如果资源数不够，就一直循环等待
    S=S-1; //如果资源数够，则占用一个资源
}
```

```
void signal (int S) { //signal 原语，相当于“退出区”
    S=S+1; //使用完资源后，在退出区释放资源
}
```

进程P0:	进程P1:	进程Pn:
...	...	...
wait(S); //进入区, 申请资源	wait(S);	wait(S);
使用打印机资源... //临界区, 访问资源	使用打印机资源...	使用打印机资源...
signal(S); //退出区, 释放资源	signal(S);	signal(S);
...	...	...

如上图，PV原语实现很简单，检查和上锁不可被中断，如果资源量S<=0的时候有一个进程进入临界区，就会一直转圈忙等，还是不满足“让权等待”的原则。

3、记录型信号量

为了解决“让权等待”的问题，人们提出了记录型信号量，就是用记录型的数据结构表示的信号量。

```

/*记录型信号量的定义*/
typedef struct {
    int value;          // 剩余资源数
    struct process *L;  // 等待队列
} semaphore;

/*某进程需要使用资源时, 通过 wait 原语申请*/
void wait (semaphore S) {
    S.value--;
    if (S.value < 0) {
        block (S.L);
    }
}

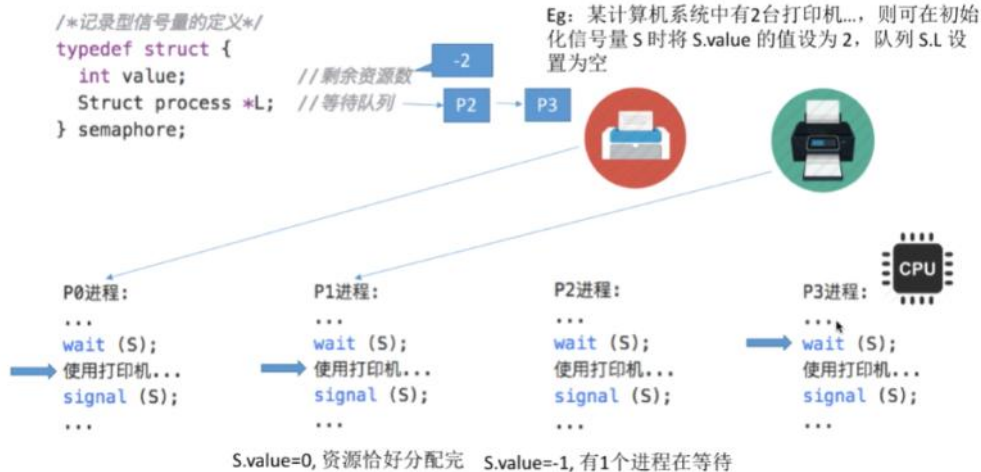
/*进程使用完资源后, 通过 signal 原语释放*/
void signal (semaphore S) {
    S.value++;
    if (S.value <= 0) {
        wakeup (S.L);
    }
}

```

如果剩余资源数不够, 使用block原语使进程从运行态进入阻塞态, 并把挂到信号量S的等待队列(即阻塞队列)中

释放资源后, 若还有别的进程在等待这种资源, 则使用wakeup原语唤醒等待队列中的一个进程, 该进程从阻塞态变为就绪态

如上图, 相比于整形信号量, 记录型多了一个等待队列, 当前资源不够的时候就用block阻塞等待队列, 当访问完临界资源的时候, 如果等待队列被阻塞, 就把它的状态恢复。例子如下图所示:



因为进程在得不到资源的时候会调用block自我进入阻塞态, 不再占用CPU资源, 所以它满足“让权等待”的原则。

4、用信号量机制实现进程的互斥

首先我们需要划定一个临界区, 对于临界资源的访问我们需要放在临界区里, 然后需要设置一个互斥信号量mutex (记录型), value的初值为1代表进入临界区的名额, 然后进程在进入区P(mutex)进行申请资源, 访问完后在退出区V(mutex)进行释放资源。需要注意的是: 不用临界资源要设置不同的互斥信号量; PV操作必须成对出现, 没有P操作就没有了block互斥访问, 没有了V操作就会导致等待的进程永不唤醒。

```

/*信号量机制实现互斥*/
semaphore mutex=1; // 初始化信号量

P1(){
    ...
    P(mutex);      // 使用临界资源前需要加锁
    临界区代码段...
    V(mutex);      // 使用临界资源后需要解锁
    ...
}

P2(){
    ...
    P(mutex);
    临界区代码段...
    V(mutex);
    ...
}

```

5、用信号量机制实现进程的同步

所谓进程的同步就是让进程的执行有一个先后顺序, 利用信号量机制的PV操作是可以实现的:

```

P1(){
    代码1;
    代码2;
    V(S);
    代码3;
}

P2(){
    P(S);
    代码4;
    代码5;
    代码6;
}

```

如上图S的value初始值设置为0, 当P1先执行的时候调用V(S)释放“资源”让value的值设置为1, 然后P2执行的时候调用P(S)会发现没有可用“资源”, 就自我阻塞下处理机, 等P1执行完V(S)后才会继续正常执行下去。因此无论谁先调用, 顺序都是P1 P2.

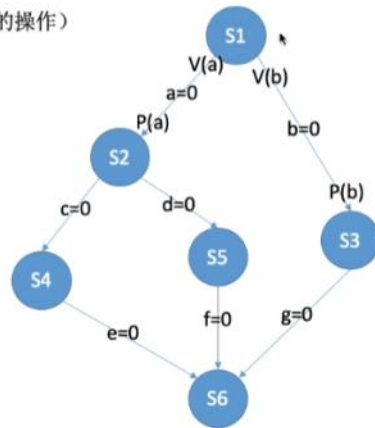
技巧: 在“前操作”后执行V操作, 在“后操作”前执行P操作->前V后P

进程 P1 中有句代码 S1, P2 中有句代码 S2, P3 中有句代码 S3 P6 中有句代码 S6。这些代码要求按如下前驱图所示的顺序来执行:

其实每一对前驱关系都是一个进程同步问题 (需要保证一前一后的操作)
因此,

1. 要为每一对前驱关系各设置一个同步信号量
2. 在“前操作”之后对相应的同步信号量执行 V 操作
3. 在“后操作”之前对相应的同步信号量执行 P 操作

P1() {	P2() {	P3() {	P4() {	P5() {	P6() {
...	...	...	...	...	...
S1;	P(a);	P(b);	P(c);	P(d);	P(e);
V(a);	S2;	S3;	S4;	S5;	P(f);
V(b);	V(c);	V(g);	V(e);	V(f);	P(g);
...	V(d);	...	...	...	S6;
}	...	}	}	}	...
	}				}



6、生产者-消费者问题

问题分析

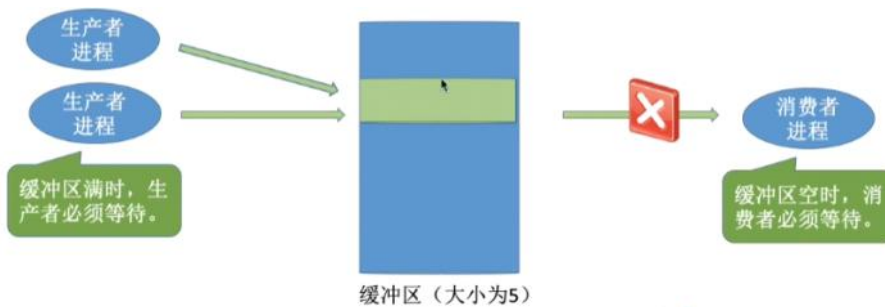
系统中有一组生产者进程和一组消费者进程, 生产者进程每次生产一个产品放入缓冲区, 消费者进程每次从缓冲区中取出一个产品并使用。(注: 这里的“产品”理解为某种数据)

生产者、消费者共享一个初始为空、大小为n的缓冲区。

只有缓冲区没满时, 生产者才能把产品放入缓冲区, 否则必须等待。

只有缓冲区不空时, 消费者才能从中取出产品, 否则必须等待。

缓冲区是临界资源, 各进程必须互斥地访问。

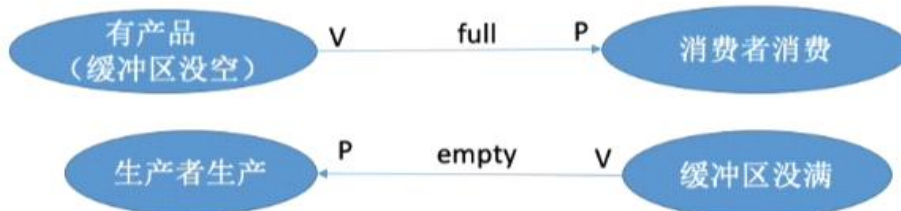


首先我们解决同步的问题:

生产者要先生产出东西来消费者才能消费, 以产品数full为信号量, 根据前V后P可以得出生产者这边V, 消费者这边P;

消费者要消费了东西, 使得空闲缓冲区没满, 生产者才能生产东西, 以空闲数empty为信号量得出生产者P消费者V;

我们还要解决互斥的问题, 设置一个mutex变量来实现互斥访问。



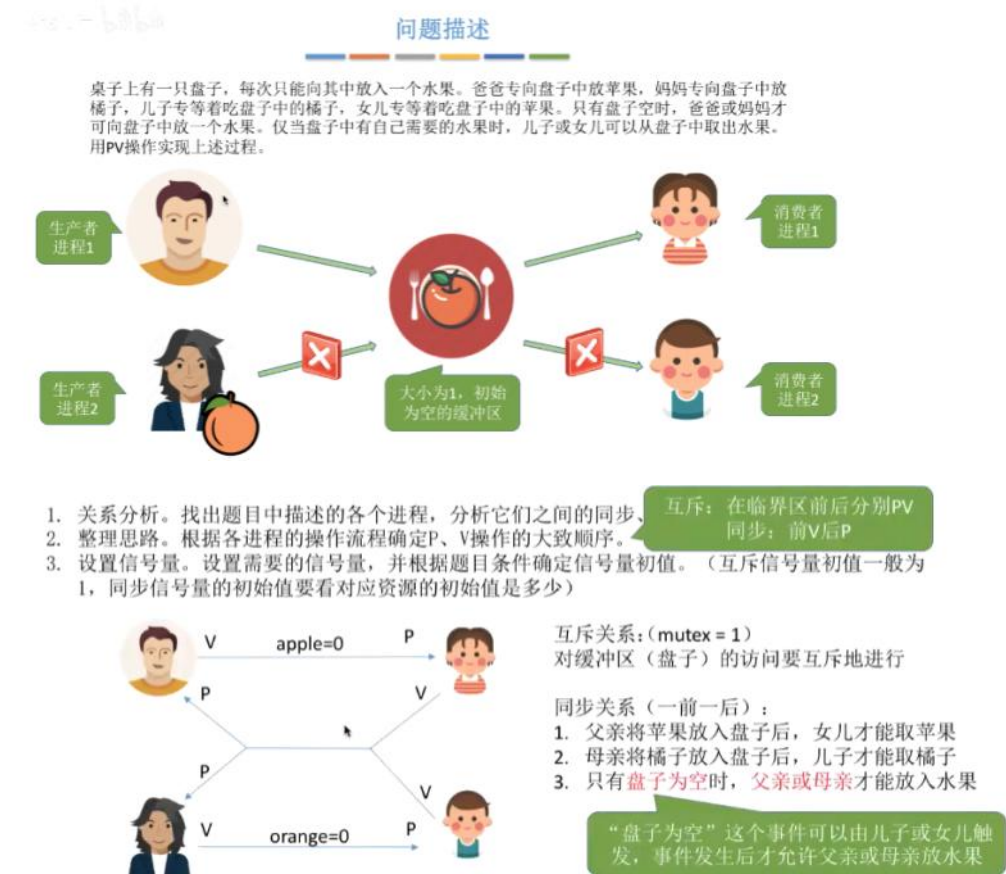
```
semaphore mutex = 1; // 互斥信号量, 实现对缓冲区的互斥访问
semaphore empty = n; // 同步信号量, 表示空闲缓冲区的数量
semaphore full = 0; // 同步信号量, 表示产品的数量, 也即非空缓冲区的数量
```

<pre>producer () { while(1) { 生产一个产品; P(empty); // 消耗一个空闲缓冲区 P(mutex); 把产品放入缓冲区; V(mutex); V(full); // 增加一个产品 } }</pre>	<pre>consumer () { while(1) { P(full); // 消耗一个产品 (非空缓冲区) P(mutex); 从缓冲区取出一个产品; V(mutex); V(empty); // 增加一个空闲缓冲区 使用产品; } }</pre>
---	---

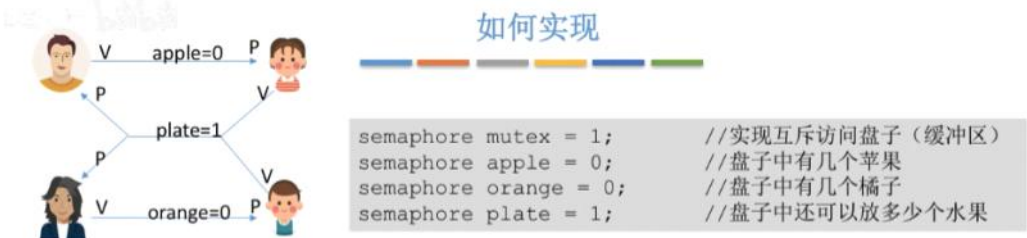
注意: P操作之间的顺序不能交换 (要先同步后互斥), 如果先P(mutex)后 P(empty)就会发生死锁, 生产者执行完P(mutex)让资源互斥, 但是被P(empty)阻塞了,

此时换成消费者，也会被P(mutex)阻塞，这样下来两边都被阻塞循环等待了，就发生了死锁。
但是v操作不会导致进程阻塞，是可以交换的。

7、多生产者-多消费者问题 （不是数量多而是不同类别多）



首先一段时间内只能有一个人访问盘子，设置互斥信号量mutex实现4个人对盘子实现互斥访问；
然后父亲得先放苹果女儿才能吃，设置初始值0的信号量apple，父亲先V女儿后P；
母亲得先放橘子儿子才能吃，设置初始值为0的信号量orange，母亲先V儿子后P；
当孩子们吃完盘子的事物使得盘子为空的时候，父母才能放入东西进去，设置初始值为1的信号量plate，孩子先V父母后P；



<pre>dad () { while(1) { 准备一个苹果; 把苹果放入盘子; } }</pre>	<pre>mom () { while(1) { 准备一个橘子; 把橘子放入盘子; } }</pre>	<pre>daughter () { while(1) { 从盘中取出苹果; 吃掉苹果; } }</pre>	<pre>son () { while(1) { 从盘中取出橘子; 吃掉橘子; } }</pre>
---	---	---	--

先实现同步关系：

<pre>dad () { while(1) { 准备一个苹果; P(plate); 把苹果放入盘子; V(apple); } }</pre>	<pre>mom () { while(1) { 准备一个橘子; P(plate); 把橘子放入盘子; V(orange); } }</pre>	<pre>daughter () { while(1) { P(apple); 从盘中取出苹果; V(plate); 吃掉苹果; } }</pre>	<pre>son () { while(1) { P(orange); 从盘中取出橘子; V(plate); 吃掉橘子; } }</pre>
---	--	--	--

然后实现互斥关系：

<pre>dad () { while(1) { 准备一个苹果; P(plate); 把苹果放入盘子; V(plate); } }</pre>	<pre>mom () { while(1) { 准备一个橘子; P(plate); 把橘子放入盘子; V(plate); } }</pre>	<pre>doughter () { while(1) { P(plate); 从盘中取出苹果; V(plate); 吃掉苹果; } }</pre>	<pre>son () { while(1) { P(plate); 从盘中取出橘子; V(plate); 吃掉橘子; } }</pre>
---	---	--	---

由于缓冲区大小才为1，无论什么情况下都只能有一个进程对缓冲区访问，因此互斥信号量mutex是可以删掉的。但是缓冲区大小大于1的时候就可能会有多个进程对缓冲区进行访问，这个时候互斥信号量不能删掉。

注意解决该类问题最重要的是要从事件的角度考虑，而不是人的角度考虑，比如儿子角度看，父亲没有放苹果母亲放了橘子才能取走；女儿角度看父亲放苹果母亲没放橘子才能取，这样就产生了4个信号量。实际上拿盘子的状态就可以抽象为1个信号量的。

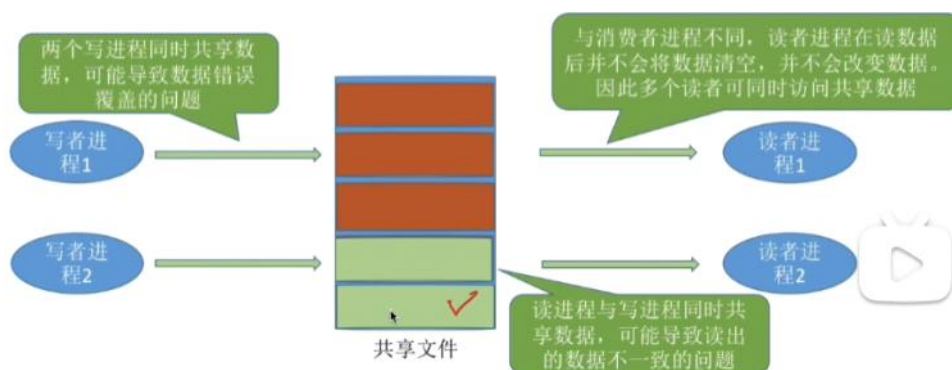


8、读者-写者问题

问题描述

问题描述

有读者和写者两组并发进程，共享一个文件，当两个或两个以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：①允许多个读者可以同时访问文件；②只允许一个写者往文件中写信息；③任一写者在完成写操作之前不允许其他读者或写者工作；④写者执行写操作前，应让已有的读者和写者全部退出。



首先读者和写者之间需要互斥的访问，就设置一个初值为1互斥信号量rw，rw=1的时候说明解锁了，读者可以读写者可以写，反之不行

```
semaphore rw=1; //用于实现对共享文件的互斥访问
```

```
writer () {
    while(1) {
        P(rw); //写之前“加锁”
        写文件...
        V(rw); //写完了“解锁”
    }
}
```

```
reader () {
    while(1) {
        P(rw); //读之前“加锁”

        读文件...

        V(rw); //读完了“解锁”
    }
}
```

但是这样会导致读者之间无法实现同时的访问，因此设置一个初始值为0的count信号量用来记录几个读进程在访问，如果是第一个读进程的，就负责“加锁”，其他的正常访问并且count++；如果是最后一个读完进程的就负责“解锁”；

```
semaphore rw=1; //用于实现对共享文件的互斥访问
int count = 0; //记录当前有几个读进程在访问文件
```

```
writer () {
    while(1) {
        P(rw); //写之前“加锁”
        写文件...
        V(rw); //写完了“解锁”
    }
}
```

```
reader () {
    while(1) {
        if(count==0) //由第一个读进程负责
            P(rw); //读之前“加锁”
        count++; //访问文件的读进程数+1

        读文件...

        count--; //访问文件的读进程数-1
        if(count==0) //由最后一个读进程负责
            V(rw); //读完了“解锁”
    }
}
```

但是还有问题，当多个读者进程并发运行时，第一个进程发现count==0后进去，但是还没加锁就切换到第二个进程，它加锁完后此时又切换回第一个进程，发现rw=0就阻塞了。造成这个情况的原因是i检查和上锁的过程中无法一气呵成，此时可以设置一个互斥信号量来保证读者进程的count访问是互斥的。

```
reader () {
    while(1) {
        P(mutex); //各读进程互斥访问count
        if(count==0) //由第一个读进程负责
            P(rw); //读之前“加锁”
        count++; //访问文件的读进程数+1
        V(mutex);
        读文件...
        P(mutex); //各读进程互斥访问count
        count--; //访问文件的读进程数-1
        if(count==0) //由最后一个读进程负责
            V(rw); //读完了“解锁”
        V(mutex);
    }
}
```

这样的话基本没问题了，但是这种方式对于读者来说是“优先”的，如果多个读者一直读一直读不解锁，写者就一直不能写，造成了“饿死”。

为了解决写进程饿死实现“写优先”，我们再设置一个互斥信号量w，写者在写文件的时候给w上锁，这时候读者就不能读；当一个读者先给w上锁后会先解锁再读文件，不妨碍后面的读者和写者的操作，这样就实现了一种“先来先服务”的效果，又称读写公平法。

```
semaphore rw=1; //用于实现对共享文件的互斥访问
int count = 0; //记录当前有几个读进程在访问文件
semaphore mutex = 1; //用于保证对count变量的互斥访问
semaphore w = 1; //用于实现“写优先”
```

```
writer () {
    while(1) {
        P(w);
        P(rw);
        写文件...
        V(rw);
        V(w);
    }
}
```

```
reader () {
    while(1) {
        P(w);
        P(mutex);
        if(count==0)
            P(rw);
        count++;
        V(mutex);
        V(w);
        读文件...
        P(mutex);
        count--;
        if(count==0)
            V(rw);
        V(mutex);
    }
}
```

9、哲学家进餐问题

问题描述

一张圆桌上坐着5名哲学家，每两个哲学家之间的桌上摆一根筷子，桌子的中间是一碗米饭。哲学家们倾注毕生的精力用于思考和进餐，哲学家在思考时，并不影响他人。只有当哲学家饥饿时，才试图拿起左、右两根筷子（一根一根地拿起）。如果筷子已在他人手上，则需等待。饥饿的哲学家只有同时拿起两根筷子才可以开始进餐，当进餐完毕后，放下筷子继续思考。

该问题的精髓就是哲学家进程需要同时持有2个临界资源才能开始运行，如何避免资源分配不当导致的死锁问题呢。

首先是定义一个筷子信号量数组chopstick[5]={1,1,1,1,1}，当每个哲学家有思考和进餐两种状态，当哲学家开始进餐之前，会拿起左右两边的筷子（P操作），吃完饭就把筷子放回去（V操作）。

```

semaphore chopstick[5]={1,1,1,1,1};
Pi () { //i号哲学家的进程
    while(1){
        P(chopstick[i]); //拿左
        P(chopstick[(i+1)%5]); //拿右
        吃饭...
        V(chopstick[i]); //放左
        V(chopstick[(i+1)%5]); //放右
        思考...
    }
}

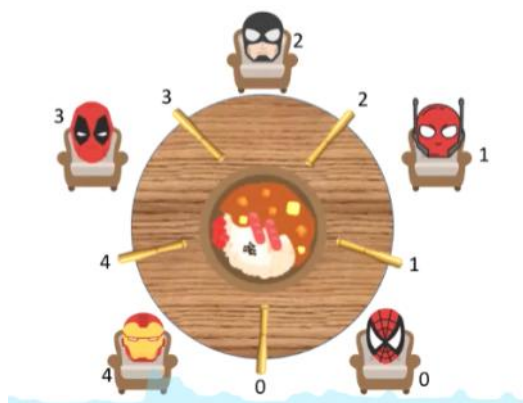
```

但是这样会有一个问题，就是当一个哲学家拿完一个筷子后切换到另一个哲学家拿一个筷子，这样5个哲学家都只拿到一个筷子，但是又得不到另一个筷子，都这么循环等待就造成死锁了。

为了解决死锁问题：

有一个方案就是最多允许4个哲学家同时进餐，这样的话就能保证有一个哲学家能拿到2双筷子，这种方法可以通过设置一个初值为4的信号量实现，当最后一个哲学家进程P操作发现 $s < 0$ 就阻塞等待。

另一个方案就是要求奇数号哲学家先拿左边的筷子，偶数号哲学家先拿右边的筷子，这样就能保证奇偶数哲学家竞争，最后只有一个哲学家能拿到筷子，另一个哲学家阻塞等待。如下图，最坏情况下4号一定能拿到2个筷子，不会死锁。



第三种方案就是让哲学家一气呵成的拿走两只筷子，设置一个互斥信号量mutex，当第一个哲学家拿走筷子的时候将 $mutex=0$ ，其他哲学家都不能拿筷子了，当第一个哲学家吃完饭将 $mutex=1$ 的时候才能拿筷子了。

```

semaphore chopstick[5]={1,1,1,1,1};
semaphore mutex = 1; //互斥地取筷子
Pi () { //i号哲学家的进程
    while(1){
        P(mutex);
        P(chopstick[i]); //拿左
        P(chopstick[(i+1)%5]); //拿右
        V(mutex);
        吃饭...
        V(chopstick[i]); //放左
        V(chopstick[(i+1)%5]); //放右
        思考...
    }
}

```

这种情况也存在一个问题，当第一个哲学家吃完饭执行V操作后，它旁边那个哲学家执行P操作正常拿起一个筷子，但是有一边的筷子被第一个哲学家拿走了，就进入阻塞状态，但此时 $mutex=0$ ，其他哲学家也拿不了筷子，直到第一个哲学家吃完饭，放回筷子才能继续下去，这样就实现了一气呵成的效果。

10、管程

由于信号量机制存在编写困难易出错的问题，例如生产者和消费者问题没有先同步后互斥造成了死锁，因此引入了管程。管程是一种高级的同步机制，相比于信号量机制更加方便不容易出错。

类似于C++中的私有类，管程由这些部分组成：局部于管程的共享数据结构说明（C++中私有类里的数据成员）、对该数据结构进行操作的一组过程（C++私有类中的成员函数）、对局部于管程的共享数据设置初始值的语句（C++中私有类数据成员的初始化）、管程有一个名字（类名）。

管程的基本特征：局部于管程的数据只能被局部于管程的过程所访问（私有类中的数据成员只能被里面的成员函数所访问）、一个进程只有通过调用管程内的过程才能进入管程访问共享数据（在私有类外的其他函数必须调用私有类里的成员函数才能间接的访问私有类的数据成员）、每次仅仅允许一个进程在管程内某个内部过程。

如下图所示是封装后的管程的具体实现。


```

monitor ProducerConsumer
condition full, empty; //条件变量用来实现同步（排队）
int count=0; //缓冲区中的产品数
void insert (Item item) { //把产品item放入缓冲区
    if (count == N)
        wait (full);
    count++;
    insert_item (item);
    if (count == 1)
        signal(empty);
}
Item remove () { //从缓冲区中取出一个产品
    if (count == 0)
        wait (empty);
    count--;
    if (count == N-1)
        signal(full);
    return remove_item();
}
end monitor;

```

6 条件变量

当一个进程进入管程后，若未能得到自己想要的资源，则会被阻塞，插入阻塞队列中。

由于阻塞的原因不同，不同的原因会被称为不同的条件变量，每个条件变量保存了一个等待队列，用于记录因该条件变量而阻塞的所有进程，对条件变量只能进行两种操作，即wait和signal。

x.wait: 当x对应的条件不满足时，正在调用管程的进程调用x.wait 将自己插入x条件的等待队列，并释放管程。

此时其他进程可以使用该管程。

x.signal: x对应的条件发生了变化，则调用x.signal，唤醒一个因x条件而阻塞的进程。

下面给出条件变量的定义和使用：

```

monitor Demo{ // 定义一个名称为Demo的管程
// 定义共享数据结构，对应系统中的某种共享资源
共享数据结构S;
condition x; // 定义一个条件变量x
对共享数据结构初始化的语句
init(){
    .....
}
take_away( ) { //源
    if(S<=0) x.wait(); //资源不够，在条件变量上阻塞等待
    资源足够，分配资源，做一系列响应处理；
    ...
}
give_back( ) { //
    对共享数据结构x的一系列处理；
    if(有进程在等待) x.signal(); //唤醒一个阻塞进程
}
}

```

条件变量和信号量的比较

相似点: 条件变量的wait/signal 操作类似于信号量的P/V操作，可以实现进程的阻塞/唤醒。

不同点: 条件变量是“没有值”的，仅实现了“排队等待”功能；

而信号量是“有值”的。信号量的值反映了剩余资源数，而在管程中，剩余资源数用共享数据结构记录

十二、死锁

1、死锁就是在并发环境下，几个进程互相等待对方的资源但是自己又持有对方想要的资源，但是最后都得不到资源而循环等待。如果没有外力进行干涉，这些进程将无法向前推进。

2、死锁，饥饿，死循环的区别

	共同点	区别
死锁	都是进程无法顺利向前推进的现象（故意设计的死循环除外）	死锁一定是“循环等待对方手里的资源”导致的，因此如果有死锁现象，那至少有两个或两个以上的进程同时发生死锁。另外，发生死锁的进程一定处于阻塞态。
饥饿		可能只有一个进程发生饥饿。发生饥饿的进程既可能是阻塞态(如长期得不到需要的I/O设备)，也可能是就绪态(长期得不到处理机)
死循环		可能只有一个进程发生死循环。死循环的进程可以上处理机运行（可以是运行态），只不过无法像期待的那样顺利推进。死锁和饥饿问题是由于操作系统分配资源的策略不合理导致的，而死循环是由代码逻辑的错误导致的。死锁和饥饿是管理者（操作系统）的问题，死循环是被管理者的问题。

3、死锁存在的必要条件：产生死锁必须同时满足以下4个条件，只要一个不成立就不发生

互斥条件: 只有必须互斥使用的临界资源才能导致死锁, 例如哲学家筷子打印机设备等, 像内存扬声器这些可以多个进程同时使用的才不会导致死锁。

不剥夺条件: 进程在获得的资源后还没用完, 只能自己释放, 不能被强行剥夺

请求和保持条件: 有进程已经保持了至少一个资源并且有其他资源的请求

循环等待条件: 存在一个进程资源的循环等待链 (我爱你, 你爱他, 他爱她, 她爱我但是都在等着对方的心)

注意: 发生死锁一定循环等待, 循环等待不一定是死锁, 因为同类资源数量大于1即使有循环等待也未必死锁, 但是同类资源只有一个就一定是死锁了。例如3号哲学家旁边有一个神仙, 可以给3号一个筷子, 这样即使都在循环等待, 也不是死锁。



4、死锁的发生的原因

对系统临界资源的竞争、进程推进顺序不对、信号量的使用都会造成死锁, 总之就是对不可剥夺资源的不合理分配。

5、死锁的处理策略——预防死锁

死锁的产生有4个必要条件, 只要破坏掉任意一个就不会发生死锁。

破坏互斥条件: 把互斥的资源改造成可以共享使用的资源, 例如打印机了SPOOLing技术就是把独占设备在逻辑上改造成共享设备, 如下图所示:



但是这种方法有一个问题就是不是所有的互斥资源都能改成共享资源, 为了系统的安全性, 必须保持互斥, 无法破坏互斥条件。

破坏不剥夺条件: 有2个方案, 第一个就是进程请求新资源得不到满足的时候, 就把进程所持有的所有资源主动释放掉, 以后重新申请, 即使是资源没用完也要强行释放, 这样就破坏了不可剥夺的条件; 第二个就是当某个进程需要的资源被其他进程占用的时候, 可以由操作系统协助把对方的持有资源强行剥夺, 这种剥夺一般都带优先级。缺点是实现复杂、反复申请系统资源增加系统开销、方案一直发生造成饥饿、造成被剥夺进程的阶段工作失效, 因此只适合容易保存和恢复现场的资源如CPU。

破坏请求和保持条件: 让进程在得到所有想要的资源后才开始投入运行, 一旦投入运行, 资源一直归他所有, 不再请求其他资源了。但是缺点也明显: 有些资源只要使用很短的时间, 整个进程的运行期间一旦保持所有资源就造成严重的资源浪费, 当然也有可能导致其他进程的饥饿。



破坏循环等待条件: 可以采用顺序资源分配法, 把每个资源进行编号, 每个进程必须按照编号递增顺序请求资源, 编号相同的同类资源一次性申请完, 这样的话进程占用小编号资源的时候才有资格申请大编号的资源, 已经有大编号的资源无法返回来申请小编号资源, 这样就不会产生循环等待了。

假设系统中共有10个资源, 编号为 1, 2, ..., 10



缺点是不方便增加新设备, 因为要重新分配编号、进程实际上使用资源的顺序和编号顺序不一样时会资源浪费、必须按照顺序申请资源也会让用户编程不方便。

6、死锁的处理策略——避免死锁

你是一位成功的银行家，手里掌握着100个亿的资金...
 有三个企业想找你贷款，分别是 企业B、企业A、企业T，为描述方便，简称BAT。
 B表示：“大哥，我最多会跟你借70亿...”
 A表示：“大哥，我最多会跟你借40亿...”
 T表示：“大哥，我最多会跟你借50亿...”
 然而...江湖中有个不成文的规矩：如果你借给企业的钱总数达不到企业提出的最大要求，那么不管你之前给企业借了多少钱，那些钱都拿不回来了...
 刚开始，BAT三个企业分别从你这儿借了 20、10、30 亿 ...

	最大需求	已借走	最多还会借
B	70	20	50
A	40	10	30
T	50	30	20



手里还有：40亿

如上图，如果B还想借30亿，那么手里只剩下了10亿，此时B还要20亿A还要30亿T还要20亿，借给企业钱的总数达不到企业的最大要求就破产了。B->T->A是不安全序列。

如果A想借20亿，那么手里还有20亿，此时可以把剩下的20亿全借给T，满足了T的最大需求，这样银行就得到了T的还款，一共有50亿，然后50亿全给B满足最大需求，得到70亿最后借给A，这样的顺序就不会再破产了。A->T->B安全序列。

同理的T->B->A也是安全序列。

安全序列就是系统按照这个顺序分配资源，每个进程都能顺利完成。

只要有一条安全序列系统就是**安全状态**。如果分配资源之后系统找不出一条安全序列那么系统就是**不安全状态**，有可能发生死锁，也有可能进程提前归还了一些系统资源，这样系统重新回到安全状态。因此系统处于安全状态就一定不会死锁，系统处于不安全状态也可能发生死锁。

银行家算法的核心思想就是在资源分配之前先判断这次分配是否会导致进入不安全状态，以此来决定是否分配资源，这样就**避免了死锁**。银行家算法的实现如下：

进程	最大需求	已分配	最多还需要
P0	(7, 5, 3)	(0, 1, 0)	(7, 4, 3)
P1	(3, 2, 2)	(2, 0, 0)	(1, 2, 2)
P2	(9, 0, 2)	(3, 0, 2)	(6, 0, 0)
P3	(2, 2, 2)	(2, 1, 1)	(0, 1, 1)
P4	(4, 3, 3)	(0, 0, 2)	(4, 3, 1)

资源总数 (10, 5, 7)，剩余可用资源 (3, 3, 2)

< (3, 3, 2)

说明如果优先把资源分配给P1，那P1一定可以顺利执行结束的。等P1结束了就会归还资源。于是，资源数就可以增加到 (2, 0, 0) + (3, 3, 2) = (5, 3, 2)

如上图(3,3,2)可以满足P1和P3的最大需求，满足后剩余可用资源到了(3,3,2)+(2,0,0)+(2,1,1)=(7,4,3)，此时又能满足P4后到了(7,4,3)+(0,0,2)=(7,4,5)，此时可用资源数能满足所有进程，说明是安全状态。

进程	最大需求	已分配	最多还需要
P0	(8, 5, 3)	(0, 1, 0)	(8, 4, 3)
P1	(3, 2, 2)	(2, 0, 0)	(1, 2, 2)
P2	(9, 5, 2)	(3, 0, 2)	(6, 5, 0)
P3	(2, 2, 2)	(2, 1, 1)	(0, 1, 1)
P4	(4, 3, 6)	(0, 0, 2)	(4, 3, 4)

资源总数 (10, 5, 7)，剩余可用资源 (3, 3, 2)

如上图(3,3,2)给P1和P3分配完资源后是(3,3,2)+(2,0,0)+(2,1,1)=(7,4,3)，此时无法满足任何一个进程的最大资源需求量，系统就是不安全状态，可能死锁。

银行家算法的代码实现如下图：

银行家算法

银行家算法

Available = (1, 2, 1)

Request₀ = (2, 1, 1)

假设系统中有 n 个进程， m 种资源
 每个进程在运行前先声明对各种资源的最大需求数，则可用一个 $n \times m$ 的矩阵（可用二维数组实现）表示所有进程对各种资源的最大需求数。不妨称为**最大需求矩阵 Max**， $Max[i, j] = K$ 表示进程 P_i 最多需要 K 个资源 R_j 。同理，系统可以用一个 $n \times m$ 的**分配矩阵 Allocation** 表示对所有进程的资源分配情况。 $Max - Allocation =$
Need 矩阵，表示各进程最多还需要多少各类资源。
 另外，还要用一个**长度为 m 的一维数组 Available** 表示当前系统中还有多少可用资源。
 某进程 P_i 向系统申请资源，可用一个**长度为 m 的一维数组 Request _{i}** 表示本次申请的各种资源量。

进程	最大需求	已分配	最多还需要
P0	(7, 5, 3)	(2, 2, 1)	(5, 3, 2)
P1	(3, 2, 2)	(2, 0, 0)	(1, 2, 2)
P2	(9, 0, 2)	(3, 0, 2)	(6, 0, 0)
P3	(2, 2, 2)	(2, 1, 1)	(0, 1, 1)
P4	(4, 3, 3)	(0, 0, 2)	(4, 3, 1)

Max 矩阵

Allocation 矩阵

Need 矩阵

可用**银行家算法**预判本次分配是否会导致系统进入不安全状态：因为它所需要的资源数已超过它所宣布的最大值。

①如果 $Request[i, j] \leq Need[i, j]$ ($0 \leq j \leq m$) 便转向②；否则认为出错。

②如果 $Request[i, j] \leq Available[j]$ ($0 \leq j \leq m$)，便转向③；否则表示尚无足够资源， P_i 必须等待。

③系统试探着把资源分配给进程 P_i ，并修改相应的数据（并非真的分配，修改数值只是为了做预判）：

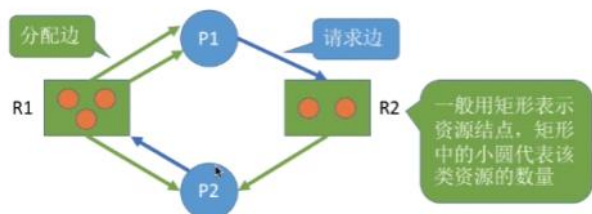
$Available = Available - Request;$

$Allocation[i, j] = Allocation[i, j] + Request[j];$

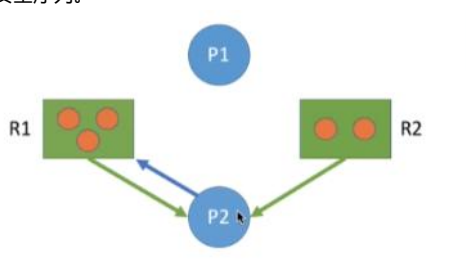
$Need[i, j] = Need[i, j] - Request[j]$

④操作系统执行**安全性算法**，检查此次资源分配后，系统是否处于安全状态。若安全，才正式分配；否则，恢复相应数据，让进程阻塞等待。

7、死锁的处理策略——检测死锁



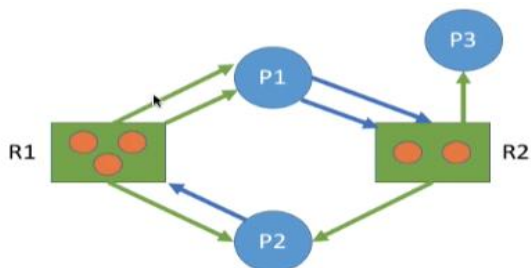
如上图所示，P表示进程，R表示资源，请求边表示进程对某资源发了申请请求，分配边表示该资源已经给谁分配了资源。上图P1请求R2资源，R2本身有2个资源分配出去一个，还有一个空闲资源，因此P1是可以顺利执行下去的；P2向R1发生请求，虽然R1资源全被分配出去了，但是P1进程结束后就有资源分配给P2了，因此这就是一个安全序列。



按照上述过程能消除所有的边说明一定没有死锁，相反如果不能消除所有边就是发生了死锁。



如下图就是发生死锁的情况，下图只有P3进程能顺利执行下去，最后还连着边的那些进程就是处于死锁状态的进程。



8、死锁的处理策略——解除死锁

并不是所有系统中的进程都是死锁状态，只有资源分配图化简后来连着边的才是死锁进程，要想解决死锁局面，可以使用以下方法：

资源剥夺法：把某些死锁进程挂起（放到外存），释放他手中的所有资源，注意要防止挂起过久导致饥饿

撤销进程法：撤销部分甚至所有死锁进程，并剥夺他们手中所有资源，这样的代价可能会很大，有些进程就快完成了就功亏一篑了

进程回退法：让一个死锁进程或多个死锁进程回退到足矣避免死锁的地步，但是要求系统要记录历史信息设置还原点，可能实现困难。

根据具体情况可以选择其中一个方法，例如进程优先级低的给剥夺、执行时间短的撤销掉、保住马上可以结束的进程杀掉其他进程、撤销使用资源多的进程、优先撤销批处理进程保住交互式进程等等。

内存管理

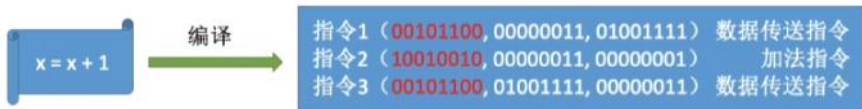
2025年6月5日星期四 上午11:10

内存管理自问自答

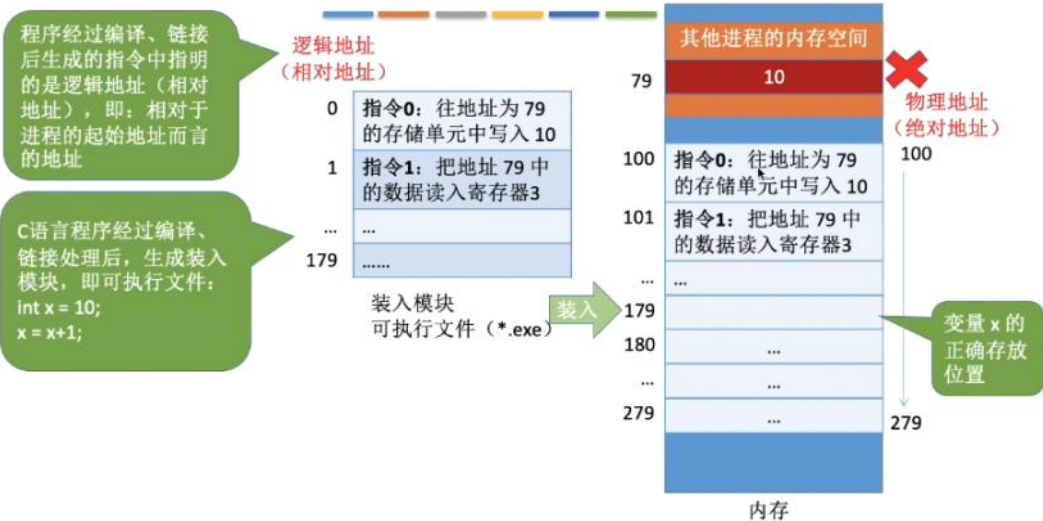
- 内存是什么？内存的作用是什么？
- 程序从编写到运行经历了什么？
- 程序的装入有哪几种？他们的具体实现步骤是什么？
- 程序的链接有哪几种？他们的具体实现步骤是什么？
- 内存的保护有哪些方式？
- 内存的连续分配有哪些方式？动态分区分配的算法有哪些？
- 外部碎片和内部碎片的定义是什么？
- 内存离散分配方式有哪些？
- 分页存储的概念是什么？页表是干嘛的？由什么组成？
- 分页存储的逻辑地址结构是什么？页表寄存器的结构是什么？地址转换过程是什么？
- 快表的加入是怎么提升查找速度的？局部性原理是什么？
- 为什么要引入两级页表？两级页表的逻辑地址结构是什么？
- 分段存储的概念是什么？段表是干嘛的，和页表区别是什么？由什么组成？
- 分段存储的逻辑地址结构是什么？段表寄存器的结构是什么？地址转换过程是什么？
- 分页存储和分段存储的区别是什么？
- 页表和段表在访问的过程中需要访存几次，列举出多级段表页表的情况？
- 段页式存储管理的概念是什么？逻辑地址结构是什么？地址转换的过程是什么？
- 虚拟内存的定义和特征是什么？请求分页管理方式是由什么组成的？
- 页面置换算法有哪些？
- 页面的分配和置换方式有哪几种？它们可以组合成哪几种情况？
- 页面是什么时候调入的？从什么地方调入的？有哪几种情况？
- 什么是抖动？内存映射文件是什么？

一、内存的基础知识

- 内存是主机的一部分，可以存放数据，程序需要先从外存中放入内存能被CPU处理——缓和CPU和硬盘速度之间的矛盾。
在多道程序下，系统中有多个进程并发执行，就是有多个程序会放入内存中，此时计算机会根据每个进程在内存中的编地址进行区分不同的进程，内存地址从0开始，每一个地址对应一个存储单元。如果计算机是“按字节编址”，则每个存储单元大小是1字节也就是1B，8个二进制位；如果计算机是“按字编址”，则每个存储单元大小是1个字，也就是2B，16个2进制位。
- 指令的工作原理



如上图一个c语言代码，在编译后会转换成3个指令，其中指令有3个“参数”，第一个是操作码，指明了这条指令要干什么事情；后面2个就是具体的数据或者物理地址。例如上面指令1就是将执行数据传送，将数据00000011放入内存单元01001111里面；指令2就是执行加法指令，将00000011与00000001相加；指令3就是将00000011放回内存单元01001111里面。



如上图一个外存中的c语言exe程序在经过编译之后变成180条指令，此时如果要让CPU执行，就需要先装入内存，在装入内存的过程中需要将原来的逻辑地

址改成物理地址，这样才不会造成冲突。此时就有2个问题：程序是怎么装入的？逻辑地址是怎么变成物理地址的？

3、程序的装入

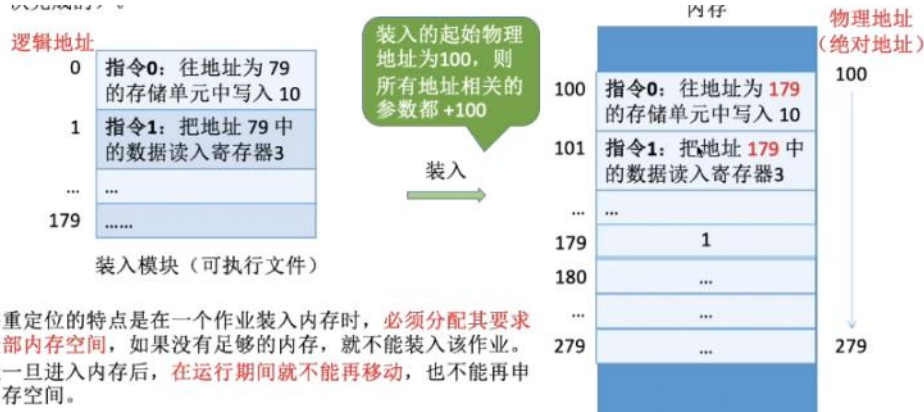
绝对装入：在编译的时候先确定装入的地址，然后把这一坨都装进去，不需要进行地址转换。程序也不会先看内存中的地址有没有被占用，直接说“我要**到**的地址空间”。

Eg: 如果知道装入模块要从地址为 100 的地方开始存放...



这种装入方式灵活性差，只适合单道程序环境，而且第一台电脑是从100开始装，如果换电脑了那可能就不能装了。

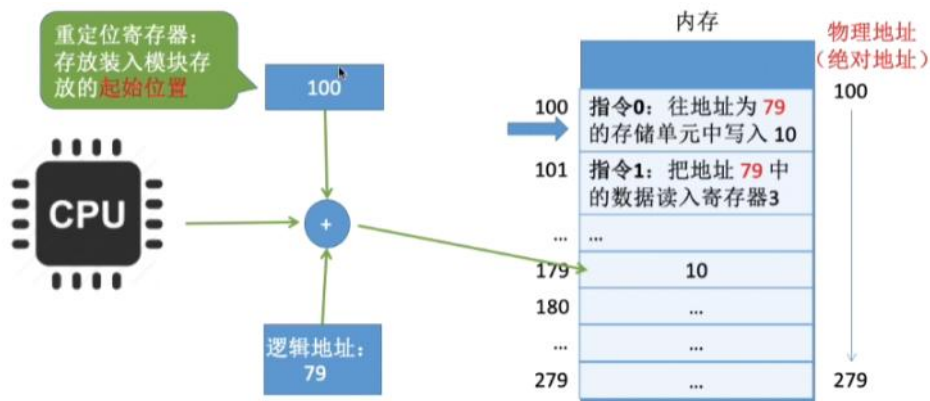
可重定位装入 (静态重定位)：编译链接的时候用的逻辑地址，程序根据内存的实际情况，挑一块合适的地址一次性把所有东西装进去，将逻辑地址转换成物理地址。在装入的时候对地址进行“重定位”。



静态重定位的特点是在一个作业装入内存时，必须分配其要求的全部内存空间，如果没有足够的内存，就不能装入该作业。作业一旦进入内存后，在运行期间就不能再移动，也不能再申请内存空间。

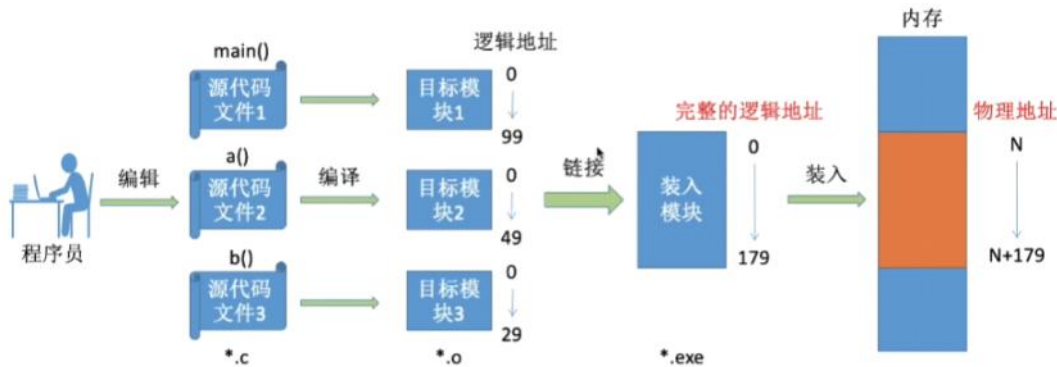
缺点也很明显：找不到合适的内存空间就装不进去，进入内存后不能再动了，灵活性差。

动态运行时装入 (动态重定位)：编译链接的时候用逻辑地址，在装入程序的时候不会立即把物理地址转成逻辑地址，而是程序要用到它的时候才进行地址转换。因此装入内存后还是逻辑地址，为了不和实际上的物理地址冲突，需要一个重定位寄存器来支持。



如上图，重定位寄存器存放的是在内存中的起始地址，在需要进行地址转换的时候将逻辑地址和起始地址相加就得到了物理地址，这种方式允许程序在内存中移动，只要修改重定位寄存器的值即可。

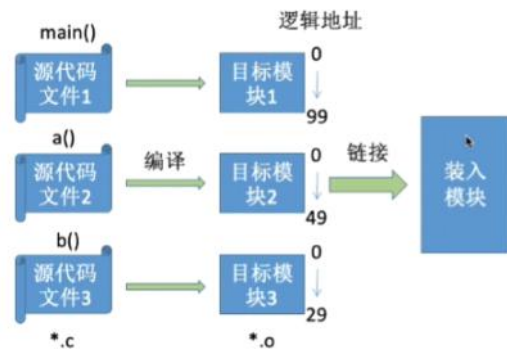
4、程序从编写到运行的过程



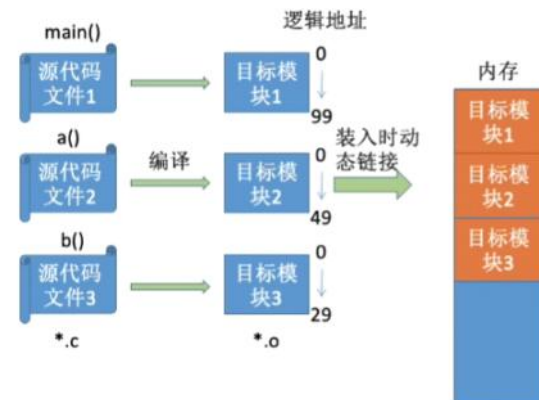
如上图：程序员在编写完程序后形成.c文件；在编译后代码变成一堆机器指令，这些指令分模块，每个模块都有自己的逻辑地址；然后将这些零散的模块连接起来形成.exe文件；最后要运行它的话就装入内存中运行，将逻辑地址转换成物理地址。

5、程序的链接

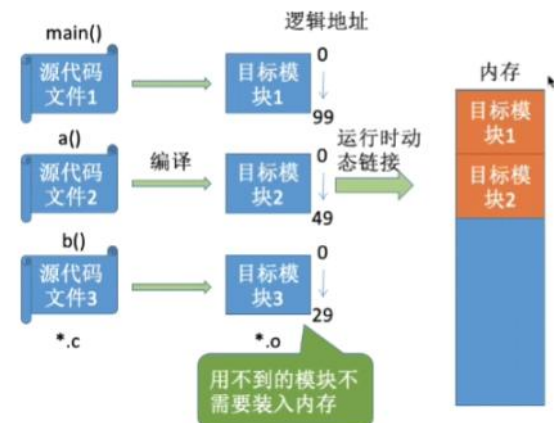
静态链接：在程序运行之前先将各目标模块以及所需库函数连接成一个完整的可执行文件（装入模块），之后不再拆开。



装入时动态链接：在将各模块装入内存的时候，边装入边链接的方式，有些用不上的模块也会装上去。



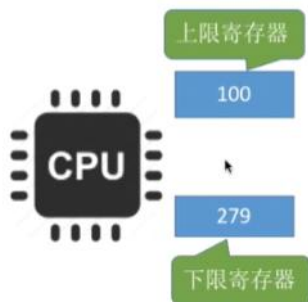
运行时动态链接：在程序执行中需要该模块的时候才进行链接，优点是加快装入过程，节省空间。



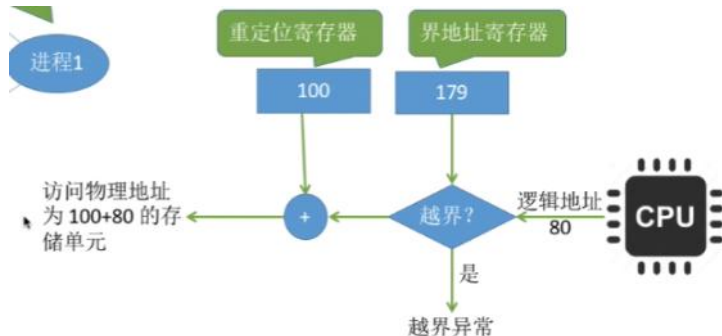
6、内存的保护

内存保护的目的是让进程只访问属于自己的那部分内存，不能访问其他进程或者操作系统的那部分内存，从而保证系统的安全性。

方法一：设置一对上下限寄存器，存放进程的上下限地址，当进程要访问某个地址空间的时候，CPU需要检查是否越界



方法二：设置**重定位寄存器**（**基地址寄存器**）和**界地址寄存器**（**限长寄存器**）进行越界检查。重定位寄存器存放进程的起始物理地址，界地址寄存器存放进程最大的逻辑地址，检查时判断逻辑长度有没有超过即可。



二、内存空间的分配与回收——连续分配

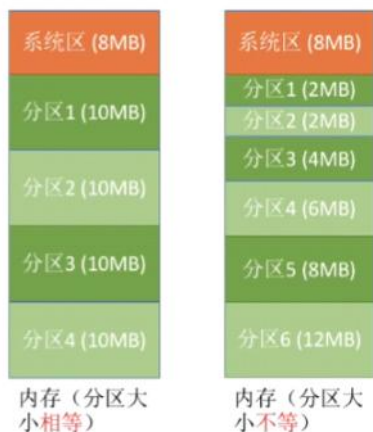
1、连续分配管理方式

单一连续分配方式：内存分为系统区和用户区，**用户程序独占整个用户区空间**，



优点是实现简单、不存在外部碎片，还不一定要采取内存保护；缺点是只适合单用户单任务操作系统，有内部碎片且内存利用率低下。

固定分区分配：把用户区分成几个固定大小的分区，分区可以相等也可以不等，每个分区只装一个作业。



分区大小相等的缺乏灵活性，适合一个计算机控制多台相同的对象的场合；分区大小不等可以满足不同大小的程序需求，灵活性更高。为了对各个分区进行管理，可以建立一个**分区说明表**来实现对各个分区的控制。需要分配内存的话检索该表有没有空闲的就行了。

分区号	大小 (MB)	起始地址 (M)	状态
1	2	8	未分配
2	2	10	未分配
3	4	12	已分配
*****	*****	*****	*****

用数据结构的数组（或链表）即可表示这个表

优点是简单**没外部碎片**，缺点是用户程序太大就装不下了，不得不采取覆盖技术来解决；但是**会产生内部碎片**，降低内存利用率。

动态分区分配：不会预先划分内存区，而是**根据进程的大小动态的建立分区**，并且使得分区大小正好满足需要。



在这种分配方式中，系统会设置一个**空闲分区表**或者**空闲分区链**记录内存的使用情况。



通过**修改空闲分区表**就可以**实现分区的分配和回收操作**，例如下面有3个空闲分区分别是8-28、32-42、60-64，要在第三个空间区插入大小为4M的程序，就在空闲分区表中删除第三个即可。



内存

对空闲分区表的修改也有几种情况：

分区号	分区大小 (MB)	起始地址 (M)	状态
1	20	8	空闲
2	10	32	空闲
3	4	60	空闲

分区号	分区大小 (MB)	起始地址 (M)	状态
1	20	8	空闲
2	10	32	空闲



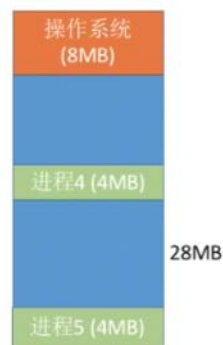
内存

情况一：回收区的后面有一个相邻的空闲分区

分区号	分区大小 (MB)	起始地址 (M)	状态
1	10	32	空闲
2	4	60	空闲

分区号	分区大小 (MB)	起始地址 (M)	状态
1	14	28	空闲
2	4	60	空闲

两个相邻的空闲分区合并为一个



内存

情况二：回收区的前面有一个相邻的空闲分区

分区号	分区大小 (MB)	起始地址 (M)	状态
1	20	8	空闲
2	10	32	空闲

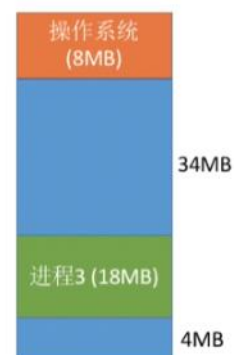
分区号	分区大小 (MB)	起始地址 (M)	状态
1	20	8	空闲
2	28	32	空闲

情况三：回收区的前、后各有一个相邻的空闲分区

分区号	分区大小 (MB)	起始地址 (M)	状态
1	20	8	空闲
2	10	32	空闲
3	4	60	空闲

分区号	分区大小 (MB)	起始地址 (M)	状态
1	34	8	空闲
2	4	60	空闲

三个相邻的空闲分区合并为一个



内存



情况四：回收区的前、后都没有相邻的空闲分区

分区号	分区大小 (MB)	起始地址 (M)	状态
1	4	60	空闲

分区号	分区大小 (MB)	起始地址 (M)	状态
1	14	28	空闲
2	4	60	空闲

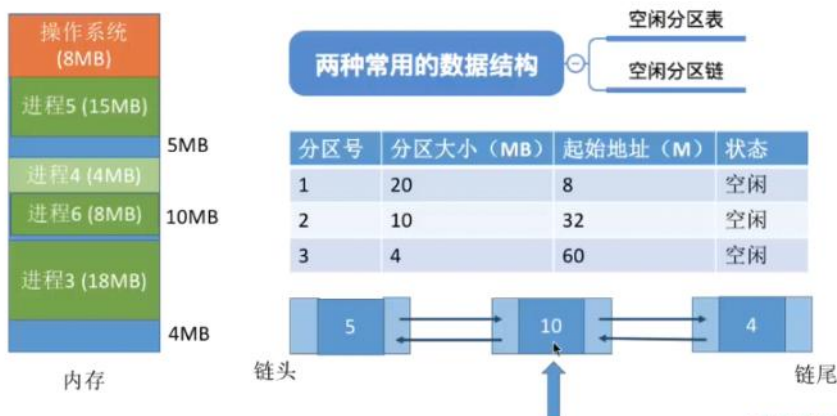
新增一个表项

注：各表项的顺序不一定按照地址递增顺序排列，具体的排列方式需要依据动态分区分配算法来确定。

总之，动态分区分配不存在内部碎片但是有外部碎片。

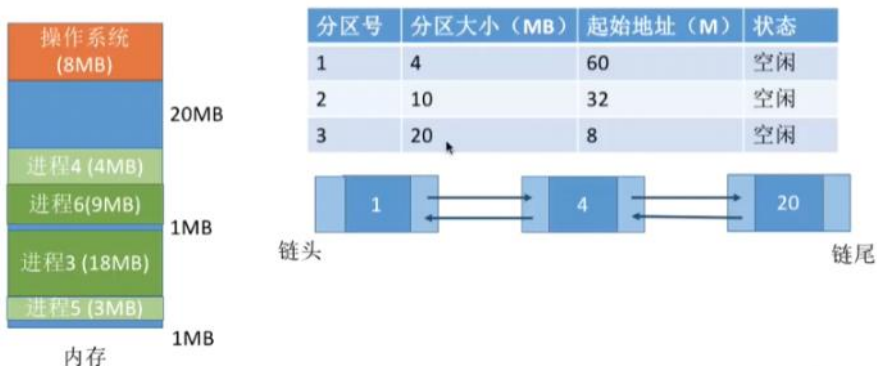
2、动态分区分配的算法

首次适应算法：按照地址从低到高一个个查找空闲分区，直到找到能满足大小的第一个空闲分区为止。



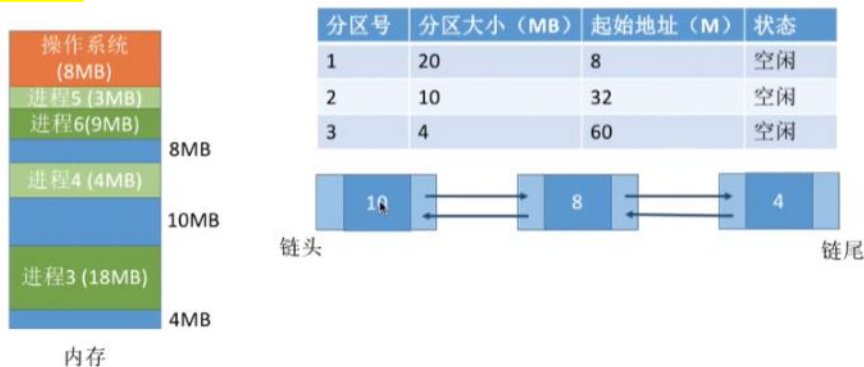
王道考研/CSKAOYAN.CN

最佳适应算法：为了保证大进程的来临时有更大的空间，优先找满足要求的最小空间区域。因此需要先对空闲分区按大小进行排序，并从小到大进行查找



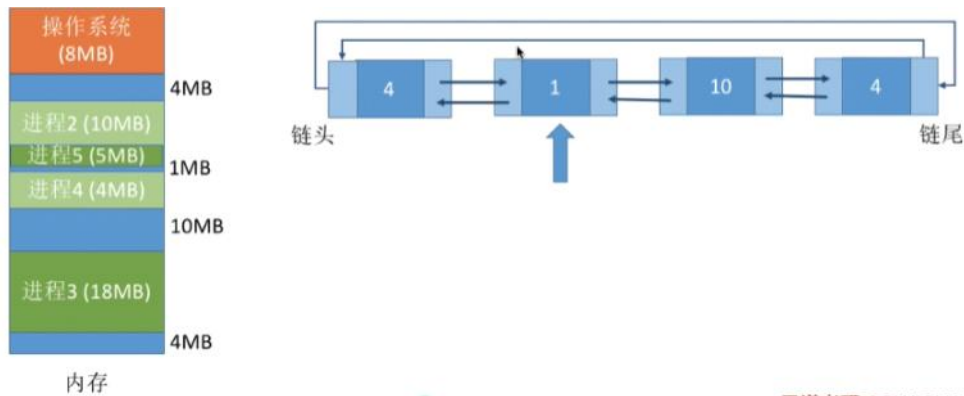
缺点：每一次都选择最小的分区进行分配，会留下越来越多难以利用的外部碎片。

最坏适应算法：为了解决最佳适应算法的问题，优先使用满足区域的最大空间区域，这样留下的外部碎片就不会太小。因此需要先大到小排序然后查找。



缺点是会让大空闲分区迅速用完，如果之后又来一个大进程，就没有足够大小的空闲区了。

邻近适应算法：按照地址从低到高一个个查找空闲分区，直到找到能满足大小的第一个空闲分区为止，然后下一次查找是从上次结束的位置开始的。



算法开销比较小的是适应和邻近适应，因为在查找前不用先进行一次排序。

算法	算法思想	分区排列顺序	优点	缺点
首次适应	从头到尾找适合的分	空闲分区以地址递增次序排列	综合看性能最好。 算法开销小 ，回收分区后一般不需要对空闲分区队列重新排序	
最佳适应	优先使用更小的分区，以保留更多大分区	空闲分区以容量递增次序排列	会有更多的大分区被保留下来，更能满足大进程需求	会产生很多太小的、难以利用的碎片； 算法开销大 ，回收分区后可能需要对空闲分区队列重新排序
最坏适应	优先使用更大的分区，以防止产生太小的不可用的碎片	空闲分区以容量递减次序排列	可以减少难以利用的小碎片	大分区容易被用完，不利于大进程； 算法开销大 （原因同上）
邻近适应	由首次适应演变而来，每次从上次查找结束位置开始查找	空闲分区以地址递增次序排列（可排列成循环链表）	不用每次都从低地址的小分区开始检索。 算法开销小 （原因同首次适应算法）	会使高地址的大分区也被用完

3、外部碎片和内部碎片

内部碎片：内存区域已经分配给进程，但是进程却没有利用上。

外部碎片：由于内存区域实在太小，难以被任何进程所利用。

由于内存空间是连续的，因此外部碎片可以通过消耗一些性能的“**紧凑技术**”来解决。

三、内存空间的分配与回收——非连续分配

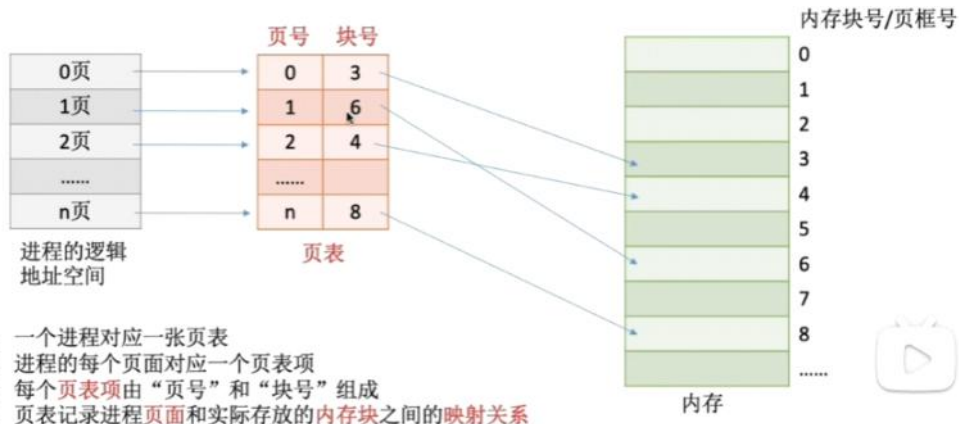
1、分页存储概念

分页就是将内存空间划分成一个个大小相等（例如4kb）的分区，每个分区就是一个**页框**（**页帧、内存块、物理块、物理页面**），每个页框都有一个编号就是**页框号**（**页帧号、内存块号、物理块号、物理页面号**），页框号从0开始。

操作系统以页框为基本单位为**各个**进程分配内存空间，进程的页面和内存的页框大小相等，是一一对应的关系。各个页面**不必连续存放**，可以放到不相邻的各个页框中。

2、页表

为了知道进程分页后各个页面在内存中的位置，操作系统给每个进程建立一张页表，页表通常放在PCB中。



假设某系统物理内存大小为4GB，页面大小4KB。

则内存块大小=页面大小=4KB=2¹²，内存总大小为4GB=2³²，则可以分成2²⁰个内存块，内存块号的范围是0-2²⁰-1，因此至少需要20bit=3B表示内存块号。

注意页表由页号和块号组成，内存块号需要3B，**页号物理上不占空间**，因为序号是连续存储的，假设页表中各页表项从内存地址x开始连续存放，需要找第i号页表，就直接通过x+3*i就可以求出页表项的存放地址。



注意j号内存块的起始地址是j*单个内存块大小

3、非连续分配的地址转换

虽然进程的各个页面在物理上是离散存放的，但是在页面的内部仍然是连续存放的。



根据这些特点，如果要访问逻辑地址A，则先确定该地址属于哪一个页号，页内偏移量是多少；然后查页表找到这个页号对应的内存块号并且求出在物理地址的起始地址，最后根据该页面在内存中的起始地址+页内偏移量得出对应的物理地址。

Eg: 在某计算机系统中，页面大小是50B。某进程逻辑地址空间大小为200B，则逻辑地址 110 对应的页号、页内偏移量是多少？



如下图，如果用2的整数幂来表示页面大小，那么硬件就能很方便的把逻辑地址改成物理地址。

假设某计算机用32个二进制位表示逻辑地址，页面大小为 4KB = 2^{12} B = 4096B

0号页的逻辑地址范围应该是 0~4095，用二进制表示应该是：
00000000000000000000000000000000 ~ 00000000000000000000000000000000111111111111

1号页的逻辑地址范围应该是 4096~8191，用二进制表示应该是：
0000000000000000000000000000000010000000000000000 ~ 00000000000000000000000000000000111111111111

2号页的逻辑地址范围应该是 8192~12287，用二进制表示应该是：
0000000000000000000000000000000010000000000000000 ~ 000000000000000000000000000000001011111111111

Eg: 逻辑地址 2，用二进制表示应该是 0000000000000000000000000000000010
页号 = $2/4096 = 0$ = 00000000000000000000000000000000, 页内偏移量 = $2\%4096 = 2$ = 000000000010

Eg: 逻辑地址 4097，用二进制表示应该是 00000000000000000000000000000001000000000001
页号 = $4097/4096 = 1$ = 00000000000000000000000000000001, 页内偏移量 = $4097\%4096 = 1$ = 000000000001

结论：如果每个页面大小为 2^k B，用二进制数表示逻辑地址，则末尾 k 位即为页内偏移量，其余部分就是页号

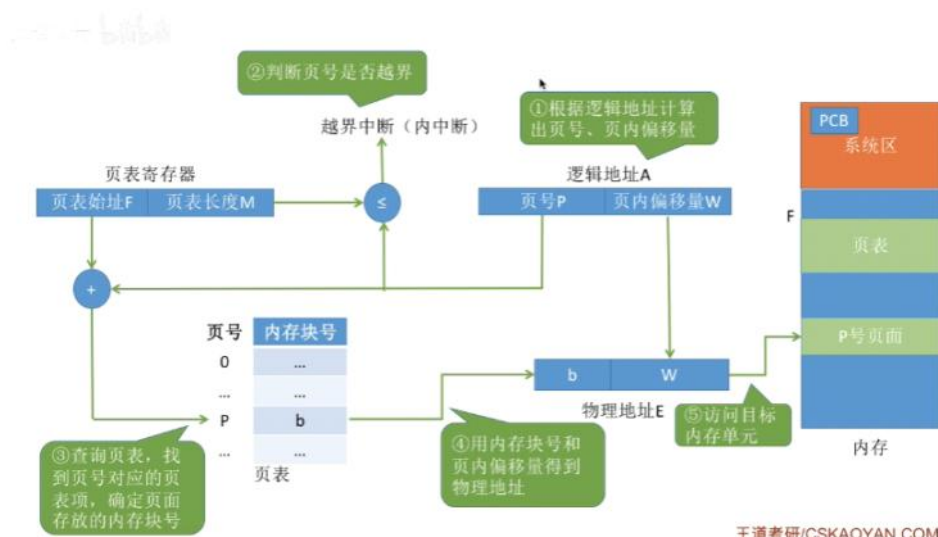
4、分页存储的逻辑地址结构



逻辑地址由页号和页内偏移量组成，如上图32位系统中，0到11位是页内偏移量，12-31表示页号。如果k位表示页内偏移量，则说明系统中一个页面大小是 2^k 个内存单元；如果M位表示页号，则说明一个进程最多允许 2^M 个页面。

5、非连续分配的地址转换机构——页表寄存器PTR

页表寄存器用于将分页后的逻辑地址转换成物理地址，存放于系统区的PCB当中。由页表的地址始址F和页表的长度M组成，其中始址用于查页表，页表长度用于检查是否越界。



如上图：先根据逻辑地址算出在哪个页号和页内偏移量，然后页号和页表长度对比是否越界，没有越界的话根据页表逻辑上的首地址查页表，得出在哪个物理地址，最后将物理首地址和页内偏移量相加即可。

注意：页面大小是2的整数幂

设页面大小为L，逻辑地址A到物理地址E的变换过程如下：

- ①计算页号P和页内偏移量W（如果用十进制数手算，则 $P=A/L$ ， $W=A\%L$ ；但是在计算机实际运行时，逻辑地址结构是固定不变的，因此计算机硬件可以更快地得到二进制表示的页号、页内偏移量）
- ②比较页号P和页表长度M，若 $P \geq M$ ，则产生越界中断，否则继续执行。（注意：页号是从0开始的，而页表长度至少是1，因此 $P=M$ 时也会越界）
- ③页表中页号P对应的页表项地址 = 页表起始地址F + 页号P * 页表项长度，取出该页表项内容b，即为内存块号。（注意区分页表项长度、页表长度、页面大小的区别。页表长度指的是这个页表中总共有几个页表项，即总共有几个页；页表项长度指的是每个页表项占多大的存储空间；页面大小指的是一个页面占多大的存储空间）
- ④计算 $E = b * L + W$ ，用得到的物理地址E去访存。（如果内存块号、页面偏移量是用二进制表示的，那么把二者拼接起来就是最终的物理地址了）

例：若页面大小L为1K字节，页号2对应的内存块号 $b = 8$ ，将逻辑地址 $A=2500$ 转换为物理地址E。
等价描述：某系统按字节寻址，逻辑地址结构中，页内偏移量占10位，页号2对应的内存块号 $b = 8$ ，将逻辑地址 $A=2500$ 转换为物理地址E。

说明一个页面的大小为 $2^{10}B = 1KB$

①计算页号、页内偏移量

页号 $P = A/L = 2500/1024 = 2$ ； 页内偏移量 $W = A\%L = 2500\%1024 = 452$

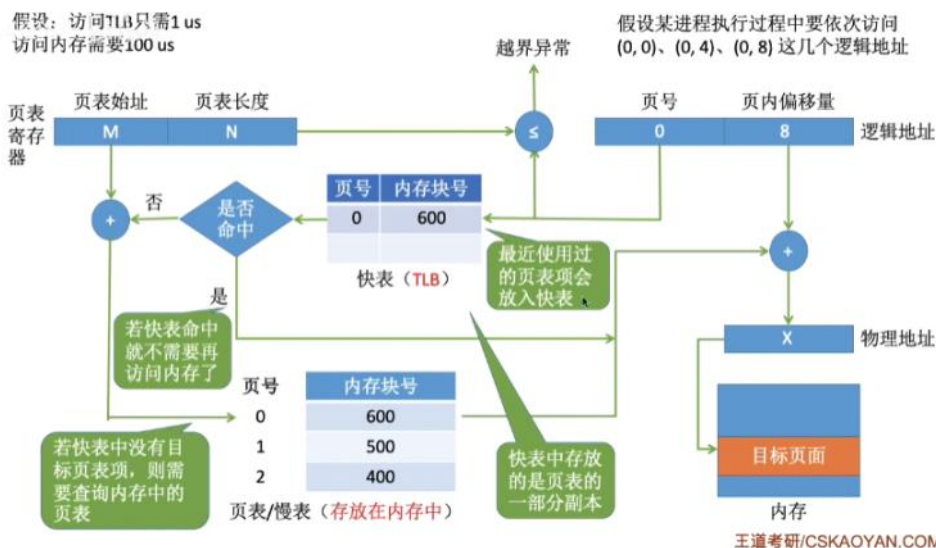
②根据题中条件可知，页号2没有越界，其存放的内存块号 $b = 8$

③物理地址 $E = b * L + W = 8 * 1024 + 452 = 8644$

注意：在实际应用中通常使得一个页框恰好放入整数个页表项，例如每个页表项占用3字节，一个页面4KB，则 $4096/3=1365$ 个页表项，但是还会产生 $4096\%3=1$ 个页内碎片，此时如果让每个页表项占用4字节，能整除就不会有页内碎片了。

6、带快表TLB的地址变换机构

快表就是一种比内存快很多的高速缓存，用来存放最近访问过的页表项副本，可以加快地址变换的速度。



如上图，相比于之前的访问方式，它查完越界后会查快表中是否有最近访问过的页表项，如果有就不查页表直接访问该地址，如果没有就正常查页表再访问地址。

注意查页表和访问物理地址都是访问内存，如果快表命中了就不需要查页表，也就是说只访问一次内存，如果没有命中就是访问2次内存。

例：某系统使用基本分页存储管理，并采用了具有快表的地址变换机构。访问一次快表耗时 $1\mu s$ ，访问一次内存耗时 $100\mu s$ 。若快表的命中率为 90% ，那么访问一个逻辑地址的平均耗时是多少？

$(1+100) \times 0.9 + (1+100+100) \times 0.1 = 111\mu s$

有的系统支持快表和慢表同时查找，如果是这样，平均耗时应该是 $(1+100) \times 0.9 + (100+100) \times 0.1 = 110.9\mu s$

根据局部性原理，带有快表效率能提升很多。

7、局部性原理

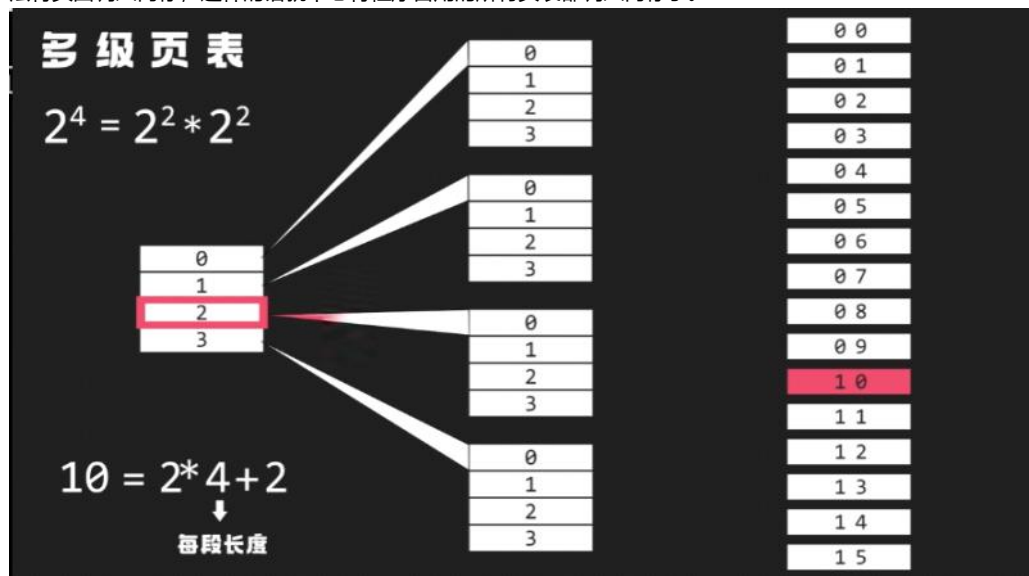
时间局部性：因为程序中有循环结构，当执行了程序的某条指令的时候，那么不久后很可能会再次执行该指令。

空间局部性：因为很多数据在内存中连续存放，当程序访问了某条内存单元，那么在不久后它附近的内存单元也很可能会被访问。

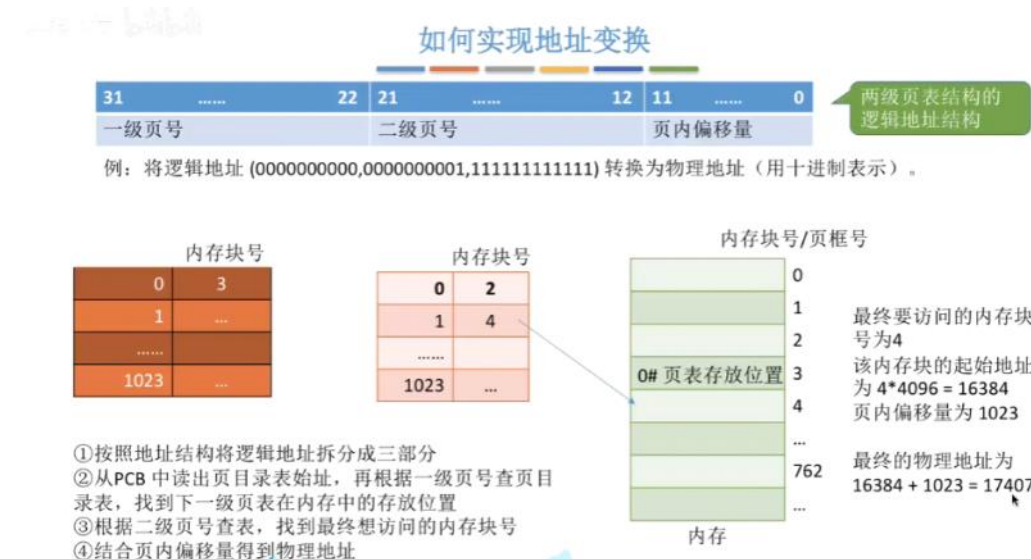
8、两级页表和多级页表

单级页表中存在一些问题，第一是页表必须连续存放，当页表很大的时候需要占用大量连续的页框；第二是根据局部性原理可知，一个进程在一段时间内只需要访问某几个页面就能正常运行，没必要将整个页表都常驻内存。

如下图：可以把连续的一大块页框进行一个分组，每个分组页框数量相同，这些分组可以是离散的，因此需要为这些离散分配的分组再建立一张页表叫**页目录表**（或者叫外层页表、顶层页表）。再加一个“是否在内存中”的指标表示该页面是否已经调入内存，如果没有的话产生缺页中断，开始页面置换算法将页面调入内存，这样的话就不必将程序占用的所有页表都调入内存了。



二级页表的地址转换就是套娃，先在一级页表中查是哪个二级页表，然后在二级页表中查物理地址首地址，最后加上偏移量即可。



注意：如果采用多级页表机制，各级页表大小不能超过一个页面；两级页表访问套娃，需要访问3次内存，如果是n级页表，则需要访问n+1次。

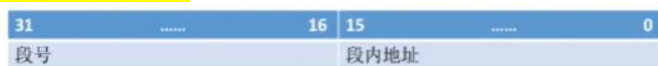
9、基本分段存储管理

一个进程可以根据自己的逻辑关系来划分若干个段，每一个段都有一个段名，从0开始编址。如下图就是分成了多个段。



这些段可以离散的分配在不同的内存空间，因此基本分段就是以段为基本单位，每个段在内存中占用连续空间，但是段与段之间是可以不相邻的。这种分段的优点就是便于用户编程，提高程序的可读性。

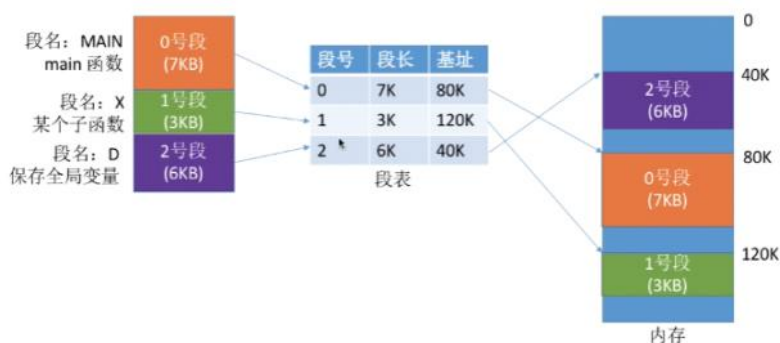
分段管理的逻辑地址：



在逻辑地址结构上，由段号和段内地址组成，段号决定了每个进程最多可以分成几段，段内地址决定了每个段的长度是多少。

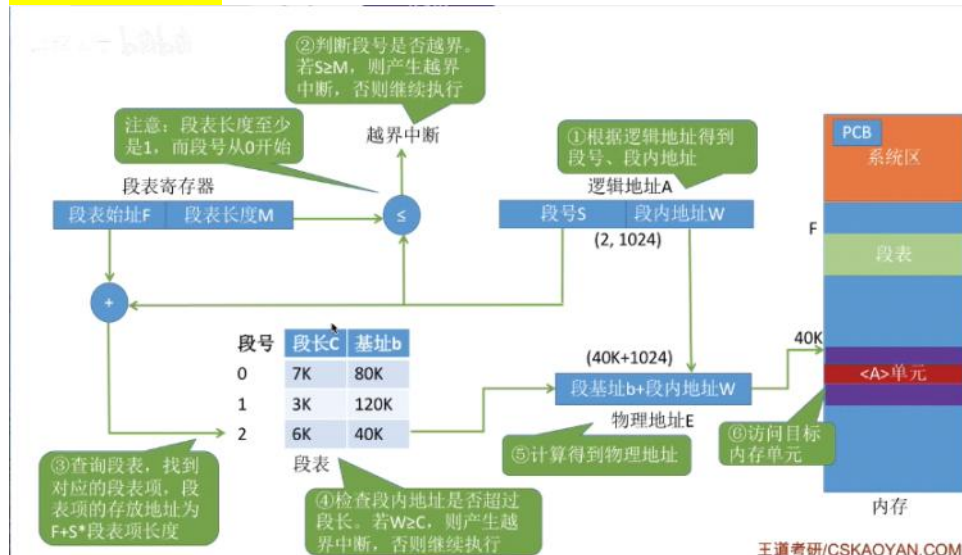
段表：

由于每个段是离散的装入内存，为了保证程序正常运行，就必须能从物理内存中找到各个逻辑段的存放位置，因此需要建立一张段表来反映逻辑地址与物理地址的映射关系。



在上图中段长反映每个段的长度，段基址反映每个段的物理起始位置，在段表中每个段表项的长度是一样的且段表占用一片连续空间，因此段号是可以隐含的，不占用存储空间。假设段表起始地址是M，每个段表项占用6B，则K号段的起始地址是M+K*6。

分段管理的地址转换：



如上图是段的地址转换过程，和页存储不同的是段的段长时参差不齐的，需要额外对比段内地址和段长以检查是否越界。

分段和分页的对比：分页是物理单位，是系统行为，对用户不可见而段是信息的逻辑单位，用户知道分成什么段那几个段，是用户行为；分页的空间是1维而分段的地址空间是2维（多了段长）。最重要的是分段更容易实现信息的共享和保护，只要设置一个段允许共享使用即可，一般来说不能被修改的纯代码或者可重入代码是可以共享的，可以被修改的代码是不能共享的。

访存：和分页一样，第一次访存是查段表，第二次是访问目标单元。也可以引入快表来提升访存速度。

10、段页式管理方式

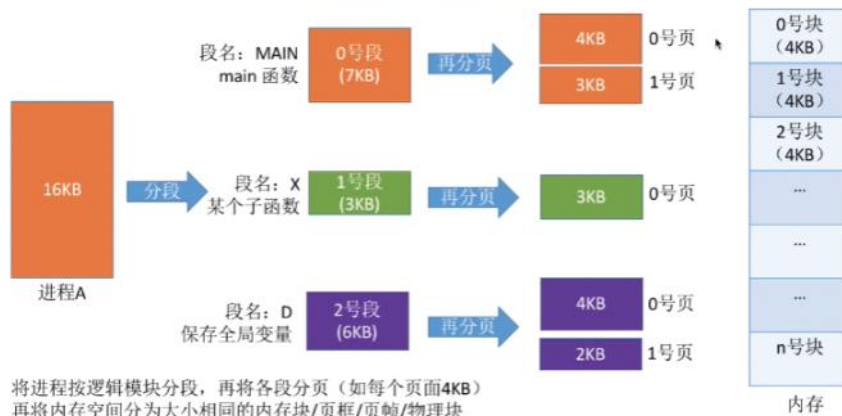
分页和分段各有自己的优缺点，如果将其结合，那么就是段页式管理。

	优点	缺点
分页管理	内存空间利用率高, 不会产生外部碎片 , 只有少量的页内碎片	不方便按照逻辑模块实现信息的共享和保护
分段管理	很方便按照逻辑模块实现信息的共享和保护	如果段长过大, 为其分配很大的连续空间会很方便。另外, 段式管理 会产生外部碎片



王道考研/CSKAQYAN.CC

段页式就是先把进程根据逻辑模块来分段, 再将各段分页, 再将内存空间分为大小相同的内存块/页框/页帧/物理块。



将进程按逻辑模块分段, 再将各段分页 (如每个页面4KB)
再将内存空间分为大小相同的内存块/页框/页帧/物理块

段页式的逻辑地址结构: 先分段后分页, 就是在段的段内地址进行一次再拆分, 拆分成页号和页内偏移量。

分段系统的逻辑地址结构由段号和段内地址 (段内偏移量) 组成。如:

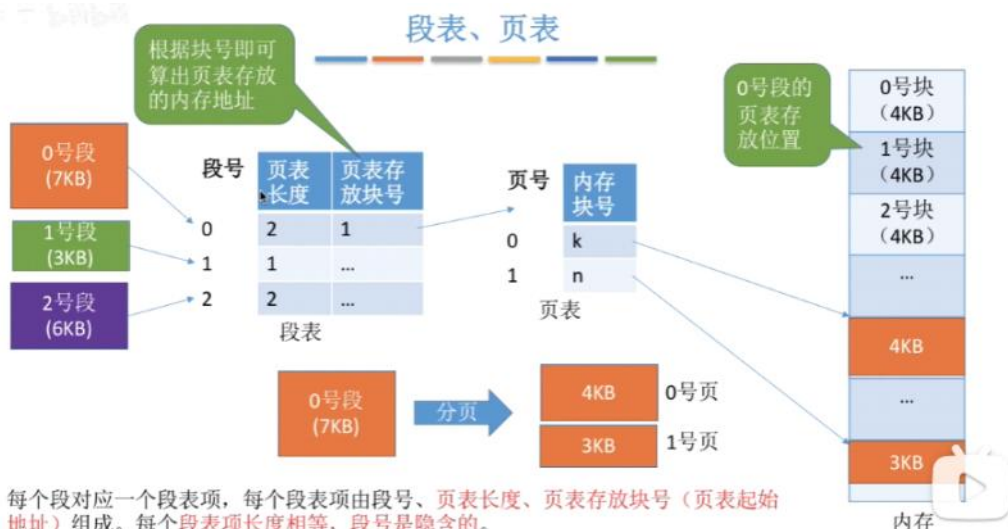
31		16	15		0
段号			段内地址		

段页式系统的逻辑地址结构由段号、页号、页内地址 (页内偏移量) 组成。如:

31		16	15		12	11		0
段号			页号		页内偏移量			

在上图中的段号决定可以分成多少段, 页号决定是段内分页后最多能分多少页, 页内偏移量决定了页面的大小。注意分页是系统行为, 用户不可见, 系统会根据段内地址自动划分页, 因此**段页式管理的地址结构是二维的!!**

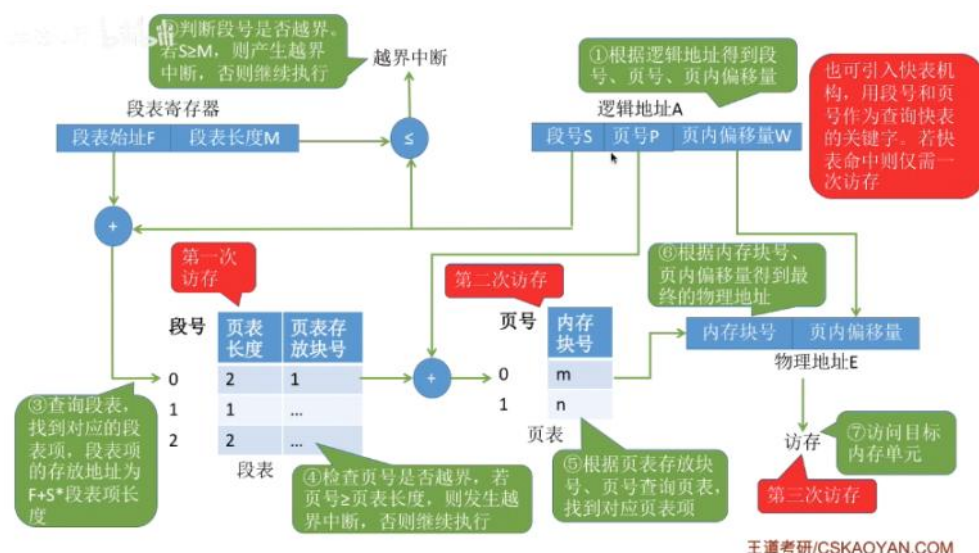
段表和页表: 知道段页式的逻辑结构后, 先给分段的设立一个段表, 再根据段表的长度决定划分成几个页, 将这几个页的数量和页号放在段表中; 然后再设置一个页表, 建立页号与物理块号的映射关系。



每个段对应一个段表项, 每个段表项由段号、页表长度、页表存放块号 (页表起始地址) 组成。每个段表项长度相等, 段号是隐含的。
每个页面对应一个页表项, 每个页表项由页号、页面存放的内存块号组成。每个页表项长度相等, 页号是隐含的。

地址转换过程:

王道考研/CSKAQYAN.CC



如上图，先根据逻辑地址求出段号页号和页内偏移，再将段号和段表长度对比查越界，没问题就查段表第一次访问，由于每个段划分的页数不一样，因此需要检查页号和页表长度是否越界，没问题就查对应的页表第二次访问，查完得到内存块号，加上页内偏移就得到位置，第三次访问访问目标单元。

四、内存空间的扩充——虚拟内存

之前讲过的连续分配非连续分配其实都out了，因为传统的存储管理都需要一次性全部装入内存才能开始运行，这样的话就有问题了，如果程序很大那么内存装不下，就只能有少量作业能运行，导致并发度下降（一次性）。还有一旦作业装入内存就会一直驻留在内存中，直到该作业结束（驻留性）。实际上一个时间段内只要一小部分数据就可以正常运行，就导致了内存中驻留了大量的数据，浪费内存资源。

1、虚拟内存的定义和特征

基于局部性的原理，可以让程序装入的时候先将必要的部分放入内存，其他不必要的就先留在外存，就可以让程序开始执行了；如果要访问的东西不在内存，就产生缺页中断，需要将这些东西从外存调入到内存然后继续执行程序；如果内存不够用了，就把内存的一些用不到的信息换出到外存。这样的话用户看来内存要比实际上的大的多。

总结下来虚拟内存有三个特征：多次性（不是一次全部装入内存）、对换性（不是常驻内存）、虚拟性（所见远大于实际）

2、虚拟技术的实现：虚拟内存技术需要建立在离散分配的内存管理方式的基础上（分页、分段、段页），加上虚拟技术就形成了请求分页、请求分段、请求段页的存储管理方式。

3、请求分页管理方式（=分页管理方式+虚拟技术）

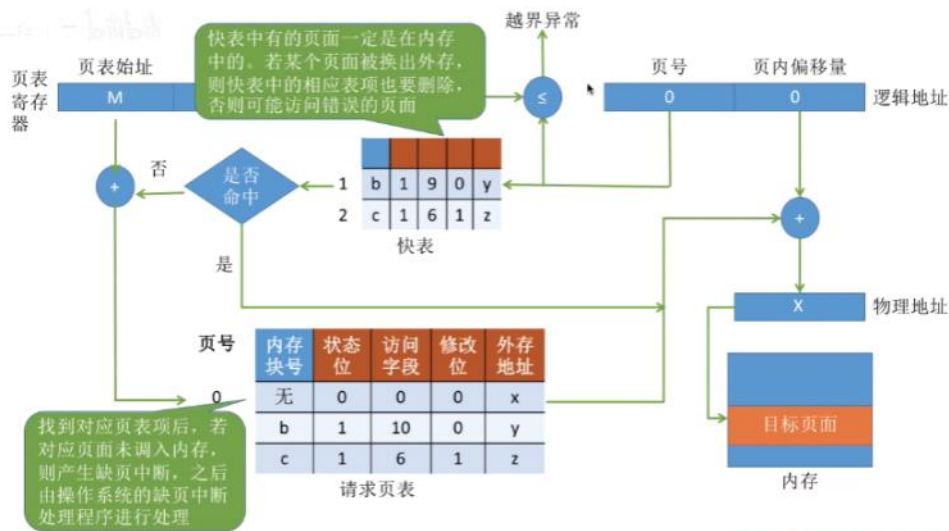
页表机制：和基本分页对比，请求分页为了实现“请求调页”，首先操作系统需要知道每个页面是否已经调入内存，如果还没调入，那么也得知道它在外存中的什么位置。当内存不够的时候要实现“页面置换”，操作系统需要通过某些指标来决定到底换出哪些页面，有的页面没有被修改过就不用写回外存直接去掉，有些页面被修改过就需要在外存中覆盖旧数据，因此需要记录各个页面是否被修改的信息。基于以上指标，请求分页存储管理的页表多了状态位（是否在内存中）、访问字段（最近访问过几次，供页面置换算法参考）、修改位（调入内存后是否被修改过）、外存地址（在外存中存放位置）。



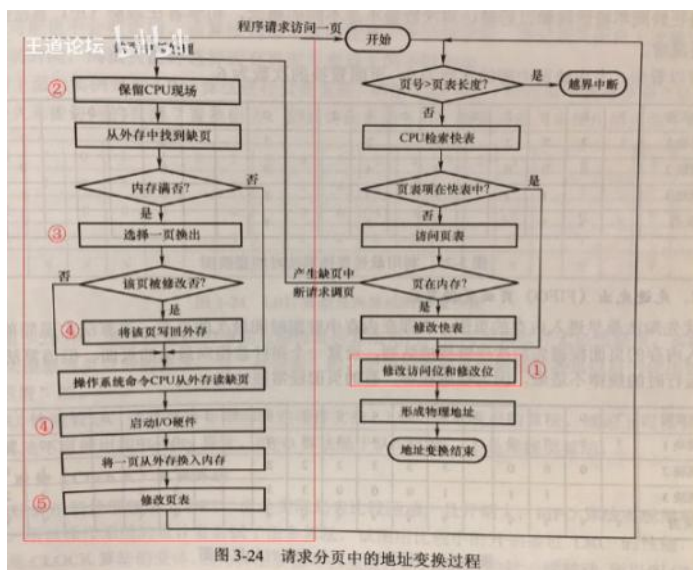
缺页中断机构：缺页中断机构就是专门修改页表的程序，当页面不在内存时产生缺页中断然后由中断处理程序处理，此时进程阻塞放入阻塞队列，调页完成后将其唤醒放回就绪队列；如果有空闲内存块，则分配一个空闲块；如果没有空闲内存块，则由页面置换算法选择一个页面进行淘汰；如果被修改过，就将其写回外存。

注意缺页中断属于内中断，一条指令可能会访问到多个内存单元，可能会产生多次缺页中断。有了缺页中断机构后才能实现请求调页功能。

地址变换机构：相比于基本分页，请求分页需要额外增加一些步骤：当访问信息不在内存时需要从外存调入内存（请求调页）、当内存空间不够的时候需要将暂时不在内存的信息换出到外存（页面置换）、需要修改页表中的选项。



王道考研/CSKAOYAN.COM



补充细节：

①只有“写指令”才需要修改“修改位”。并且，一般来说只需修改快表中的数据，只有要将快表项删除时才需要写回内存中的慢表。这样可以减少访问次数。

②和普通的中断处理一样，缺页中断处理依然需要保留CPU现场。

③需要用某种“页面置换算法”来决定一个换出页面（下节内容）

④换入/换出页面都需要启动慢速的I/O操作，可见，如果换入/换出太频繁，会有很大的开销。

⑤页面调入内存后，需要修改慢表，同时也需要将表项复制到快表中。

王道考研/CSKAOYAN.COM

4、页面置换算法

最佳置换算法OPT：选择淘汰未来永不使用或者未来长时间不使用的页面，保证最低缺页率。但是这个无法实现，因为未来无法预知。

在图中是向后看，最后出现的页号就淘汰掉。

例：假设系统为某进程分配了三个内存块，并考虑到有一下页面号引用串（会依次访问这些页面）：
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

访问页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
内存块1	7	7	7	2		2		2		2		2		2				7		
内存块2		0	0	0		0		4		0		0		0				0		
内存块3			1	1		3		3		3		3		1				1		
是否缺页	√	√	√	√		√		√		√		√		√				√		

选择从0, 1, 7中淘汰一页。按最佳置换的规则，往后寻找，最后一个出现的页号就是要淘汰的页面

整个过程缺页中断发生了9次，页面置换发生了6次。
注意：缺页时未必发生页面置换。若还有可用的空闲内存块，就不用进行页面置换。

先进先出置换算法FIFO：淘汰最早进入内存的页面。

访问页面	3	2	1	0	3	2	4	3	2	1	0	4
内存块1	3	3	3	3			4	4	4	4	0	0
内存块2		2	2	2			2	3	3	3	3	4
内存块3			1	1			1	1	2	2	2	2
内存块4				0			0	0	0	1	1	1
是否缺页	√	√	√	√			√	√	√	√	√	√

分配四个内存块时，缺页次数：10次
分配三个内存块时，缺页次数：9次

注意该算法虽然实现简单但是没有考虑到实际进程运行的规律（局部性原理），当进程分配的物理块增大时，缺页次数可能会不降反增，这种情况叫Belady异常。

最近最久未使用置换算法LRU：淘汰最近的最久没有使用过的页面，它的实现需要硬件的支持，需要设置一个t值表示该页面自从上次访问以来所经历的时间，在淘汰一个页面的时候选择t最大的淘汰掉。

访问页面	1	8	1	7	8	2	7	2	1	8	3	8	2	1	3	1	7	1	3	7
内存块1	1	1		1		1					1						1			
内存块2		8		8		8					8						7			
内存块3				7		7					3						3			
内存块4						2					2						2			
缺页否	√	√		√		√					√						√			

在手动做题时，若需要淘汰页面，可以逆向检查此时在内存中的几个页面号。在逆向扫描过程中最后一个出现的页号就是要淘汰的页面。

该算法是最接近OPT的，但是需要专门的硬件支持，算法开销很大。

时钟置换算法CLOCK（或者最近未用算法NRU）：LRU很好但是开销大，FIFO实现简单但是性能差，因此折中后形成CLOCK算法。该算法就是给每一个页面设置一个访问位，当它最初进入内存或者被访问过后设置成1；当该页面被扫描时将会变成0。然后将内存中的页面通过链接指针链接成一个循环队列，当内存块满了需要置换时，从队首开始扫描，会优先找出第一个访问位为0的页面淘汰掉。

页号	内存块号	状态位	访问位	修改位	外存地址
----	------	-----	-----	-----	------

访问位为1，表示最近访问过；
访问位为0，表示最近没访问过

例：假设系统为某进程分配了五个内存块，并考虑到有以下页面号引用串：
1, 3, 4, 2, 5, 6, 3, 4, 7

页号	内存块号	状态位	访问位	修改位	外存地址
----	------	-----	-----	-----	------

访问位为1，表示最近访问过；
访问位为0，表示最近没访问过

例：假设系统为某进程分配了五个内存块，并考虑到有以下页面号引用串：
1, 3, 4, 2, 5, 6, 3, 4, 7

页号	内存块号	状态位	访问位	修改位	外存地址
----	------	-----	-----	-----	------

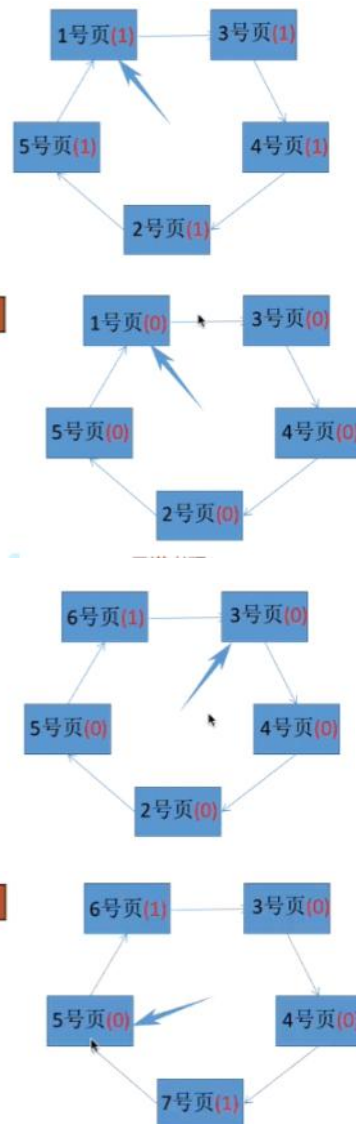
访问位为1，表示最近访问过；
访问位为0，表示最近没访问过

例：假设系统为某进程分配了五个内存块，并考虑到有以下页面号引用串：
1, 3, 4, 2, 5, 6, 3, 4, 7

页号	内存块号	状态位	访问位	修改位	外存地址
----	------	-----	-----	-----	------

访问位为1，表示最近访问过；
访问位为0，表示最近没访问过

例：假设系统为某进程分配了五个内存块，并考虑到有以下页面号引用串：
1, 3, 4, 2, 5, 6, 3, 4, 7



改进型的时钟置换算法：简单的时钟置换算法只考虑了一个页面最近是否被访问过，实际上在其他条件相同的时候可以优先淘汰没有修改过的页面，这样就不用额外花时间写外存，能加快速度。因此该算法新增一个修改位用于表示页面是否被修改过。例如在（访问位，修改位）中（1，1）表示这个页面最近被访问过也被修改过。

综合考虑访问位和修改位，形成了四轮淘汰的优先级：

算法规则：将所有可能被置换的页面排成一个循环队列

第一轮：从当前位置开始扫描到第一个(0,0)的帧用于替换。本轮扫描不修改任何标志位

第二轮：若第一轮扫描失败，则重新扫描，查找第一个(0,1)的帧用于替换。本轮将所有扫描过的帧访问位设为0

第三轮：若第二轮扫描失败，则重新扫描，查找第一个(0,0)的帧用于替换。本轮扫描不修改任何标志位

第四轮：若第三轮扫描失败，则重新扫描，查找第一个(0,1)的帧用于替换。

由于第二轮已将所有帧的访问位设为0，因此经过第三轮、第四轮扫描一定会有一个帧被选中，因此改进型CLOCK置换算法选择一个淘汰页面最多会进行四轮扫描

第一优先级：最近没访问，且没修改的页面

第二优先级：最近没访问，但修改过的页面

第三优先级：最近访问过，但没修改的页面

第四优先级：最近访问过，且修改过的页面

因此选择一个淘汰页面最多会进行四轮扫描。

5、页面分配、置换

驻留集：指的是分页存储管理中给进程分配物理块的集合，在虚拟技术的系统中驻留集一般小于进程总大小，如果大于就不存在缺页了，但是这样的话虚拟性就没了，会导致并发度下降资源利用率低，同理过小会导致频繁换页。

固定分配：运行期间驻留集大小不变

可变分配：运行期间驻留集大小可变

局部置换：发生缺页的时候只能选进程自己的物理块置换

全局置换：发生缺页的时候操作系统保留的空闲物理块分给它或者别的进程持有的物理块置换到外存。

注意全局置换必须是可变分配。

	局部置换	全局置换
固定分配	√	—
可变分配	√	√

全局置换意味着一个进程拥有的物理块数量必然会改变，因此不可能是固定分配

6、页面分配、置换策略

固定分配局部置换：缺点是很难再刚开始的时候确定要分配多少物理块才合理，灵活性低。

可变分配全局置换：当系统没有空闲物理块的时候就从其他进程中薅出一个物理块给他用，但是对于被薅的程序来说，物理块会减少缺页会增加。

可变分配局部置换：折中方案，当进程缺页的时候只允许自己的物理块选一个换出外存，当频繁换页的时候系统额外给他分配物理块直到不频繁换页；反之当缺页率过低的时候系统也会抽出空闲物理块。

注意后两者的区别：可变分配全局置换就是一有缺页就分配新物理块，可变分配局部置换时根据缺页的频率来动态增加或者减少物理块。

7、何时调入页面

预调页策略（运行前调入）：根据局部性原理，如果一次性调入几个相邻的页面，可能比一次性调入一个页面更高效，因此可以先预测不久之后可能访问到的页面预先调入内存，但是预测成功率仅仅百分之50。主要用于进程的首次调入，由程序员指出先调入哪些部分。

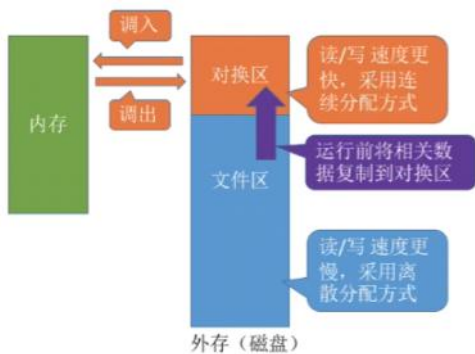
请求调页策略（运行期间调入）：在运行期间发现缺页的时候才将缺少的页面调入内存，但是由于每次只能调入一页且每次都要磁盘IO操作，因此IO开销大。

8、从何处调入页面

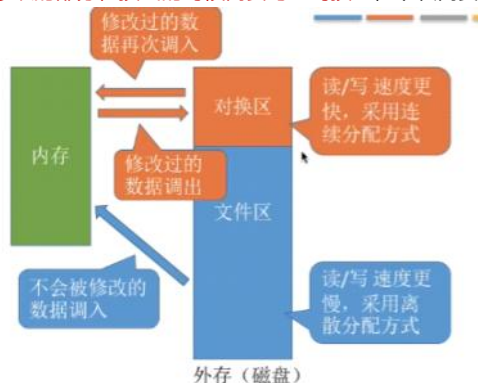
从外存调入，外存由对换区和文件区组成，对换区空间小速度快，文件区空间大速度慢。



如果系统拥有足够的对换区空间，那么页面的调入和调出都是在内存与对换区之间进行来保证页面的调入调出速度都很快。在进程运行前需要将相关的数据从文件区复制到对换区。



如果系统没有足够的对换区空间，凡是不会被修改过的数据都是直接从文件区调入，这些文件换出的时候不必写回磁盘，下次直接调入即可。对于可能会被修改的部分，换出的时候需要写入对换区，下次需要直接从对换区调入。



对于UNIX方式，就是在运行之前进程有关的数据全部放在文件区，故未使用过的页面都是从文件区调入，换出时写入对换区，下次需要就直接在对换区调入。

9、抖动（颠簸）现象

刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出外存，这种频繁的现象称为抖动或颠簸。产生这个原因主要就是分配的物理块太少，但是物理块太多又会失去虚拟性，因此需要研究为每个进程分配多少物理块。

工作集：在某段时间间隔内进程实际访问页面的集合，工作集是根据窗口尺寸确定。

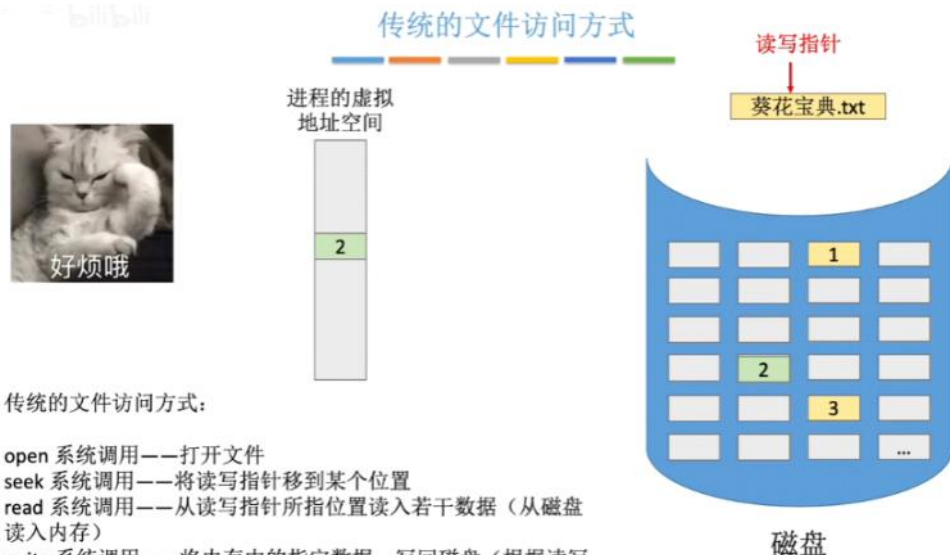
某进程的页面访问序列如下，窗口尺寸为4，各时刻的工作集为？



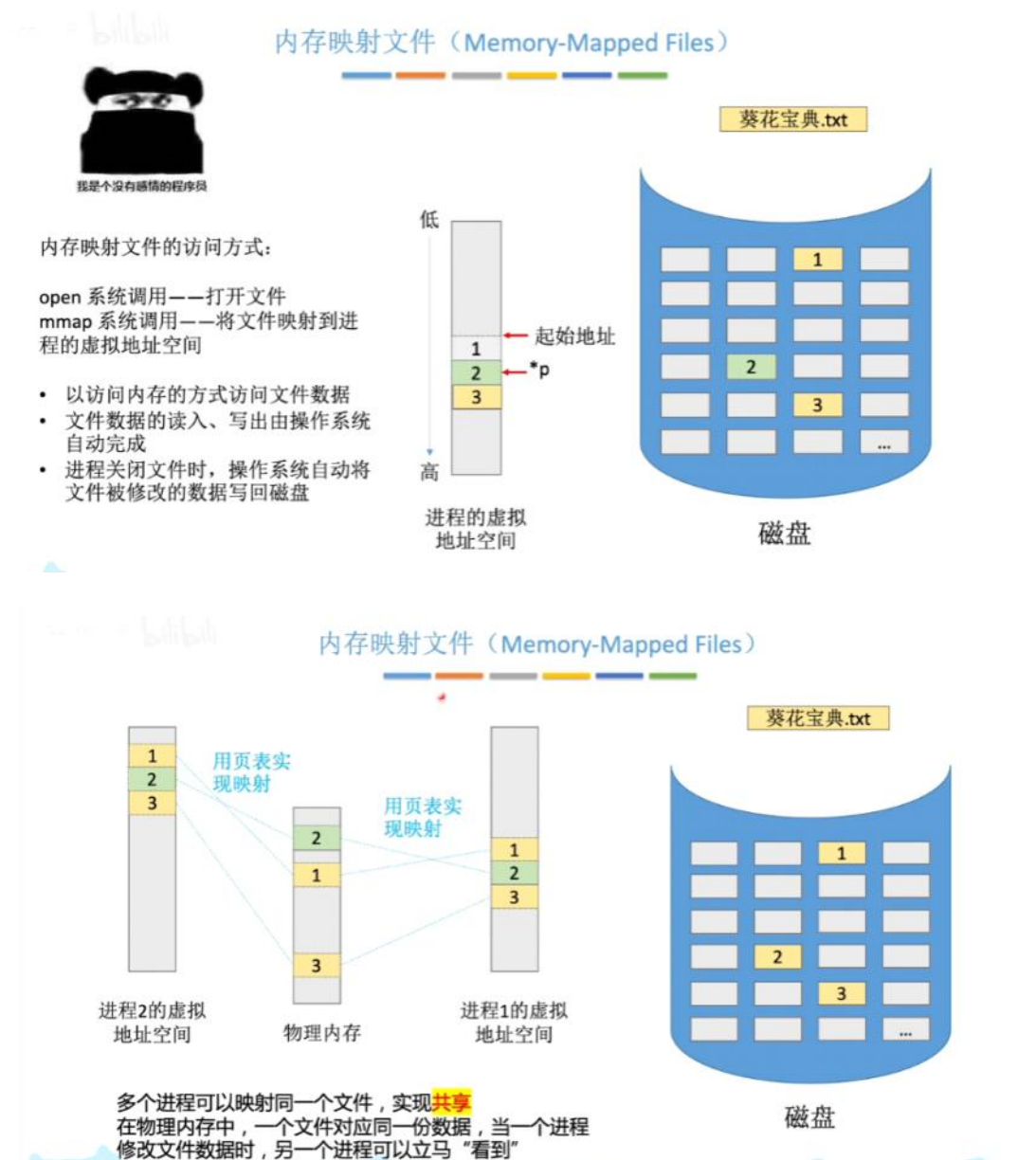
工作集大小可能小于窗口尺寸，在实际应用中操作系统可以统计驻留集大小，一般来说驻留集大小不能小于工作集大小，否则将会出现频繁换页。

10、内存映射文件

这玩意就是操作系统向上层程序员提供的系统调用，能方便程序员访问文件数据，还能方便多个进程共享同一文件。



如上图，一个txt文件拆成多个页面离散放入内存中，一个个找很麻烦。



文件管理

2025年6月9日星期一 上午9:44

文件管理自问自答

文件有哪些属性啊，什么属性最重要？

无结构文件有结构文件区别是什么？

逻辑上文件的组织形式有哪些？

文件目录是什么？文件目录的操作有哪些？文件目录的结构有哪些？

物理上文件的组织形式有哪些？FAT是什么？

计算机磁盘的划分？管理空闲盘快的方法有哪些？

文件有哪些操作，具体实现的系统调用是什么？

文件共享有哪几种方式？

文件保护有哪几种方式？

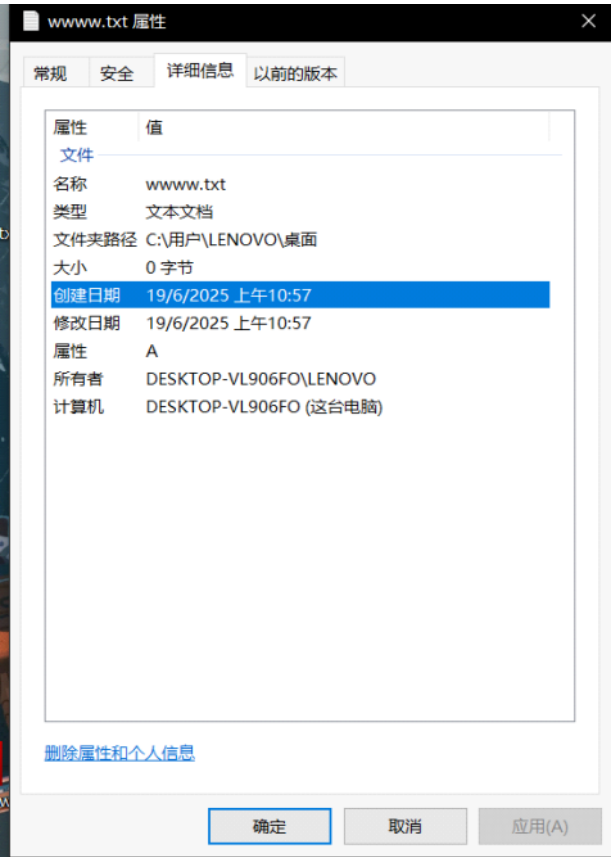
虚拟文件系统VFS是干啥的？

一、初识文件系统

文件就是一组有意义的信息或数据的集合。

1、文件的属性

如下图，一个文件的属性包括文件名



文件名：创建的用户决定文件名方便后面找到该文件，同一个目录下不能有重名的文件（最重要的信息）

标识符：操作系统用于区分不同文件的内部名称，用户看不到的

类型：操作系统可以为不同类型的文件设置一个默认打开的软件，比如txt用记事本，mp4用视频播放器打开等等

位置：文件的存放路径，用户可以通过存放路径来一步步找到文件，用户双击文件的时候操作系统要把外存中的文件放到内存，此时外存中的地址有必要知道

保护信息：对文件进行保护的访问控制信息，设置好保护信息就可以让文件只有自己的电脑账户访问

除了这些还有文件大小、创建时间、修改时间、文件所有者这些必要信息。

2、文件的组织形式

无结构文件：仅仅由一些二进制或者字符流组成，又称流式文件；如txt文件

有结构文件：由一组相似的记录组成，带结构又称记录式文件；如excle文件；其中记录是一组相关数据项的集合，例如学号姓名这些记录；数据项是文件中最基本的数据单位，根据各条记录的长度是否相等可以分为定长记录（int,char这些）和可变量记录(varchar这些)文件。

学号	姓名	性别	专业
1120112100	张三	男	挖掘机
1120112101	李四	女	挖掘机
1120112102	王五	男	数据挖掘
1120112103	赵六	男	挖掘机
1120112104	钱七	女	挖掘机
1120112105	狗剩	男	数据挖掘
1120112106	铁柱	女	数据挖掘
1120112107	如花	女	数据挖掘
1120112108	二狗	男	数据挖掘
1120112109	傻根儿	男	数据挖掘
1120112110	旺财	女	数据挖掘

32 B 学号	32 B 姓名	4 B 性别	60 B 专业
------------	------------	-----------	------------

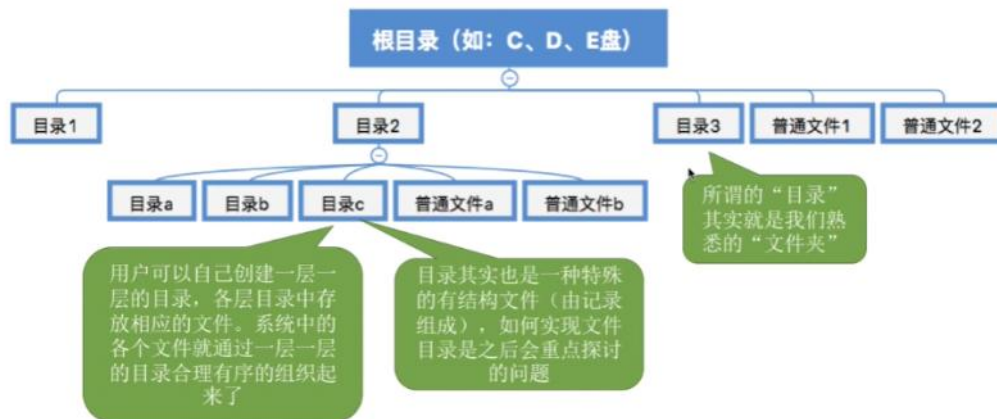
这个有结构文件由**定长记录**组成，每条记录的长度都相同（共 128 B）。各数据项都处在记录中相同的位置，具有相同的顺序和长度（前32B一定是学号，之后32B一定是姓名……）

学号	姓名	性别	特长
1120112100	张三	男	腿特长
1120112101	李四	女	腿毛特长
1120112102	王五	男	
1120112103	赵六	男	
1120112104	钱七	女	
1120112105	狗剩	男	
1120112106	铁柱	女	
1120112107	如花	女	
1120112108	二狗	男	熟读唐诗三百首，琴棋书画样样精通，上得了厅堂下得了厨房，精通Java、C++、Python和任何一种脚本语言~(后面还有1万字……)
1120112109	傻根儿	男	
1120112110	旺财	女	

32 B 学号	32 B 姓名	4 B 性别	(长度不确定) 特长
------------	------------	-----------	---------------

这个有结构文件由**可变长记录**组成，由于各个学生的特长存在很大区别，因此“特长”这个数据项的长度不确定，这就导致了各条记录的长度也不确定。当然，没有特长的学生甚至可以去掉“特长”数据项。

文件之间应该怎样组织起来？



二、文件的逻辑结构

根据文件之间的各条记录在逻辑上的组织，可以分为顺序文件、索引文件、索引顺序文件三种。

1、顺序文件

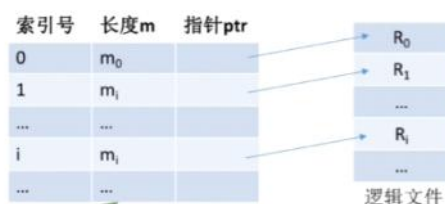
文件中的记录在逻辑上一个接一个的顺序排列，记录可以是**定长**也可以是**可变长**的。各个记录在物理上可以是**顺序存储**和**链式存储**，顺序存储中逻辑上相邻那么在物理上也相邻，链式存储在逻辑上相邻的物理上不一定相邻。

顺序文件又可以分为**串结构**和**顺序结构**，串结构记录之间的顺序与关键字无关，通常按存入的时间排序；顺序结构记录之间的顺序按照关键字的顺序排列。

是否可以实现随机存取？首先链式存储是肯定不能实现的，因为链表只能从头到尾查找；如果是顺序存储的话，可变长的无法实现随机存取，因为每个记录的大小不一样无法通过计算偏移量的方式来随机存取；如果是顺序存储定长记录是可以实现随机存取的，如果是顺序存储顺序结构，可以通过关键字快速找到记录（例如二分查找），如果是串结构就无法快速找到关键字对应的记录。

2、索引文件

可变长记录无法实现随机存取，查找速度慢，此时就需要索引文件来支持了。索引文件建立了一张**索引表**，每个记录对应一条表项，每个记录在物理上可以离散存放，但是索引表之间的表项必须连续存放来保证随机存取。另外要查找的话还可以根据索引号来实现二分查找等。还可以为其他关键字快速的建立一张索引表，就像SQL中的主键可以给学号或者姓名之类的。



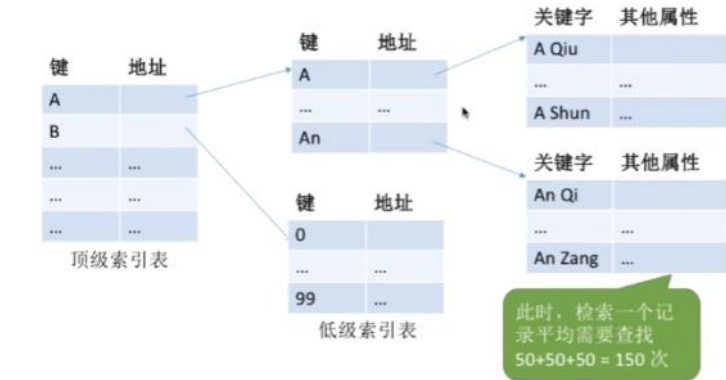
注意每当增删一个记录的时候就需要及时的修改索引表，因为索引表的检索速度很快，因此索引表**主要用于对信息处理及时性要求高的场合**。而且每个记录对应一个索引项，因此**索引表可能会很大**，空间的利用率就低了。

3、索引顺序文件

索引顺序文件就是给一堆记录进行分组，例如名字A开头一组B开头一组等。分组后形成一张索引表，每个表项对应一个小的索引表，这样查找的话可以通过**分块查找**来提高效率（例如顺序文件10000记录，则顺序查找平均找5000个，如果是索引顺序文件，就分成100组每组对应100个记录，分组的大表项查找平均查50次，小表项平均查找50次，就平均查5+50=100个）。



如果有1000000条记录，算下来还是平均查1000次依然很多，此时可以建立多级索引顺序文件来给文件瘦身。



三、文件目录——文件夹

文件目录的本质就是一个文件，txt文件类型是txt，新建文件夹的文件类型是目录。

1、文件控制块FCB

FCB的有序集合称为“文件目录”，如下图的表格就是一个文件目录下的所有FCB。FCB包含的信息包括文件名、物理地址、逻辑结构、物理结构等**基本信息**，还包括可读/可写、访问黑名单这些**存取控制信息**、还有建立时间、修改时间这些**使用信息**，但是**最重要最基本的**还得是**文件名、文件存放的物理地址**。FCB实现了文件名与文件之间的映射，可以让用户实现“按名存取”。

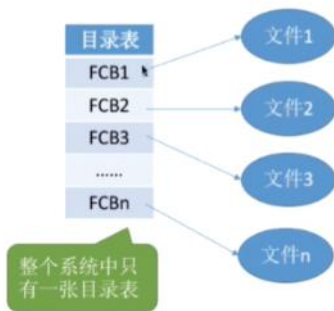
文件名	类型	存取权限		物理位置
2015-08	目录	只读	...	外存25号块
2015-09	目录	读/写	...	外存278号块
.....			...	
2016-02	目录	读/写	...	外存152号块
.....			...	
微信截图_20180826102629	PNG	只读	...	外存995号块

2、文件目录的操作

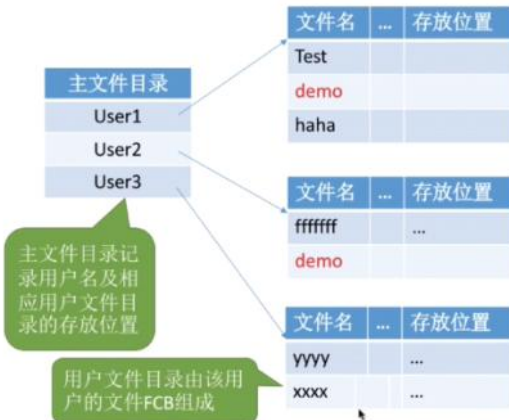
- 搜索**：用户要用一个文件需要先根据文件名找到他对应的目录项。
- 创建文件**：创建新文件需要再所属目录中增加一个目录项
- 删除文件**：删除文件需要再目录中删除对应的目录项
- 显示目录**：用户可以请求查看目录中的内容，例如显示目录中的所有属性
- 修改目录**：文件属性保存在目录中，属性在变化时（例如文件重命名）需要修改目录项。

3、目录结构

单级目录结构：整个系统就1张目录表，每个文件占用一个目录项，只适用于早期操作系统，缺点很明显是**文件不允许重名**。



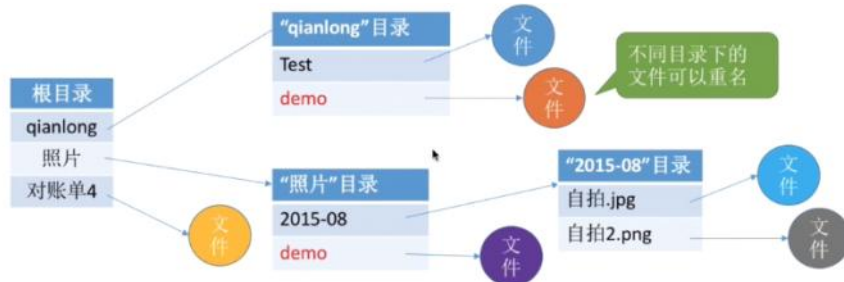
两级目录结构：为了应对多用户操作系统，提出两级目录结构。分为**主文件目录**和**用户文件目录**，主文件目录记录用户名及其对应文件目录的位置，每个用户占用一个目录项，用户文件目录由用户文件的FCB组成；在这个结构不同用户的文件名可以相同，也可以实现不同用户之间的访问限制。



缺点就是**用户无法对自己的文件进行分类**，无法新建文件目录。

多级目录结构（树形目录结构）：用户要访问某个文件要用**文件路径名**标识文件，文件路径名就是个字符串，用/隔开，分为**从根目录出发的绝对路径**和从**当前目录出发的相对路径**。例如D:\翁飞\获奖证书\1.jpg就是一个文件路径名，操作系统在找到这个文件时需要先**从外存读入根目录d的目录表**，找到“翁飞”目录的存放位置后，**从外存读入对应的目录表**；再找到“获奖证书”目录的存放位置后**再次从外存读入对应目录表**；最后才找到这个jpg文件，这个过程**需要进行3次读磁盘操作**。如果多次查询同一文件夹下的不同文件，可以设置一个“**当前目录**”来提升查找效率。

注意**树形结构下的文件被打开后，使用文件描述符fd来实现对文件的访问**。



树形结构方便分类，层次清晰，能有效进行文件的管理和保护。但是树形结构**不便于实现文件的共享**，因此有了**无环图目录结构**。

无环图目录结构：这玩意其实就是在**树形的基础上加一些指向同一节点的有向边**，使得整个目录变成一个**有向无环图**，方便共享。可以用不同的文件名指向同一个文件，甚至是同一个目录。



引入共享后删除就没这么简单了，需要删除时需要设置一个共享计数器，一个用户删除的时候只会删除该用户的FCB，计数器-1，当计数器为0的时候才会真正删除掉这个共享节点。

四、文件的物理结构

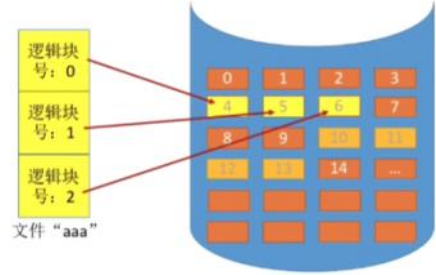
文件的物理结构是由存储介质决定的，例如磁带只能顺序存取连续分配，而磁盘可以随机存取离散分配。

1、文件块与磁盘块

在内存管理中，进程的逻辑地址空间被分为一个个页面，同样在外存中，文件的逻辑地址空间被划分成一个个的块，和页表一样文件的逻辑地址可以由（逻辑块号、块内地址）表示。

2、文件的分配方式——连续分配

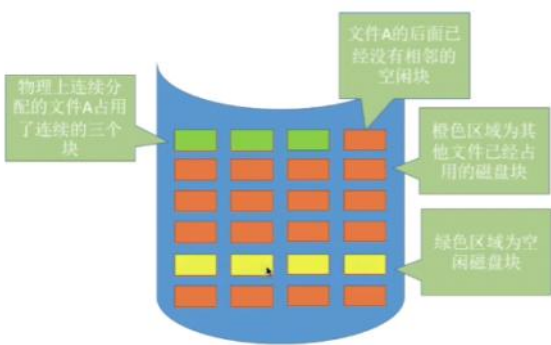
特点是逻辑上相邻物理上也相邻，如果将逻辑地址转换成物理地址，那就是（逻辑块号，块内地址）转换成（物理块号，块内地址），只需要转换块号即可，块内地址保持不变。



地址转换借助文件目录实现，文件目录保存了文件的起始物理块号以及长度，起始块号+逻辑块号即可找到对应的目录项。例如要找文件aaa的第二个逻辑块，查下面的目录表后发现起始块号是4，因此访问的就是4+2=6号物理块。当然要检查长度和逻辑块号是否越界。

文件名	起始块号	长度
aaa	4	3
bbb	10	4
.....	...	...

连续分配的优点就是支持顺序访问和随机访问，访问速度快；并且磁盘移动磁头的距离也短，访问速度快。



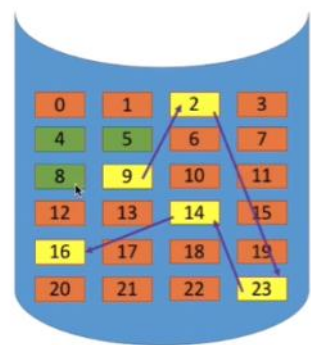
缺点是文件的扩展不方便，如果要增加块但是后面的块被占用了，为了保证连续性就必须整体进行迁移，开销很大。并且会产生磁盘碎片，需要紧凑技术解决，花费很大的时间代价。

3、文件分配方式——链接分配

采用离散分配的方式，可以为文件分配离散的磁盘块，有显式链接和隐式链接两种方式。

隐式链接的方式类似于链表，目录中记录了文件存放的起始块号和结束块号，除了文件的最后一个磁盘块之外，每个磁盘块都会保存指向下一个盘块的指针，这些指针用户不可见。采用这个方式就只能顺序访问，查找效率低；另外指向下一个盘快的指针也需要耗费存储空间。但是它扩展的效率高啊，直接修改结束块号的指针即可，不会有碎片问题。

文件名	起始块号	结束块号
aaa	9	16



显式链接的方式设置了一个文件分配表FAT，记录了每一个物理块对应的下一块号，每个磁盘仅仅设置一个FAT，开机时FAT读入内存并常驻内存。FAT在物理上是连续存储的并且每个表项的长度相同，因此可以隐含物理块号节省空间。



如果要地址转换，需要先查询FAT表，往后找到i号逻辑块号对应的物理块号，访问FAT的过程可以随机访问，转换的过程不需要读写磁盘。因此它支持顺序和随机访问，访问速度快很多，没有外部碎片方便扩展。唯一缺点就是FAT占用一些存储空间。

注意如果题目只说了链接分配，那就默认隐式链接。

4、文件分配方式——索引分配

索引分配的也是离散存储在各个磁盘之中，系统为每个文件建立一张类似于页表的索引表，索引表记录了每个逻辑块对应的物理块，其中索引表存放的磁盘块为索引块、文件数据存放的磁盘块为数据块。



假设某个新建的文件“aaa”的数据依次存放在磁盘块 2 → 5 → 13 → 9。7号磁盘块作为“aaa”的索引块，索引块中保存了索引表的内容。

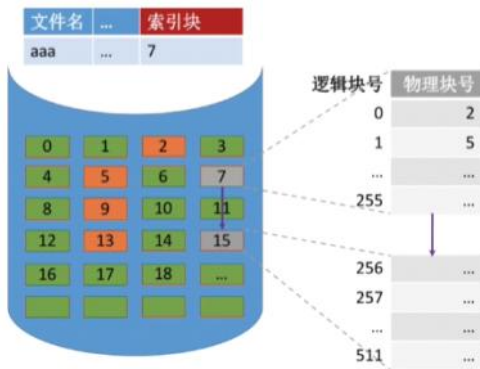
如上图，文件aaa的索引块是7，那么索引表就存在7号磁盘块，在7号磁盘块中有一张索引表，里面记录了aaa文件的数据存放的位置，也就是说数据存放于2、5、13、9号磁盘块。

和FAT的区别就是FAT表整个磁盘就一张，而索引表每个文件对应一张。

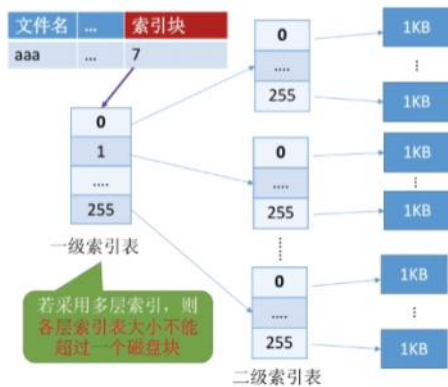
索引分配的地址转换就是先从文件FCB中找到索引块，再访问磁盘索引块查索引表，有了索引表就可以通过逻辑块号转换成物理块号，注意索引表连续且大小一样且逻辑块号有序，支持随机访问，文件的扩展也很方便，只需要给文件分配一个空闲块，改一下索引表即可。缺点是索引表占空间。

有个问题就是当文件大小超过了256个块，那么一个磁盘块装不下怎么办，此时有2种方案解决，分别是链接方案、多层索引、混合索引。

链接方案：把多个索引块用指针链接起来，那么文件只需要记录第一个索引块的磁盘号，如果要查后面的索引块就只能顺序查找，按顺序一个个遍历，查找速度慢，读取磁盘IO次数多。



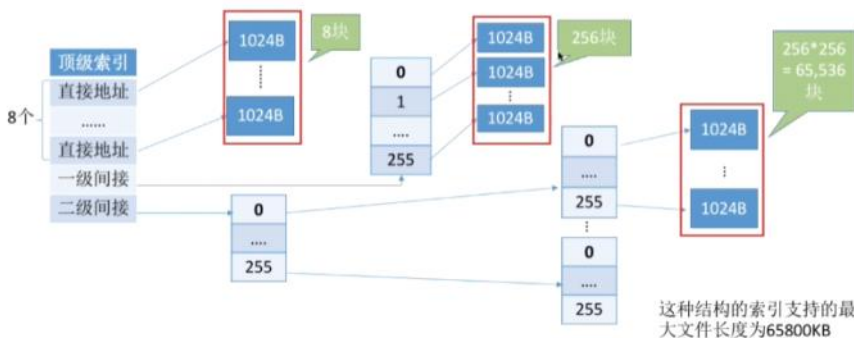
多层索引方案：类似于多级页表，建立多层索引，一个磁盘块可以存放256个索引项，那么n层索引最大可以存储256的n次方个索引项。



查找的话只需要相除取余即可精准定位, 例如要查1026号逻辑块, 那么是第 $1026/256=4$ 号二级索引表的 $1026\%256=2$ 号索引项对应的磁盘块号。此时需要3次磁盘IO操作, 第一次读取一级索引表、第二次读取二级索引表、第三次读取磁盘块号。如果是k层索引表, 那么是k+1次磁盘IO操作。

如果需要读取1号逻辑块, 那么每次读取都需要3次磁盘IO, 效率低, 因此又有了折中方案。

混合索引方案: 一个文件的顶级索引表除了包含直接地址索引(直接指向数据块), 又包含一级间接索引(1层索引表), 还包含两级间接索引(2层索引表)等。如下图, 直接地址索引占用8块磁盘块, 1级占用256块, 2级占用256的平方次块, 依次类推可以加法得出一个顶级索引指向了多少个数据块。



在磁盘IO次数上, 直接地址需要查顶级索引和读取磁盘块数据, 2次IO; 1级间接需要查顶级索引、1级索引表、读取磁盘块数据总共3次IO; 2级间接需要查顶级索引、1级索引表、2级索引表、读取磁盘块数据总共4次IO; 因此n级间接需要n+2次磁盘IO操作。

对于小文件来说这个是有利的, 访问一个数据块所需要的IO次数更少。

	How?	目录项内容	优点	缺点
顺序分配	为文件分配的必须是连续的磁盘块	起始块号、文件长度	顺序存取速度快, 支持随机访问	会产生碎片, 不利于文件拓展
链接分配	隐式链接 除文件的最后一个盘块之外, 每个盘块中都存有指向下一个盘块的指针	起始块号、结束块号	可解决碎片问题, 外存利用率高, 文件拓展实现方便	只能顺序访问, 不能随机访问。
	显式链接 建立一张文件分配表(FAT), 显式记录盘块的先后关系(开机后FAT常驻内存)	起始块号	除了拥有隐式链接的优点之外, 还可通过查询内存中的FAT实现随机访问	FAT需要占用一定的存储空间
索引分配	为文件数据块建立索引表。若文件太大, 可采用链接方案、多层索引、混合索引	链接方案记录的是第一个索引块的块号, 多层/混合索引记录的是顶级索引块的块号	支持随机访问, 易于实现文件的拓展	索引表需占用一定的存储空间。访问数据块前需要先读入索引块。若采用链接方案, 查找索引块时可能需要很多次读磁盘操作。

五、文件的存储空间管理

1、存储空间的划分和初始化

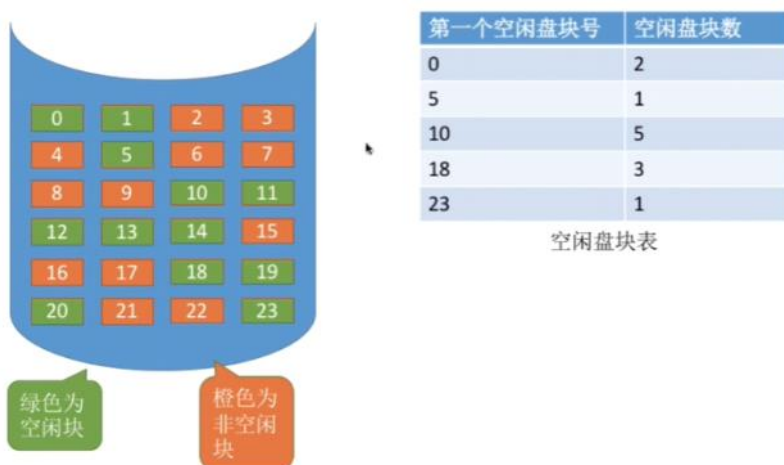
计算机的磁盘是可以划分的, 划分成多个文件卷(新加卷)例如C盘E盘F盘这些, 其中每个文件卷由目录区和文件区组成, 目录区存储文件的目录信息FCBI以及磁盘的空间管理信息等; 文件区则存放文件数据。

安装 Windows 操作系统的时候，一个必经步骤是——为磁盘分区（C: 盘、D: 盘、E: 盘等）



2、空闲表法

该方法就是设置一个空闲盘快表，记录了磁盘中空闲盘的其实地址以及块数。

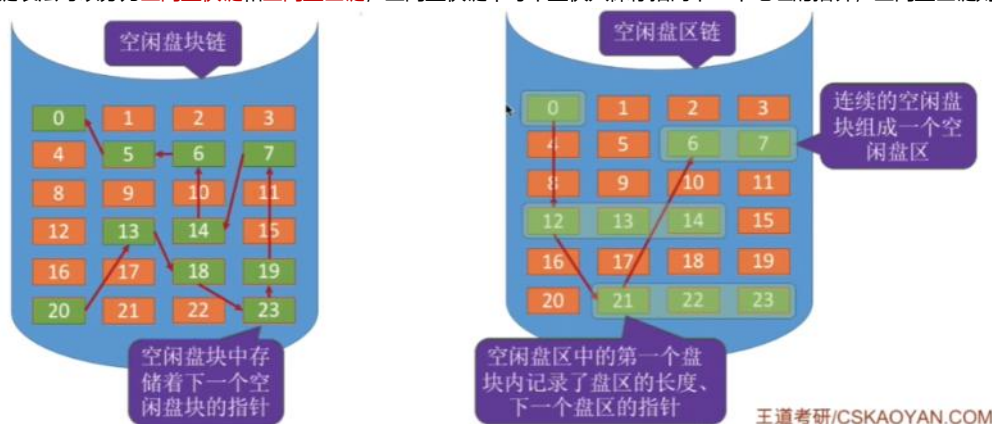


这种方式适合连续分配方式，分配一个连续的空间就和内存管理的一样，可以采用首次适应、最佳适应、最坏适应等算法来决定给文件分配哪些空间。

回收磁盘块的话也和连续分配动态分区一样，分成四种情况来修改空闲盘快表：回收区前后没相邻空闲区、回收区前后都是空闲区、回收区前面是空闲区、回收区后面是空闲区。

3、空闲链表法

空闲链表法可以分为空闲盘快链和空闲盘区链，空闲盘快链中每个盘快只保存指向下一个地址的指针，空闲盘区链则多了个本盘区的长度。

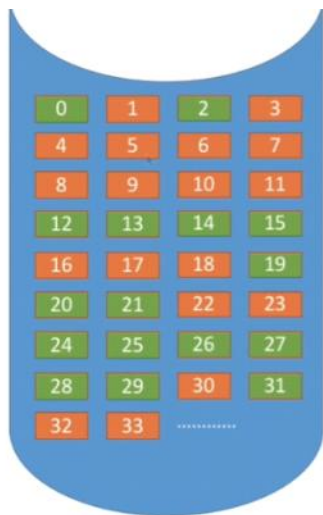


空闲盘快链中操作系统保存着链头、链尾指针，分配的K个盘快时候从链头开始依次摘下K个盘快进行分配，并修改空闲链的链头指针即可，回收的话是将空闲盘快依次接到链尾并修改链尾指针。适用于离散分配的物理结构，分配多个盘快时需要一个一个的摘下来。

空闲盘区链中操作系统也是保存链头链尾指针，分配K个盘快的时候像内存管理的首次适应、最佳适应等算法从链头开始找到合适大小的空闲盘区，如果没有单个空闲盘区装得下，就将不同盘区同时分配给一个文件，注意要修改相应的链指针、盘区大小等数据。回收的时候需要考虑空闲盘区合并的问题，如果后面有相邻的盘区就合并，没有的话就作为单独的一个分区挂到链尾。支持连续分配离散分配，为一个文件分配多个盘快的效率更高。

4、位示图法

设置一个位示图，类似于一个二维数组，每个单元对应一个盘快的状态，0表示空闲1表示已分配。位示图用连续的“字”来表示，每个字对应16个盘快。



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	0	1	1	1	1	1	1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	1	1	0	0	0	0	0	0	1	0
2	1	1	...													
...																

位示图：每个二进制位对应一个盘块。在本例中，“0”代表盘块空闲，“1”代表盘块已分配。位示图一般用连续的“字”来表示，如本例中一个字的字长是16位，字中的每一位对应一个盘块。因此可以用（字号，位号）对应一个盘块号。当然有的题目中也描述为（行号，列号）

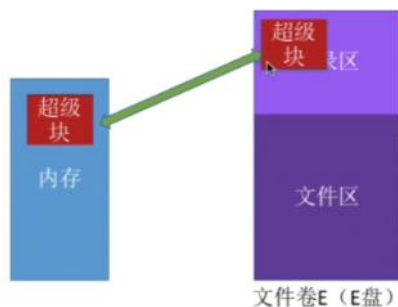
因此可以用盘块号推出字号和位号，也可以反推：（字号，位号）的二进制位对应盘块号 $b=n*i+j$ ，例如下标从1开始的(1,2)表示第 $1*16+2=18$ 号盘块。b号盘块对应的字号 $i=b/n$ 位号 $j=b\%n$ ，例如22号盘块对应 $(22/16=1, 22\%16=6)=(1,6)$ 号盘块。

分配的时候如果需要分配k个块，则顺序扫描位示图找出k个相邻或不相邻的0，算出对应的盘块号分配出去即可，最后状态从0变成1。

回收的时候先根据盘块号算出对应的字号位号，将状态从1设置为0即可。

5、成组链接法

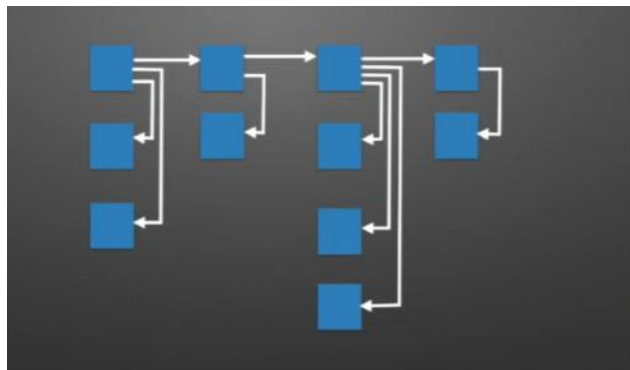
空闲表法和空闲链表法不适用于大型文件系统，因为设置的表可能会很大。在UNIX系统中采用了成组链接法对空闲块进行管理。在磁盘目录区设置一个超级块，启动的时候读入内存并保持内外存的一致。



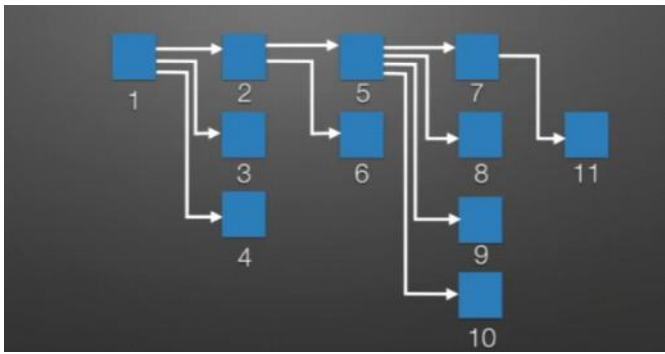
如下图是空闲盘区链，有个问题就是如果没有相邻的空闲磁盘块时依然会很长的链表。



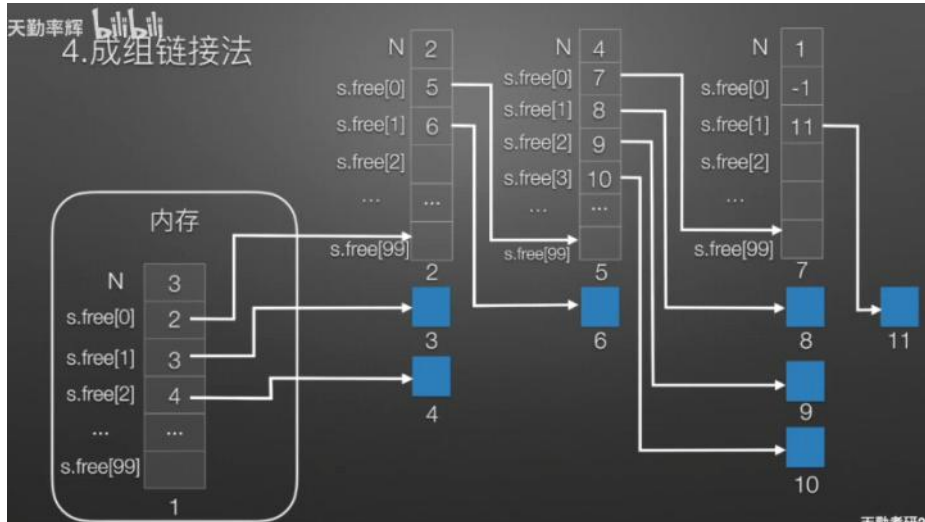
那么就变成了这样，每个盘区的区首不仅指向下一个盘区，还指向自己的盘区内的所有盘快



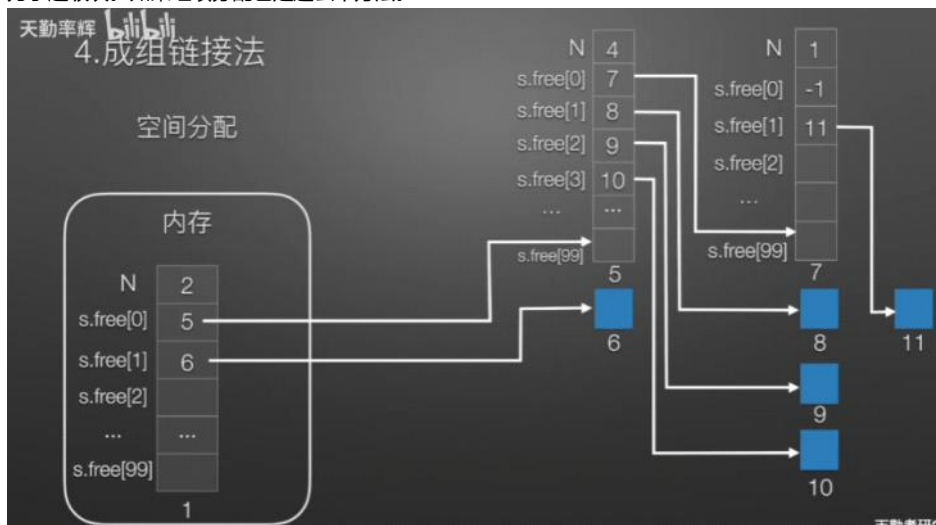
变下结构就有了一张索引表。



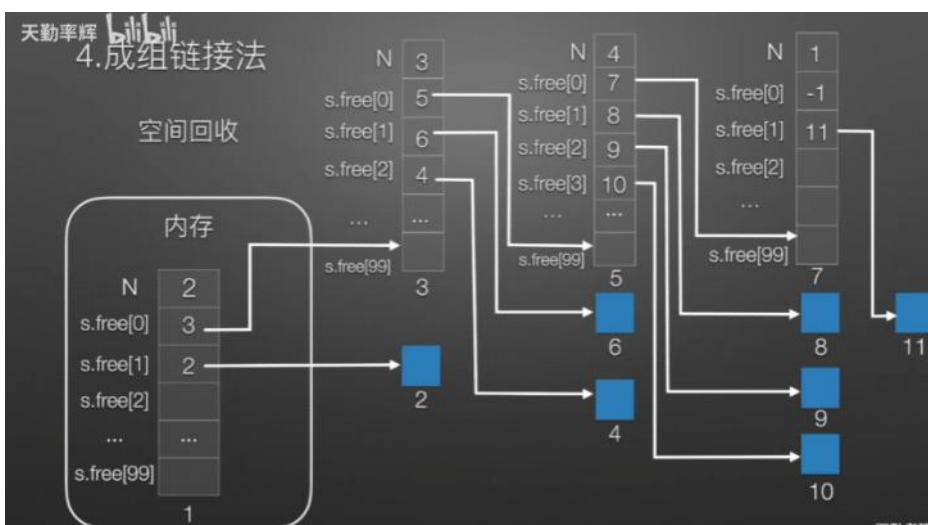
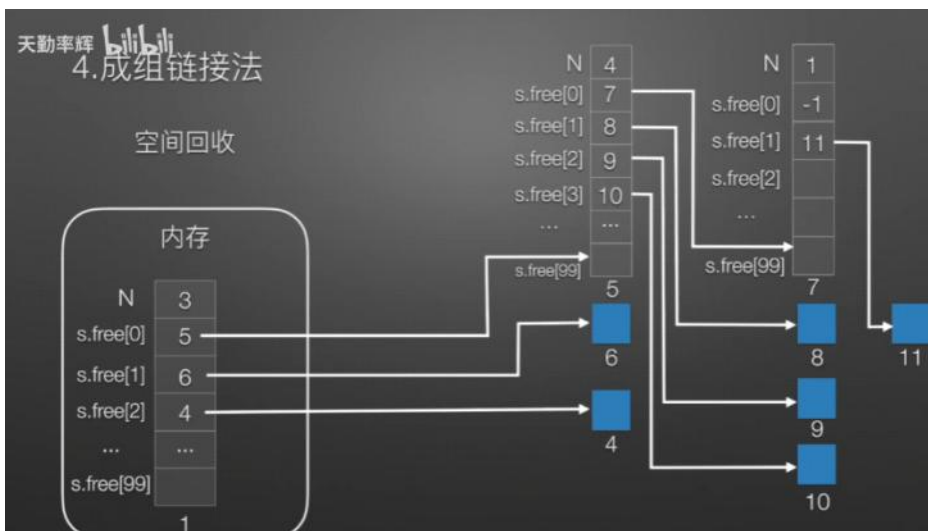
第一个区就是内存中的超级块，完善下信息就有了下图，每个盘快项如果指向了一个盘快区，那么就有一个指向区的盘快表记录了下一个盘快区的所有盘快信息。



如上图如果要分配空闲盘快，那么从超级块指向区从后往前开始，先是4、3分配出去，当超级快的长度为1的时候，就会被后面指向的盘快覆盖掉，上图就是2成为了超级块。如果继续分配也是这么个方法。



当回收盘快的时候，来一个空闲盘快就压入超级块，当超级块满了后独立出去更新新的超级块，原来超级块成为了第一个被指向的盘区，然后就继续压入超级块。



六、文件操作

1、创建文件

调用Create(需要磁盘空间大小、文件存放路径、文件名)这个系统调用即可。操作系统看到了该系统调用，就做了两件事情：在磁盘中找到空闲空间、创建该文件对应的FCB。



2、删除文件

调用Delete(文件存放路径、文件名)这个系统调用。操作系统看到了后先根据文件名找到目录项FCB，然后根据FCB记录的相关信息来回收文件占用的磁盘块，最后从目录表中删除目录项FCB。



可以“删除文件”（点了“删除”之后，图形化交互进程通过操作系统提供的“删除文件”功能，即 **delete** 系统调用，将文件数据从外存中删除）

3、打开文件

调用 **open(文件路径、文件名、读or写操作)** 系统调用。操作系统看到后会 **根据文件路径找到对应的目录项**，然后 **检查是否是指定的权限**，为了提升文件的访问速度，就 **将目录项复制到内存中的打开文件表中**，返回打开文件表的编号，之后用户使用编号来指明要操作的文件。

用户进程A的“打开文件表”

编号	文件名	...
1	...	...
2	test.txt	...
3		

复制

文件名	...	外存中的位置
test.txt	...	...
...	...	...

之后用户进程A再操作文件就不需要每次都重新查目录了，这样可以加快文件的访问速度

内存中的打开文件表

内存

Demo 目录

外存

注意打开文件表有2个，一个是**系统的打开文件表**记录了打开的所有文件，**打开计数器**表示此时多少进程打开此文件，打开计数器不为0的时候系统提示不能删除；一个是**用户进程的打开文件表**记录了当前进程打开了什么文件，读写指针记录了文件读写操作到的位置。系统打开文件表包含了用户的打开文件表。

编号	文件名	读写指针	访问权限	系统表索引号
1	...	...	...	...
2	test.txt	...	只读	k
3				

用户进程A的“打开文件表”

编号	文件名	...	外存地址	打开计数器
1	...	...	...	...
...	...	...	...	...
k	test.txt	...	...	2
...	...	...	...	...

系统的打开文件表（整个系统只有一张）

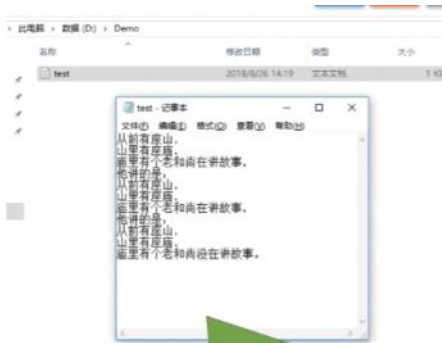
编号	文件名	读写指针	访问权限	系统表索引号
1	test.txt	...	读和写	k

用户进程B的“打开文件表”

4、关闭文件

调用 **Close** 系统调用可以关闭文件，操作系统看到后先**把用户进程的打开文件表的表项删除**，然后**回收内存资源**，打开计数器 $count-1$ ，当 $count=0$ 的时候则删除系统打开文件表的表项。

5、读文件和写文件



可以“读文件”，将文件数据读入内存，才能让CPU处理（双击后，“记事本”应用程序通过操作系统提供的“读文件”功能，即 **read 系统调用**，将文件数据从外存读入内存，并显示在屏幕上）

编号	文件名	读写指针	访问权限	...
1	test.txt	...	读和写	...

“记事本”进程的“打开文件表”

进程使用 **read** 系统调用完成读操作。需要指明是哪个文件（在支持“打开文件”操作的系统中，只需要提供文件在打开文件表中的索引号即可），还需要指明要读入多少数据（如：读入 **1KB**）、指明读入的数据要放在内存中的什么位置。操作系统在处理 **read** 系统调用时，会从读指针指向的外存中，将用户指定大小的数据读入用户指定的内存区域中。



可以“写文件”，将更改过的文件数据写回外存（我们在“记事本”应用程序中编辑文件内容，点击“保存”后，“记事本”应用程序通过操作系统提供的“写文件”功能，即 **write 系统调用**，将文件数据从内存写回外存）

编号	文件名	读写指针	访问权限	...
1	test.txt	...	读和写	...

“记事本”进程的“打开文件表”

进程使用 **write** 系统调用完成写操作，需要指明是哪个文件（在支持“打开文件”操作的系统中，只需要提供文件在打开文件表中的索引号即可），还需要指明要写出多少数据（如：写出 **1KB**）、写回外存的数据放在内存中的什么位置。操作系统在处理 **write** 系统调用时，会从用户指定的内存区域中，将指定大小的数据写回写指针指向的外存。

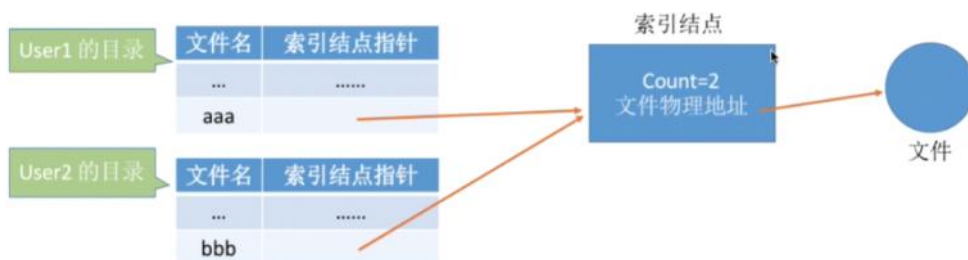
写文件write步骤：按照文件描述符找到文件的目录项，检查访问的合法性、物理块号转为逻辑块号、开始读写IO

七、文件共享

首先需要明确共享文件指的是只有一个文件多个用户访问，一个用户修改后其他用户也能看到，这和多个用户复制同一文件有本质区别。

1、基于索引节点的共享方式（硬链接）

如下图2个用户要共享一个索引节点的文件，则双方的索引节点指针都指向一个索引节点，索引节点中的count表示当前几个用户链接了它，当一个用户要删掉这个文件的时候count-1并且把该用户目录的目录项删除，当count=0的时候才是系统真正的删除这个文件。



2、基于符号链的共享方式（软链接）

说白了就是**新建快捷方式**，这个快捷方式是个link文件记录了共享文件的存放路径，操作系统根据存放路径一层一层的查找目录最终找到目录文件。如果link指向的文件被删了那么link文件是还在的，他会说找不到对应的目录项。因为软链接需要操作系统根据文件路径一层一层的查，会有多次IO操作，因此访问速度比硬链接慢。

八、文件保护

文件保护分级从大到小分别是：系统级（登录注册，不注册不让用系统）、用户级（什么用户才有权访问该文件）、文件级（给文件加密等）

1、口令保护

给文件设置一个口令，用户访问它要**输入口令输入正确才能访问文件**，口令放在FCB中或者索引节点中。优点就是**开销小**，缺点就是口令放在系统内部，有人入侵系统就完犊子了，**不够安全**。

2、加密保护

使用某个密码对文件进行加密，访问时要提供正确的密码才能进行正确的解密。下面的异或加密就是最简单的加密方式

Eg: 一个最简单的加密算法——异或加密
假设用于加密/解密的“密码”为“01001”

文件的原始数据: 0 0 1 0 1 0 1 1 0 0 0 1 1 1 0 1 0 0 0 1 ...

加密密码: 0 1 0 0 1

加密结果: 0 1 1 0 0

加密结果: 0 1 1 0 0 0 0 1 0 1 0 0 1 1 1 1 1 0 0 0 ...

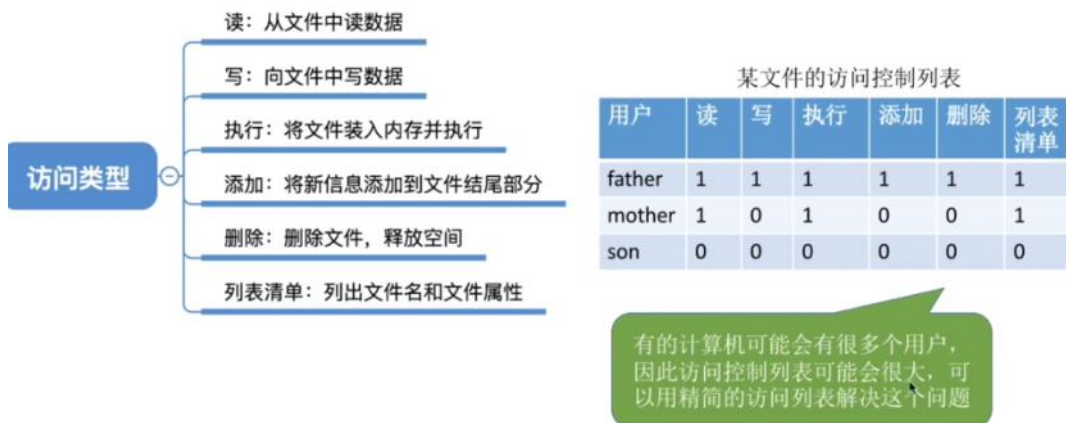
解密密码: 0 1 0 0 1

解密结果: 0 0 1 0 1

优点是保密性强, 缺点是加密解密需要时间。

3、访问控制

在FCB中加一个访问控制表, 限制不同用户可以进行的操作。



如果用户很多, 可以根据不同身份来进行访问限制, 这样用户就以组为单位, 提高效率了。

	完全控制	执行	修改	读取	写入
系统管理员	1	1	1	1	1
文件主	0	1	1	1	1
文件主的伙伴	0	1	0	1	0
其他用户	0	0	0	0	0

精简的访问控制列表

九、文件系统布局

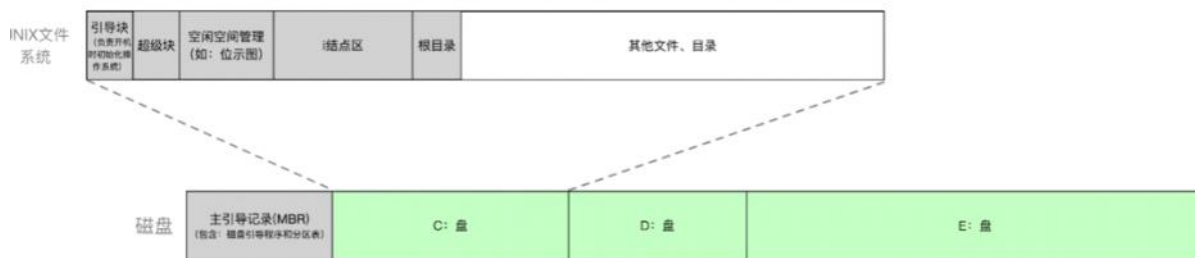
1、物理格式化 (低级格式化)



物理格式化, 即低级格式化——划分扇区, 检测坏扇区, 并用备用扇区替换坏扇区

注意替换坏扇区不是真给他俩换了, 而是操作系统访问到坏扇区的时候会跳到对应的备用扇区。

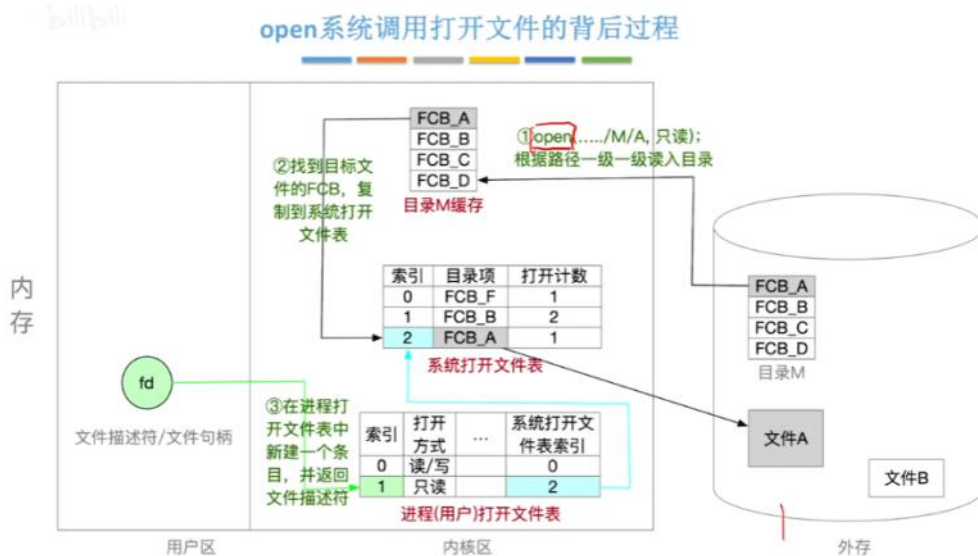
2、逻辑格式化 (高级格式化)



逻辑格式化后，磁盘分区（分卷 Volume），完成各分区的文件系统初始化
 注：逻辑格式化后，灰色部分就有实际数据了，白色部分还没有数据

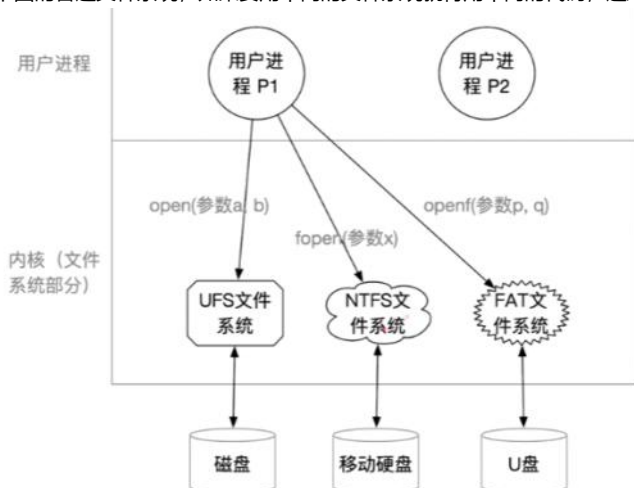
其中**引导块**负责初始化；**超级块**用来分配空闲磁盘块；**位示图**拿来快速判断哪个磁盘块空闲；**节点区**就是一个超大的数组，数组放的都是一个一个的索引节点，这些索引节点连续存放大小相同因此可以随机访问；任何文件都是从**根目录**出发的。

3、文件系统在内存中的结构

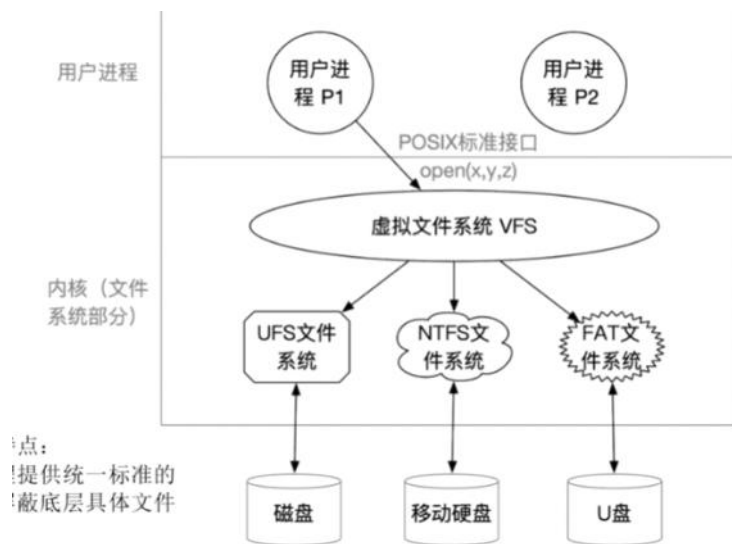


十、虚拟文件系统

如下图的普通文件系统，如果要用不同的文件系统就得用不同的代码，这对程序员来说很麻烦，为了让操作更简单，就向上封装一层虚拟文件系统VFS。



虚拟文件系统就是向上层用户提供统一标准的系统调用接口来屏蔽掉底层具体文件系统的差异。



IO设备管理

2025年6月9日星期一 上午9:44

IO设备管理自问自答

IO设备是啥？可以怎么分类？

IO设备由哪两部分组成的？IO控制器在哪个部分？

IO控制器有哪些功能？IO设备由什么部分组成？

IO的控制方式有哪些，分析他们的读写过程、CPU干预、传输单位、数据流向和优缺点。

IO软件的层次结构有哪些？每层的功能是什么？物理设备名和逻辑设备名的转换是通过什么来实现的？

IO应用程序的接口有哪些？阻塞IO和非阻塞IO区别是什么？设备驱动的程序接口是谁来决定的？

IO核心子系统包括哪些？SPOOLing技术是什么？实现步骤是什么？SPOOLing系统的组成是什么？

设备分配算法有哪几种？从不同角度考虑。

设备分配管理中的数据结构有哪些，每个模块的功能是什么？设备分配的步骤又是什么？

缓冲区的作用是什么？有哪几种缓冲区？顺便评估一下他们的处理一个数据块的平均耗时。

磁盘由什么组成？物理地址三元组是什么？磁盘可以怎么分类？

磁盘的读写操作需要什么时间？磁盘调度算法有哪些？

减少磁盘延迟时间的方法有哪些？

磁盘的初始化有哪些？引导块是什么？坏块可以怎么管理？

一、IO设备的概念与分类

1、概念

所谓IO就是输入输出，IO设备是将数据输入到计算机，或者可以接收计算机输出数据的外部设备，属于计算机中的硬件。例如鼠标键盘就是最经典的**输入型设备**，显示器这些是**输出型设备**，U盘这些是**输入输出型设备**。

2、IO设备的分类

IO设备**按照使用特性分类**，分为人机交互类外部设备、存储设备和网络通信设备。**人机交互类设备**的速度传输慢，因为需要等人操作完，传输的单位大小一般挺大的，键盘鼠标这种就是人机交互设备。**存储设备**的速度传输快，例如光盘移动硬盘这些。而**网络通信设备**的传输速度介于两者之间，用于网络通信，例如调制解调器这些。

按照传输速率分类分为低速设备、中速设备、高速设备，高中低速没有明确的界限，**低速设备**就像鼠标键盘这些，传输速率是每秒几个到几百个字节，**中速设备**例如激光打印机这些传输速率每秒数千数万个字节；**高速设备**例如磁盘等每秒数千字节到千兆字节的设备例如磁盘这些。

按照信息交换的单位分类可以分成块设备和字符设备和网络设备。**块设备**的传输单位是“块”，传输的速率高**可以寻址**（可以随机读写任意一块）；**字符设备**例如键盘这些的传输单位是字符，传输速率较慢**不可寻址**，输入输出的时候经常**采用中断驱动**的方式。

二、IO控制器

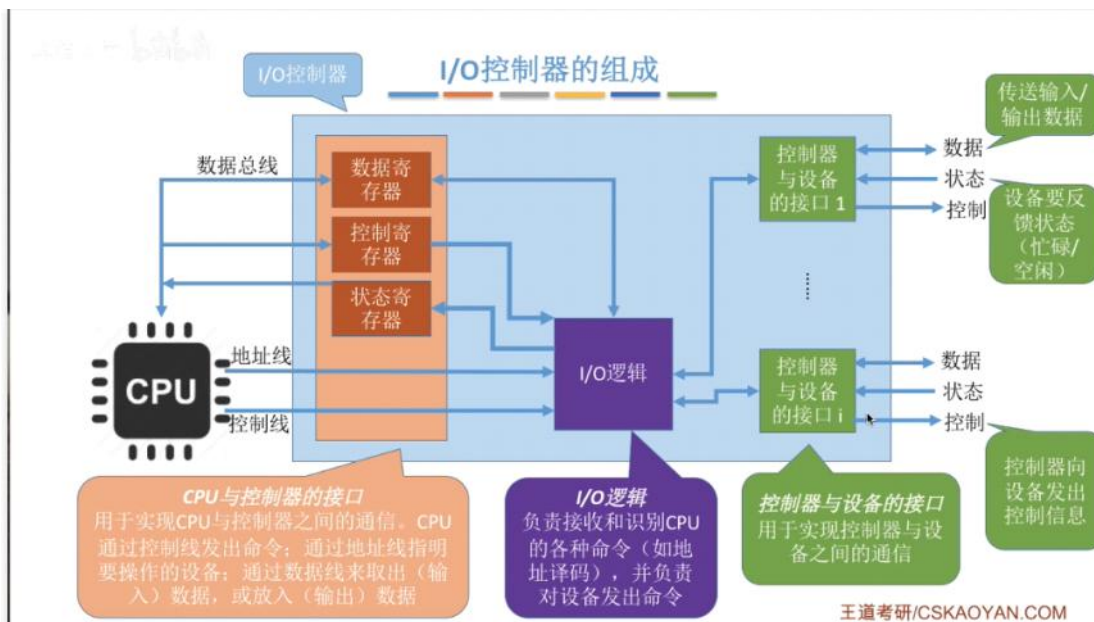
IO设备由机械部件和电子部件组成，**机械设备**用来执行具体的IO操作，例如键盘的键帽，显示器的LED屏幕这些；而**电子设备**是插入主板扩充槽的一个电路板，例如鼠标线的插入口拆开是一个电路板。CPU是无法直接控制机械部件的，此时需要电子部件作为一个中介来实现CPU对设备的控制，**这个中介就是IO控制器**，是嵌在电路板中的一个硬件。

1、IO控制器的功能

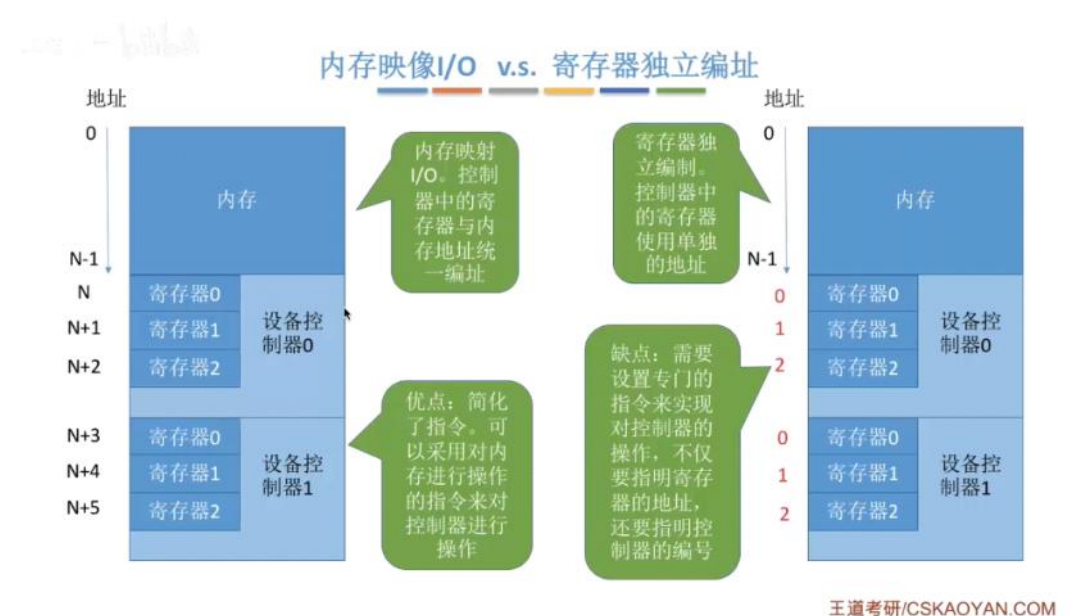
IO控制器可以**接收和识别CPU发出的命令**，例如CPU发来的read/write指令，IO设备会有一个相应的**控制寄存器**来存放命令和参数；IO控制器可以**向CPU报告设备的状态**，在控制器中会有**状态寄存器**记录状态，1是空闲0是忙碌；IO设备**可以实现数据交换**，设置了一个**数据寄存器**用于暂存数据，输入时寄存器用于暂存设备发来的数据，之后CPU从寄存器中取走数据，输出时寄存器暂存CPU发来的数据，之后控制器寄存器来取走数据。IO设备**可以进行地址识别**，IO设备中会有多个寄存器，为了方便CPU识别具体是哪个，就**给各个寄存器设置一个特定的地址**，IO控制器通过CPU提供的地址来判断CPU读写的是哪个寄存器。

2、IO控制器的组成

IO控制器简单来说就是由CPU与控制器、控制器与设备的接口以及IO逻辑组成。



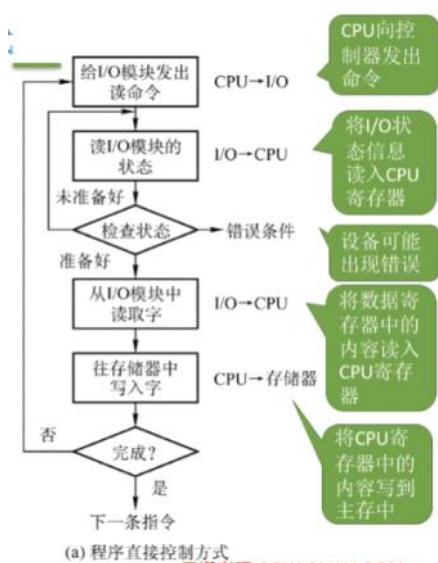
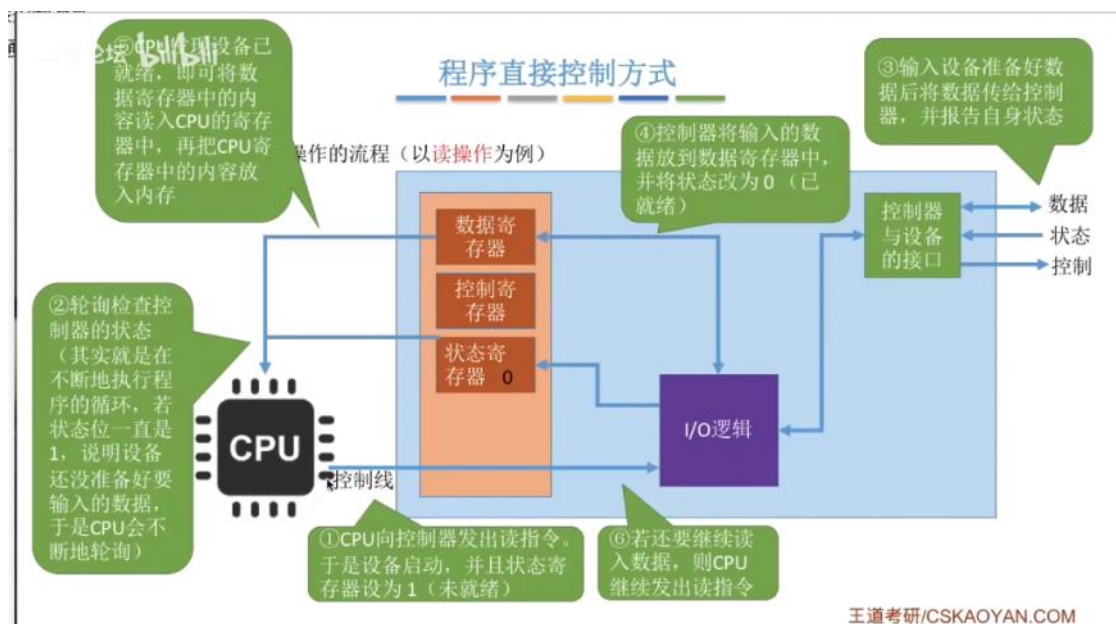
注意一个IO控制器可能会对应多个设备，因此数据寄存器、控制寄存器、状态寄存器这些一个肯定不够用，可能会有多个才能方便CPU操作。多个寄存器需要编址才能方便计算机识别，有些计算机让寄存器占用地址的一部分称为**内存映像IO**；有些计算机则采用IO专用地址即**寄存器独立编址**。他俩的区别就是各设备控制器的地址是否相互独立。



三、IO控制方式

1. 程序直接控制方式

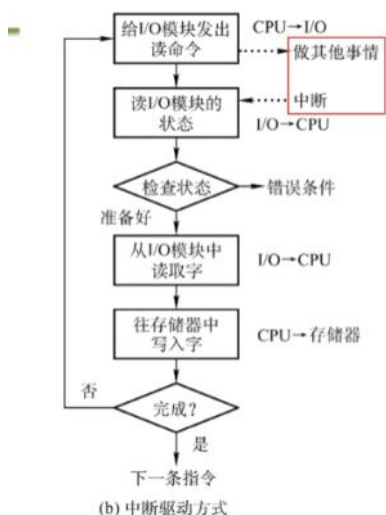
这种方式是通过CPU的轮询来实现的，就像老师教学生那样，老师会一直问一直问“学会了吗”，直到学生回答说“学会了”才停止。



以上就是完成一次读写操作的流程，可以看出CPU的干预是很频繁的，因为CPU等待IO需要不断的轮询，并且数据传送的单位是字，即每次读写都是一个字，数据的流向在数据输入上是IO设备→CPU→内存，在数据输出上是内存→CPU→IO设备，每个字的读写都需要CPU的帮助。优点就是简单，缺点是只能串行工作，CPU轮询的时候会忙等，利用率低下。

2. 中断驱动方式

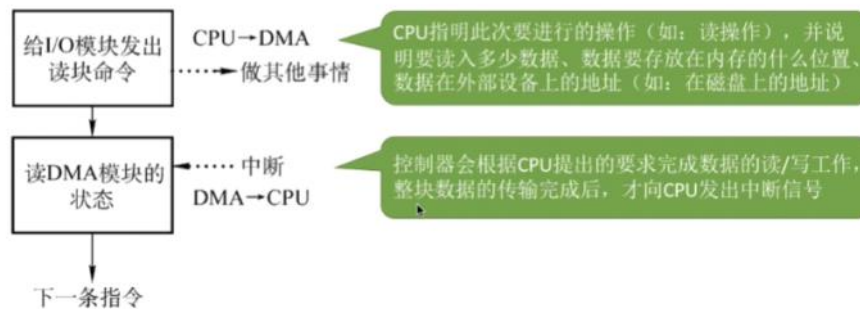
由于IO设备速度慢，可以先让IO进程阻塞掉，让CPU切换到其他的进程执行，当IO设备准备好后会向CPU发送一个中断信号，CPU检测到后就处理中断，处理中断的过程中CPU从IO控制器读一个字的数据传送到CPU寄存器再写入主存，中断处理完后恢复IO进程或者其他进程的运行环境继续执行。就像老师不会再向学生一直问“学会了没”，而是先干其他事情，当学生学会了就中断老师当前的事情并告诉老师学会了。



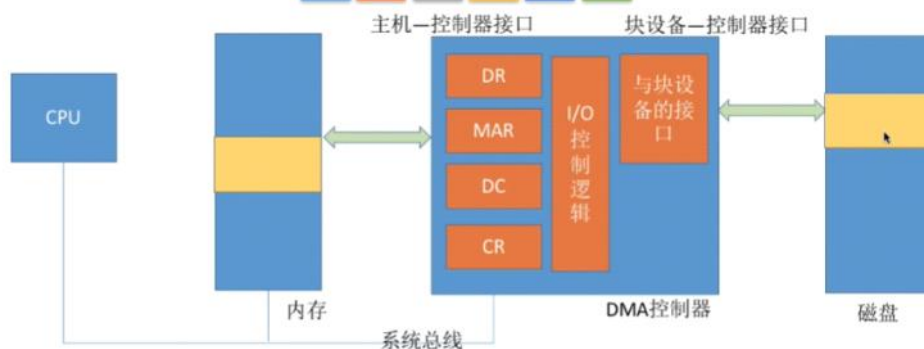
注意CPU在每个指令周期的末尾都会例行检查中断，中断处理过程中需要保存恢复进程的运行环境因此会有系统开销，如果中断过于频繁就会降低系统性能。采用中断驱动方式可以降低CPU干预频率，传送单位是读写一个字，数据流向在读操作时是IO->CPU->内存，写操作是内存->CPU->IO设备，还是需要CPU的介入，优点就是可并行工作提升CPU利用率，但是频繁中断就出问题了。

3. DMA方式（直接存储器存取）

DMA主要用于块设备的IO控制，数据的传输单位是“块”而不是字了，传送速率更高，并且数据流向是直接设备放入内存或者内存直接到设备，不需要CPU的介入了，仅仅在传送一个数据块的开始和结束的时候才要CPU干预。



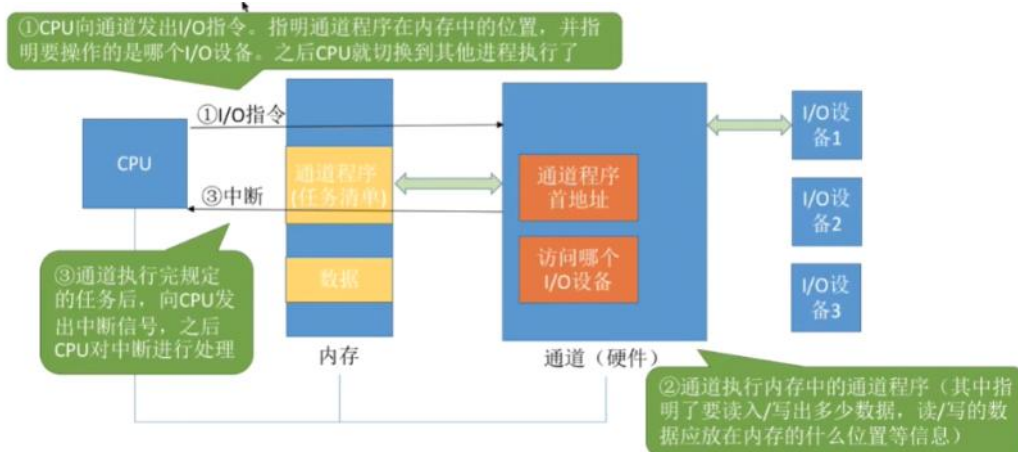
DMA控制器的组成与IO控制器相似，由主机-控制器接口、块设备-控制器接口、IO控制逻辑组成，其中DR是数据寄存器暂存设备到内存或者内存到设备的数据、MAR是内存地址寄存器表示数据应该放到内存中的什么位置、DC是数据计数器表示剩余要读写的字节数、CR是状态寄存器存放CPU发来的IO命令或者设备的状态信息。



这种方式CPU的干预频率进一步降低，并行性进一步提升，数据传送的单位变成了“块”（但是每次读写的只能是连续的多个块，离散块的话需要多次读写）传输效率提升，数据流向在读操作下是IO设备->内存，写操作下是内存->IO设备。

4. 通道控制和方式

通道是一种硬件，可以理解为一个CPU，可以识别执行通道指令。CPU只需要告诉通道程序的位置，操作哪个IO设备就行了，通道自己识别任务清单并执行，执行完毕给CPU发送中断信号。

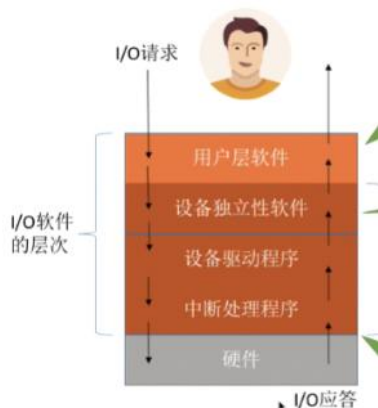


这种方式下CPU干预是最低的，每次传输的还是块单位，数据流向也是内存和IO设备之间流向不带CPU，资源利用率很高，唯一缺点就是实现复杂并且需要专门的硬件支持。

	完成一次读/写的过程	CPU干预频率	每次I/O的数据传输单位	数据流向	优缺点
程序直接控制方式	CPU发出I/O命令后需要不断轮询	极高	字	设备→CPU→内存 内存→CPU→设备	每一个阶段的优点都是解决了上一阶段的缺点。总体来说，整个发展过程就是要尽量减少CPU对I/O过程的干预，把CPU从繁杂的I/O控制事务中解脱出来，以便更多地去完成数据处理任务。
中断驱动方式	CPU发出I/O命令后可以去做其他事，本次I/O完成后设备控制器发出中断信号	高	字	设备→CPU→内存 内存→CPU→设备	
DMA方式	CPU发出I/O命令后可以去做其他事，本次I/O完成后DMA控制器发出中断信号	中	块	设备→内存 内存→设备	
通道控制方式	CPU发出I/O命令后可以去做其他事。通道会执行通道程序以完成I/O，完成后通道向CPU发出中断信号	低	一组块	设备→内存 内存→设备	

四、IO软件层次结构

IO由下到上的层次分别是硬件、中断处理程序、设备驱动程序、设备独立性软件、用户层软件这5层结构，类似于计网的OSI分层结构，越下层越接近硬件，每一层都是根据下层的服进行封装并向高层提供服务。其中io软件是上面4层，除去硬件的部分。



1. 用户层软件

用户层软件主要实现了与用户交互的接口，用户可以直接调用该层提供的库函数来操作设备，例如printf函数这些。用户调用库函数后用户层软件将其翻译成IO请求并且通过系统调用来请求系统内核的服务，例如printf会翻译成write系统调用。



2. 设备独立性软件 (设备无关性软件)

该层次主要向上层提供统一的系统调用接口例如read调用write调用这些；实现设备的保护，这和文件保护是一样的，因为操作系统将设备看成一种特殊的文件；差错处理就是对设备的一些错误进行处理；设备的分配与回收；数据缓冲区管理就是通过缓冲技术来屏蔽设备之间数据传输速度大小的差异；建立逻辑设备名到物理设备名的映射关系，根据设备类型来选择相应的驱动程序，其中逻辑设备名是用户看到的设备名（打印机1打印机2），物理设备名是操作系统看到的设备名，映射关系是通过逻辑设备表LUT确定的，确定好物理设备后就使用对应的设备驱动程序。



可以采用2种方式来管理逻辑设备表LUT，第一种就是操作系统只设置一张LUT，但是这样的话设备逻辑名是不能一样的，只适用于单用户操作系统；第二种是给每一个用户设置一张LUT并存放在用户管理进程的PCB中。

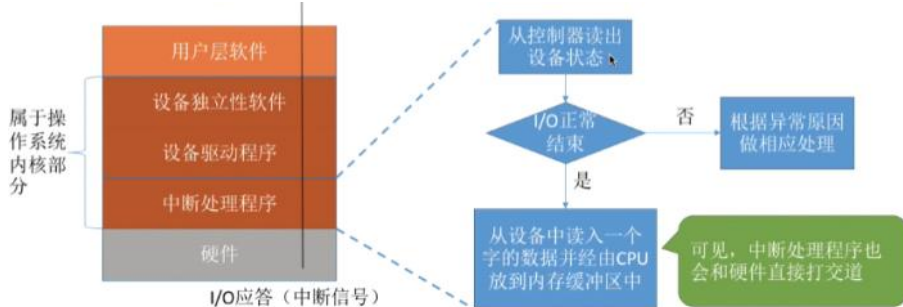
3. 设备驱动程序 (主要面向设备)

该层次主要负责将上层的一系列命令转换成IO硬件能听懂的一系列操作，包括设置设备寄存器，检查设备等，例如将read转换成特定设备能看懂的语句。注意设备驱

动程序是由厂家提供，用户电脑插上设备后会设置一个驱动程序入口地址，然后自动下载相应的驱动程序。注意驱动程序会以一个独立进程的方式存在。设备驱动程序还需要接收进程发来的IO命令参数并检验合理性、查询IO设备状态、发出IO命令和启动IO设备（面向IO设备）。

4、中断处理程序

io任务完成的时候io控制器发送一个中断信号，系统根据信号类型找到相应的中断处理程序并执行，处理流程如下：



注：中断处理程序和设备驱动程序在实际运行中是相互配合的，没有明确的界限。

5、硬件

执行io操作的，有机械部件和电子部件。

五、IO应用程序接口和设备驱动程序接口

前面学过IO设备有字符设备、块设备和网络设备，用户层io软件通过系统调用来请求服务，设备不同系统调用也无法统一，因此需要不同的IO应用程序接口来实现。



1、字符设备接口

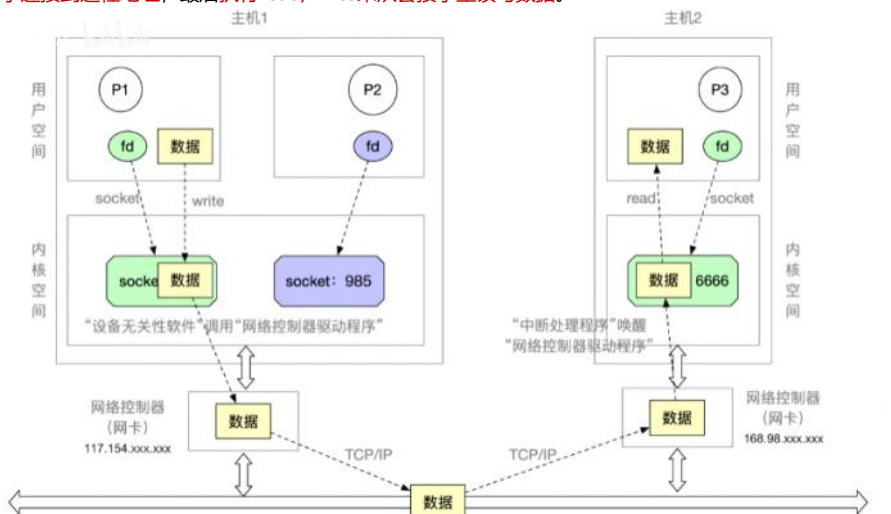
因为传输的单位是字符，所以直接用get/put系统调用一个字符一个字符的传输就行了。

2、块设备接口

传输单位是块，因此调用read/write这些系统调用，向块设备读写指针的位置一次性输入或输出多个字符，通过seek调用来修改指针的位置。

3、网络设备接口（网络套接字socket接口）

传输单位是网络套接字，首先需要创建一个socket套接字，指明好相应的网络协议(TCP, UDP)，然后通过bind将套接字绑定到某个本地的端口，然后通过connect将套接字连接到远程地址，最后执行read/write来从套接字里读写数据。



4、阻塞io和非阻塞io

阻塞io在发出io系统调用时进程需要转为阻塞态等待，例如scanf需要用户输入完才会解除阻塞继续运行；非阻塞io在发出系统调用时无需阻塞等待，例如块设备接口往磁盘写数据的时候用的write调用。

5、设备驱动程序接口

如果各个公司生产的设备驱动程序接口不统一，操作系统就很难调用设备的驱动程序。



因此操作系统会统一标准让各个设备公司遵守标准，不然不支持该软件。各个公司就会专门为操作系统开发专门的程序接口，例如windows接口和MAC接口是不一样的。



六、IO核心子系统

IO层次由下到上分别是硬件、中断处理程序、设备驱动程序、设备独立性软件、用户层软件这5层，其中IO核心子系统就是中间那3层，简称IO系统。



1、IO调度

就是用算法确定一个好的顺序来处理各个IO的请求，例如磁盘调度算法，当多个磁盘IO请求到来时使用先来先服务还是SCAN等算法；除了磁盘还有打印机等设备也可以用这些算法来确定IO调度顺序。

2、设备保护

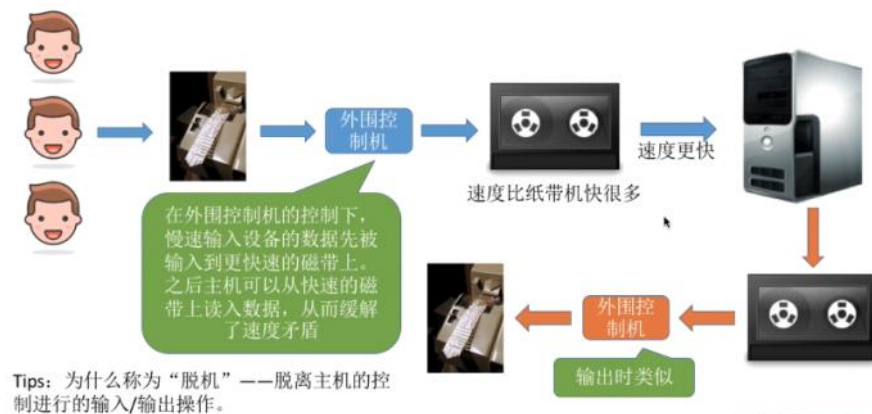
把设备看成一种特殊的文件，用文件保护的方式来实现设备保护。

3、假脱机SPOOLing技术

虽然在用户层软件，但是408也将其归类为IO核心子系统需要实现的功能。

七、假脱机SPOOLing技术

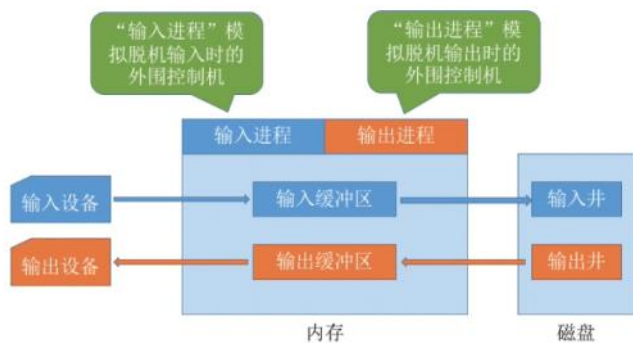
在前面批处理中讲过批处理技术（脱机技术），用于缓和人机速度矛盾，磁带作为缓冲的桥梁用于缓和速度矛盾，外围控制机用于控制这些设备。



Tips: 为什么称为“脱机”——脱离主机的控制进行的输入/输出操作。

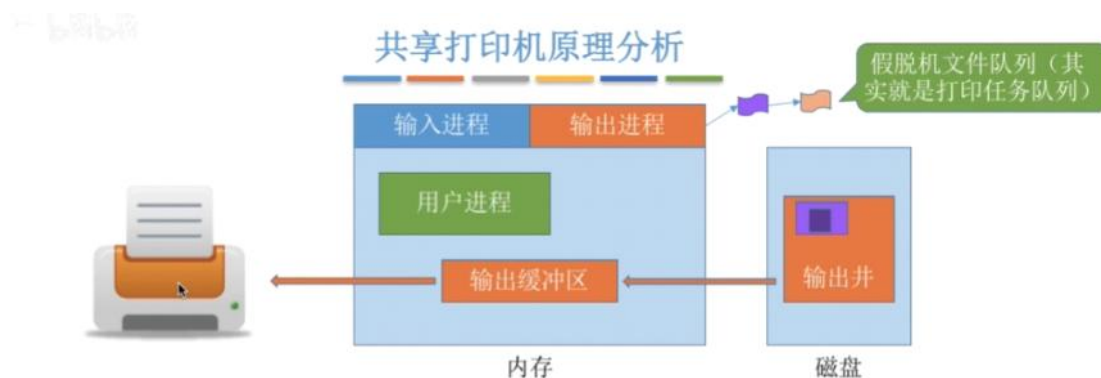
如上图的脱机技术，因为IO过程中不需要主机和CPU的干预，全靠外围控制机来实现，因此叫做脱机技术（脱离主机的控制进行IO操作）。

基于脱机技术，发明了假脱机技术，为啥不是真脱机呢，因为它用软件的方式模拟出来的脱机技术，不用任何外围计算机的支持，SPOOLing系统的组成如下：



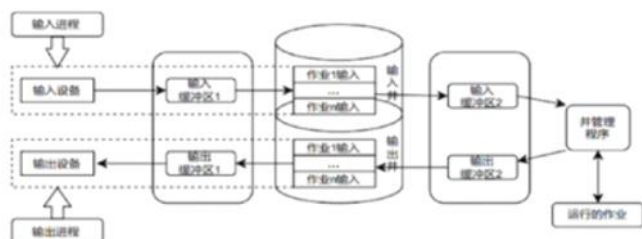
SPOOLing就是用软件的方式模拟出外围控制机，输入输出井用于模拟磁带来缓冲，内存中的输入输出缓冲区输入井暂存和输出井转存的数据。

以共享打印机为例，它属于**独占式设备**，只允许各个进程串行使用，一段时间内只能满足一个进程的请求。但是通过SPOOLing技术可以将其改造成共享设备，一个物理打印机在逻辑上分成多个打印机。



当多个用户提出打印请求的时候，系统会答应请求但是不是真的把打印机分给他，自欺欺人一下先把用户唬住。此时SPOOLing进程为每个进程做2件事，第一是在磁盘上申请一个空闲缓冲区并将数据送进去，然后为用户进程申请一张空白的**打印请求表**，用户的打印请求填进去，把该表挂到假脱机队列上。当打印机空闲下来的时候从队头取下一张打印请求表来进行打印。**打印完并不会直接输出到显示设备而是放在输出井（外存）中等待输出设备，当输出设备空闲了就输出。**

这样虽然只有一台打印机，但是通过在输出井设立存储区，在**逻辑上分成了多个打印机**，使得每个用户都感觉打印机都在为自己服务，从而实现打印机的“共享”。



如上图，SPOOLing系统由**预输入进程（输入进程）**和**缓输出进程（输出进程）**和**井管理程序**组成。

八、设备的分配与回收

1、设备的种类

按照固有属性可以分成独占设备、共享设备和虚拟设备。

独占设备就是一个时间段内只能分配给一个进程。

共享设备就是一个时间段内可以给多个进程使用（有些是微观上交替分配的）

虚拟设备就是用虚拟技术将独占设备改造成共享设备，例如SPOOLing技术

2、设备分配算法

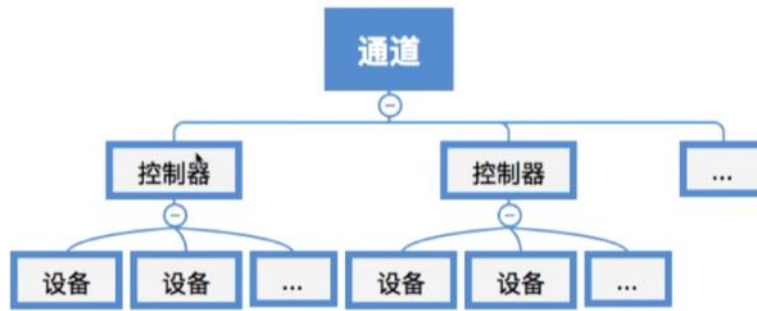
和作业调度一样，有先来先服务，短作业优先这些。

从安全性上考虑，有**安全分配方式**（给进程分配一个设备后进程阻塞，IO完成后才唤醒）和**不安全分配方式**（不阻塞，进程可以继续执行或者发出新的IO请求，只有没资源的时候才阻塞）。这两个中安全分配算法破坏了“请求保持”条件，不会死锁缺点就是效率低，只能串行工作；不安全分配算法可以并行工作效率高但是可能死锁，需要通过死锁的避免、检测、解除来处理。

从运行上考虑有**静态分配**（运行前分配好所有资源，运行结束释放资源）和**动态分配**（运行过程中请求资源）

3、设备分配管理中的数据结构

“设备、控制器、通道”之间的关系：



一个通道可控制多个设备控制器，每个设备控制器可控制多个设备。

为了实现设备操作控制，系统为每一个设备设立一个**设备控制表DCT**（Device Control Table）用于记录设备情况，DCT的组成如下：

设备控制表（DCT）	
设备类型	如：打印机/扫描仪/键盘
设备标识符	即物理设备名，系统中的每个设备的物理设备名唯一
设备状态	忙碌/空闲/故障...
指向控制器表的指针	每个设备由一个控制器控制，该指针可找到相应控制器的信息
重复执行次数或时间	当重复执行多次I/O操作后仍不成功，才认为此次I/O失败
设备队列的队首指针	指向正在等待该设备的进程队列（由进程PCB组成队列）

为了实现控制器的操作控制，系统为每一个设备控制器设置一张**控制器控制表COCT**（Controller Control Table）用于管理设备控制器，COCT组成如下：

控制器控制表（COCT）	
控制器标识符	各个控制器的唯一ID
控制器状态	忙碌/空闲/故障...
指向通道表的指针	每个控制器由一个通道控制，该指针可找到相应通道的信息
控制器队列的队首指针	指向正在等待该控制器的进程队列（由进程PCB组成队列）
控制器队列的队尾指针	

为了实现通道的操作控制，系统为每一个通道设立**通道控制表CHCT**（Channel Control Table），CHCT的组成如下：

通道控制表（CHCT）	
通道标识符	各个通道的唯一ID
通道状态	忙碌/空闲/故障...
与通道连接的控制器表首址	可通过该指针找到该通道管理的所有控制器相关信息（COCT）
通道队列的队首指针	指向正在等待该通道的进程队列（由进程PCB组成队列）
通道队列的队尾指针	

为了记录系统全部设备的情况，系统设置**系统设备表SDT**（System Device Table），每个设备对应一张表目。

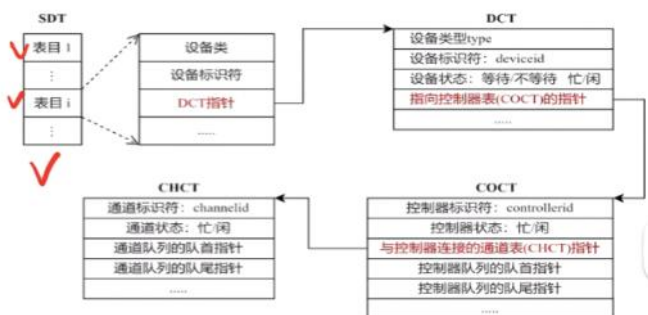


4. 设备分配的步骤

首先根据**物理设备名**在系统控制表SDT中查找设备控制表DCT，**设备**忙碌就将PCB挂到设备等待队列中，不忙碌就分配设备给进程。然后根据DCT找到COCT，**控制器**忙碌就挂到等待队列，空闲就分配给进程。然后根据COCT找到CHCT，若**通道**忙碌就挂到等待对垒，空闲就分配通道。

只有设备、控制器、通道三者都分配成功时，设备分配才算成功，之后启动IO设备进行数据输送。

这种分配步骤缺点很明显，第一是用户编程时必须使用“物理设备名”，这个设备名是底层的东西对用户不透明，不方便编程；第二是换了一个物理设备就无法运行；第三是请求的物理设备正在忙碌，即使系统中还有同类型的设备，进程也必须阻塞等待。



改进方法就是建立逻辑设备名与物理设备名的映射关系，用户提供逻辑名，操作系统负责转换。

设备分配步骤的改进

- ①根据进程请求的逻辑设备名查找SDT（注：用户编程时提供的逻辑设备名其实就是“设备类型”）
- ②查找SDT，找到用户进程指定类型的、并且空闲的设备，将其分配给该进程。操作系统在逻辑设备表（LUT）中新增一个表项。
- ③根据DCT找到COCT，若控制器忙碌则将进程PCB挂到控制器等待队列中，不忙碌则将控制器分配给进程。
- ④根据COCT找到CHCT，若通道忙碌则将进程PCB挂到通道等待队列中，不忙碌则将通道分配给进程。

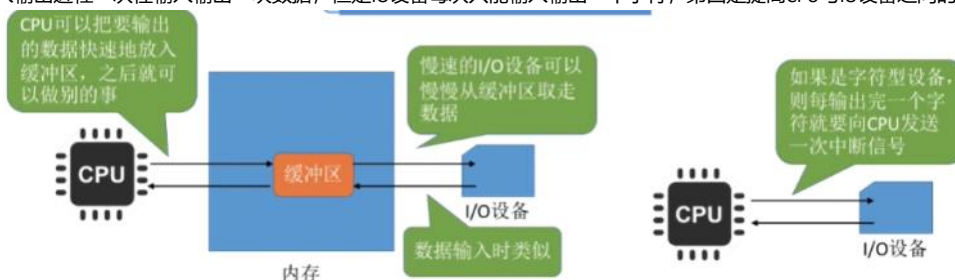


九、缓冲区管理

缓冲区就是一块存储区域用于缓冲速度矛盾，该区域可以是专门的硬件也可以是内存划分出来的一块。硬件缓冲区的成本高容量小，一般用于速度要求非常高的场合，例如快表；一般情况下更多的是利用内存缓冲区，本章就是介绍内存缓冲区。

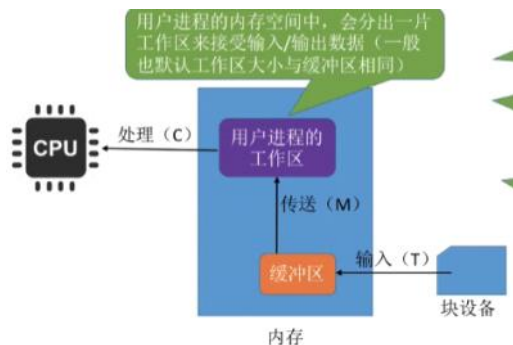
1、缓冲区作用

第一就是缓和速度不匹配的矛盾，CPU速度快，可以瞬间把缓冲区填满数据然后干其他事，之后IO设备慢慢来取走，数据流向反过来也是这样；第二就是减少CPU的终端频率，因为字符型设备每当输出一个字符就发送一个中断信号把CPU打断，采用缓冲区可以减少打断频率；第三就是解决数据粒度不匹配的问题，没有缓冲区的话输入输出进程一次性输入输出一块数据，但是IO设备每次只能输入输出一个字符；第四是提高CPU与IO设备之间的并行性。



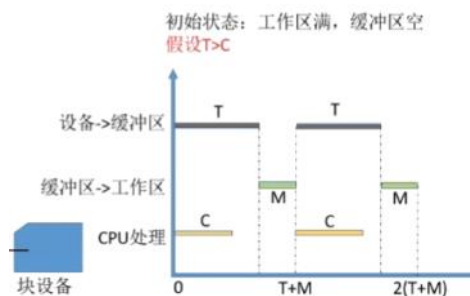
2、单缓冲

操作系统在主存中分配一个缓冲区，一般大小是一个块，当缓冲区为空的时候可以冲入数据，当缓冲区满了后就不能冲入数据了，只有缓冲区满了才能把数据传出。

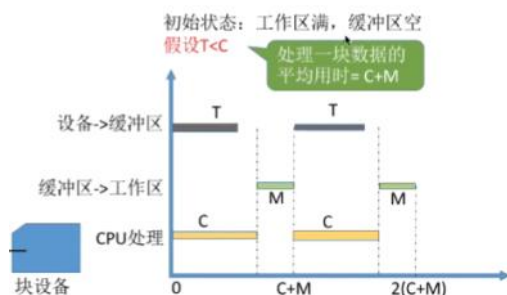


此时如果要计算出具体时间，根据输入时间 t 和处理时间 c 有几种情况。

输入时间大于处理时间的时候处理完一个数据块平均花费的时间就是输入时间+传输时间，当 $T > C$ 的时候输入完后CPU处理完了上一块数据，可以直接开始传，传完后才开始下一个数据块，此时的时间取决于输入的时间+传输的时间



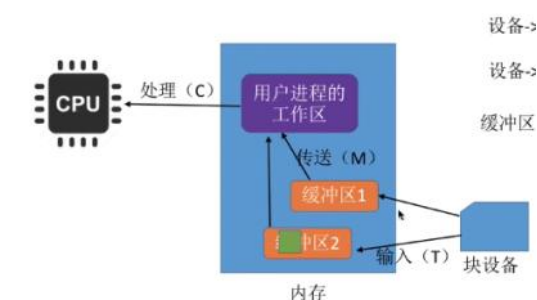
输入时间小于处理时间的时候，输入完后由于CPU还没处理完上一块数据，需要等待CPU处理完才开始传输，传完后就是下一个数据块，此时的时间取决于处理时间+传输时间



总结：采用单缓冲的话处理一块数据平均耗时是 $\text{Max}(\text{设备到缓冲区时间}, \text{CPU处理时间}) + \text{缓冲区到工作区时间}$

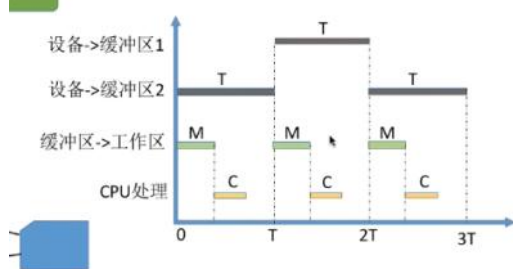
3. 双缓冲

双缓冲就是系统在主存设置2个缓冲区，一个缓冲占用一个块，一个缓冲区满了还可以向另一个缓冲区输入数据。



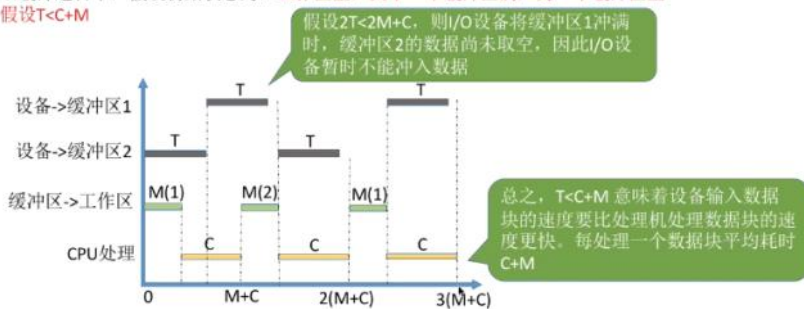
如果要计算处理一块数据的平均用时，也有几种情况：

当输入时间 $T >$ 处理时间 C +传送时间 M 时，输入完毕的时候CPU已经传输和处理好上一个数据了，就可以直接传过去，在输入下一个数据块的时候处理该数据，此时的时间取决于输入的时间。



当 $T < C+M$ 时，输入完毕后CPU还没处理好上一块数据，需要等待CPU处理完，此时花费时间取决于上一块数据的传输+处理时间

双缓冲题目中，假设初始状态为：工作区空，其中一个缓冲区满，另一个缓冲区空
假设 $T < C + M$

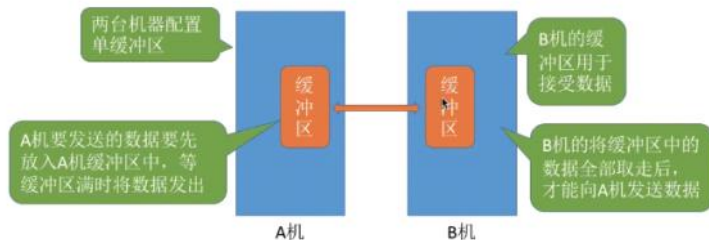


注：M(1) 表示“将缓冲区1中的数据传送到工作区”；M(2) 表示“将缓冲区2中的数据传送到工作区”

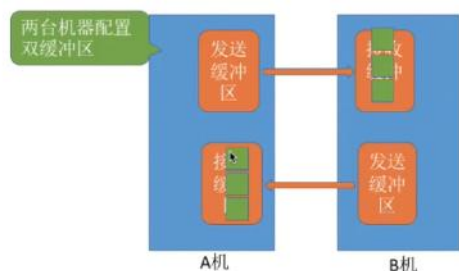
总结：双缓冲的平均耗时是 $\text{MAX}(\text{输入时间}, \text{传输} + \text{处理时间})$ ；

4、通信时的单双缓冲

由于缓冲区只有塞满的时候才能取出，如果只有一个缓冲区，就只能实现单向传输半双工通信



设置双缓冲就可以实现全双工通信了，可以同一时刻双向传输



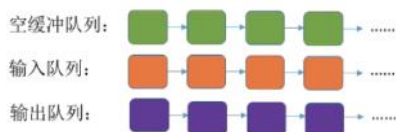
5、循环缓冲区

有时候2个缓冲区还是不够用，可以设置一个循环缓冲队列，缓冲区输出完或者满了修改指针即可。但是它只能针对一个进程设立。



6、缓冲池

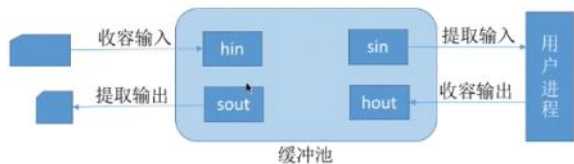
缓冲池由一系列缓冲区组成，缓冲区存放于内存中，当使用一个缓冲区或者还回一个缓冲区的时候只需要操作缓冲池就行了。按照使用的状态可以分为空闲缓冲队列、装满输入数据的缓冲队列（输入队列）、装满输出数据的缓冲队列（输出队列）。前面的缓冲技术都是一个进程专用的，而这个缓冲池可以适合多个进程并发使用。



根据缓冲区的功能可以分成收容输入缓冲区、收容输出缓冲区、提取输入缓冲区、提取输出缓冲区

收容输入就是输入数据，空缓冲队列取出一个塞进数据并放到输入队列中；收容输出就是将输出的数据，空缓冲队列取出一个塞进数据并放到输出队列中。

提取输入就是取出输入队列中的东西输出，变成空缓冲区后放到空缓冲队列；提取输出就是取出输出队列中的东西输出，变成空缓冲区后放到空缓冲队列。

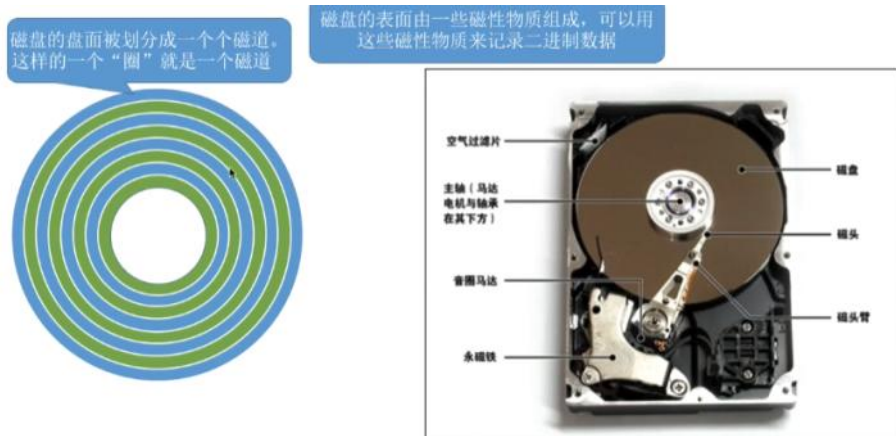


相当于从设备到进程中间隔着条河（缓冲池），缓冲池是船，收容就是上船(空变满)，提取就是运到对面了要下船(满变空)。

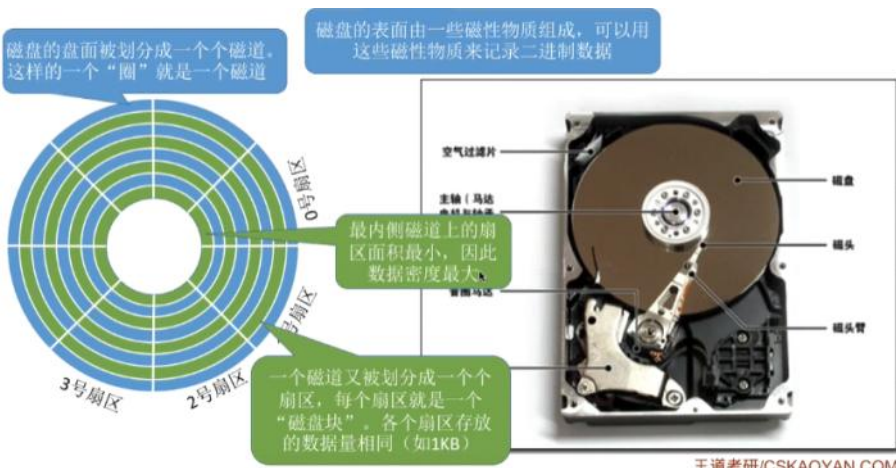
十、磁盘的结构

1、磁盘、磁道、扇区

磁盘表面由一些磁性物质组成，可以用来记录二进制数据；**磁道**就是磁盘表面上的一个个圈；

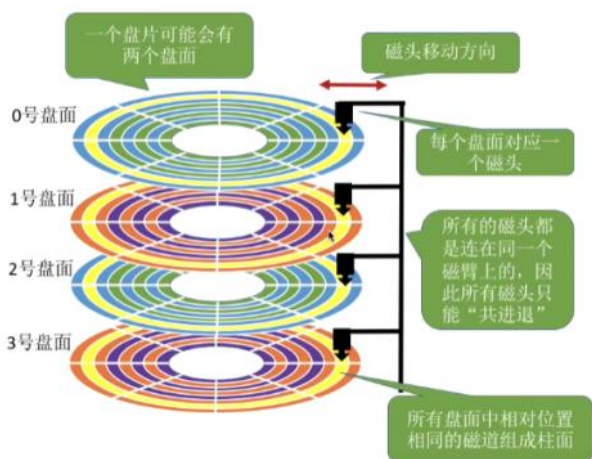


一个磁道又可以划分为多个**扇区**，每个扇区存放的数据量一样。



2、磁盘读写数据

磁盘**通过磁头来读写数据**，需要将磁头移动到读写的扇区所在的磁道，磁盘会转起来，让目标扇区从磁头下划过，才能完成对扇区的读写操作。磁头是被**磁头臂**带动的。在现实中的磁盘会有多个盘面，需要多个磁头，这些磁头统一由磁头臂来带动，**所有磁头共进退**。**柱面**是由所有盘面同一位置的磁道组成的柱面如下图黄色部分。



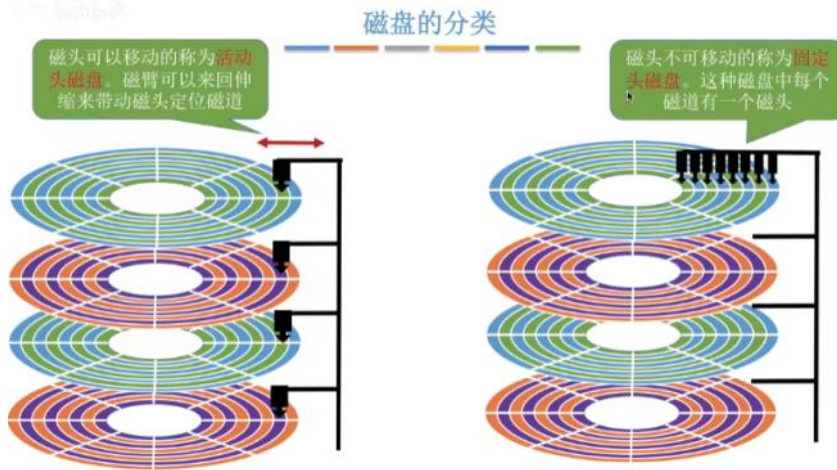
3、磁盘的物理地址

由上面可知，一个磁盘块可以通过**（柱面号、盘面号、扇区号）**来定位，文件数据存放于外存几号块，可以转换成该地址的形式。

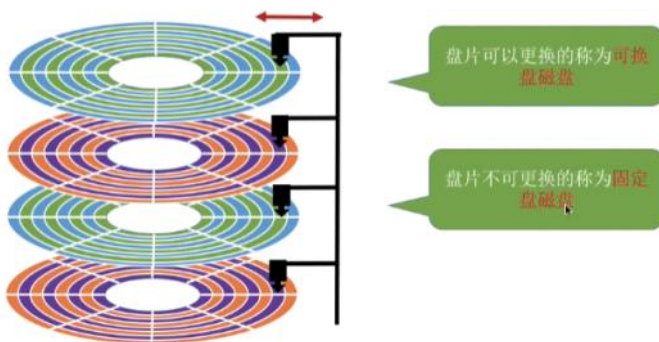
在读取文件中，**根据柱面号来移动磁臂**让磁头指向柱面，然后激活指定的**盘面对应的磁头**，然后磁盘旋转，让**指定扇区从磁头下面划过**完成读写操作。

4、磁盘的分类

根据磁头是否可以移动可以分为**活动头磁盘**和**固定头磁盘**，活动头磁盘每个盘面只有一个磁头，该磁头可以移动；固定头磁盘的每个磁道都有一个磁头，不用移动。



根据盘片是否可以更换分为**可换盘磁盘**和**固定盘磁盘**。



十一、磁盘调度算法

1、一次磁盘读写操作需要的时间

首先是**寻道时间**，寻道时间=磁头臂启动时间+移动磁头的时间，假设寻道时间 T_s ，启动磁头臂时间 s ，假设磁头匀速移动，跨越一个磁道的时间是 m ，需要跨越 n 条磁道，则**总共耗时** $T_s = s + m * n$ 。

然后是**延迟时间**，延迟时间就是转动磁盘所耗费的时间，假设转速是 r ，则 $1/r$ 就是转一圈的时间，找到目标扇区平均需要转半圈，因此再乘以 $1/2$ ，因此平均所需的延迟时间 $T_r = (1/2) * (1/r) = 1/2r$ 。磁盘转速越高延迟越小。

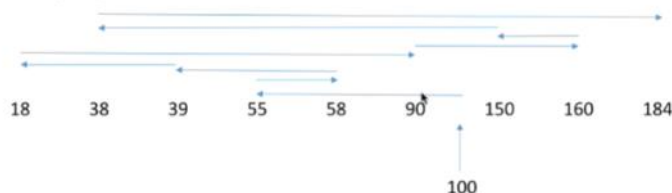
最后是**传输时间**（和读取时间/磁盘的转速有关），假设每个磁道可以存 N 字节数据，因此 b 字节需要 b/N 个磁道存储，而读写一个磁道所需的时间刚好又是转一圈所需的时间 $1/r$ ，因此传输时间就是 $T_t = (1/r) * (b/N) = b/(rN)$ 。

总的**平均存取时间** $T_a = T_s + 1/2 * r + b/(rN)$ 。延迟时间和传输时间取决于硬件，操作系统无法优化，但是**寻道时间可以优化**，因此出现了磁盘调度算法。

2、先来先服务算法FCFS

根据进程请求访问磁盘的先后顺序进行调度。

假设磁头的初始位置是100号磁道，有多个进程先后陆续地请求访问55、58、39、18、90、160、150、38、184号磁道
按照FCFS的规则，按照请求到达的顺序，磁头需要依次移动到55、58、39、18、90、160、150、38、184号磁道



磁头总共移动了 $45+3+19+21+72+70+10+112+146 = 498$ 个磁道

响应一个请求平均需要移动 $498/9 = 55.3$ 个磁道（平均寻找长度）

优点：公平；如果请求访问的磁道比较集中的话，算法性能还算过的去

缺点：如果有大量进程竞争使用磁盘，请求访问的磁道很分散，则FCFS在性能上很差，寻道时间长。

3、最短寻找时间优先算法SSTF

优先处理离当前磁头最近的磁道，根据贪心算法的思想。

假设磁头的初始位置是100号磁道，有多个进程先后陆续地请求访问 55、58、39、18、90、160、150、38、184 号磁道



磁头总共移动了 $(100-18) + (184-18) = 248$ 个磁道
 响应一个请求平均需要移动 $248/9 = 27.5$ 个磁道（平均寻找长度）
 优点：性能较好，平均寻道时间短
 缺点：可能产生“饥饿”现象

Eg：本例中，如果在处理18号磁道的访问请求时又来了一个38号磁道的访问请求，处理38号磁道的访问请求时又来了一个18号磁道的访问请求。如果有源源不断的18号、38号磁道的访问请求到来的话，150、160、184号磁道的访问请求就永远得不到满足，从而产生“饥饿”现象。

产生饥饿的原因在于：磁头在一个小区域内来回来去地移动

4、扫描算法SCAN

移动方式和电梯相似，往一个方向一条路走到底然后换方向继续走到底。

假设某磁盘的磁道为0~200号，磁头的初始位置是100号磁道，且此时磁头正在往磁道号增大的方向移动，有多个进程先后陆续地请求访问 55、58、39、18、90、160、150、38、184 号磁道



磁头总共移动了 $(200-100) + (200-18) = 282$ 个磁道
 响应一个请求平均需要移动 $282/9 = 31.3$ 个磁道（平均寻找长度）
 优点：性能较好，平均寻道时间较短，不会产生饥饿现象
 缺点：①只有到达最边上的磁道时才能改变磁头移动方向，事实上，处理了184号磁道的访问请求之后就不需要再往右移动磁头了。

②SCAN算法对于各个位置磁道的响应频率不均匀（如：假设此时磁头正在往右移动，且刚处理过90号磁道，那么下次处理90号磁道的请求就需要等磁头移动很长一段距离；而响应了184号磁道的请求之后，很快又可以再次响应184号磁道的请求了）

只有到了最边上的磁道才能改变磁头移动方向

5、LOOK边移动边观察调度算法

扫描算法的缺点就是到了最边上才换方向，实际上不需要到最边上就可以换方向了，因此该算法在SCAN算法的基础上增加一项：如果磁头移动方向没有别的请求就立刻改变方向。

假设某磁盘的磁道为0~200号，磁头的初始位置是100号磁道，且此时磁头正在往磁道号增大的方向移动，有多个进程先后陆续地请求访问 55、58、39、18、90、160、150、38、184 号磁道



磁头总共移动了 $(184-100) + (184-18) = 250$ 个磁道
 响应一个请求平均需要移动 $250/9 = 27.5$ 个磁道（平均寻找长度）
 优点：比起SCAN算法来，不需要每次都移动到最外侧或最内侧才改变磁头方向，使寻道时间进一步缩短

如果在磁头移动方向上已经没有别的请求，就可以立即改变磁头移动方向

6、循环扫描C-SCAN算法

为了解决SCAN算法各个磁道相应不均匀的问题，它在到磁盘尾部返回的时候不处理任何请求而是快速返回到起始地点。

假设某磁盘的磁道为0~200号，磁头的初始位置是100号磁道，且此时磁头正在往磁道号增大的方向移动，有多个进程先后陆续地请求访问 55、58、39、18、90、160、150、38、184 号磁道



磁头总共移动了 $(200-100) + (200-0) + (90-0) = 390$ 个磁道
 响应一个请求平均需要移动 $390/9 = 43.3$ 个磁道（平均寻找长度）
 优点：比起SCAN来，对于各个位置磁道的响应频率很平均。
 缺点：只有到达最边上的磁道时才能改变磁头移动方向，事实上，处理了184号磁道的访问请求之后就不需要再往右移动磁头了；并且，磁头返回时其实只需要返回到18号磁道即可，不需要返回到最边缘的磁道。另外，比起SCAN算法来，平均寻道时间更长。

只有到了最边上的磁道才能改变磁头移动方向。磁头返回途中不处理任何请求

7、C-LOOK算法

C-SCAN算法缺点就是到达最边上磁道才能改变磁头移动的方向并且磁头返回时也不一定要返回到起始点，因此它发现磁头移动方向没有请求就调换方向返回，返回的时候前面没有请求的时候再次调换方向。

假设某磁盘的磁道为0~200号，磁头的初始位置是100号磁道，且此时磁头正在往磁道号增大的方向移动，有多个进程先后陆续地请求访问55、58、39、18、90、160、150、38、184号磁道



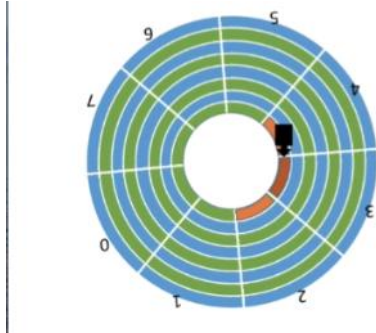
磁头总共移动了 $(184-100) + (184-18) + (90-18) = 322$ 个磁道
响应一个请求平均需要移动 $322/9 = 35.8$ 个磁道（平均寻找长度）

优点：比起 C-SCAN 算法来，不需要每次都移动到最外侧或最内侧才改变磁头方向，使寻道时间进一步缩短

十二、减少磁盘延迟时间的方法

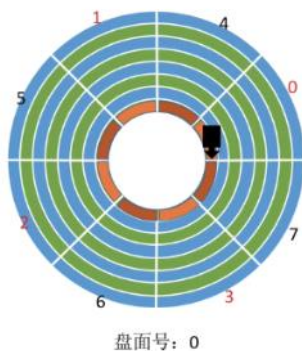
延迟时间就是将目标扇区移动到磁头下面所花费的时间，由于磁头读取完一个扇区后需要一小段时间处理数据，而磁盘又在不断的旋转，因此假设2、3号扇区相邻着排列，读取完2号扇区后就无法连续不断的读入3号扇区，必须等待盘片旋转完一圈后再次回到3号扇区才能被读取。

逻辑上相邻的扇区如果在物理上也相邻，那么延迟时间就会很长。



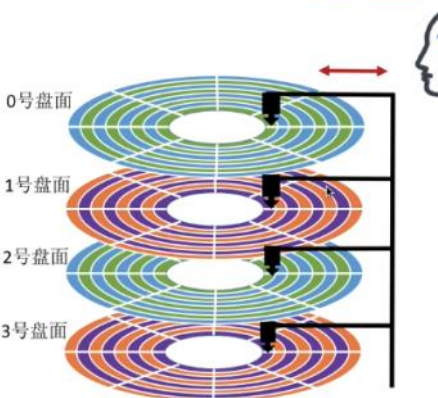
1、减少延迟时间的方法——交替编号

让逻辑上相邻的扇区在物理上有一定的间隔，可以使得读取连续逻辑扇区所需要的延迟时间更少。

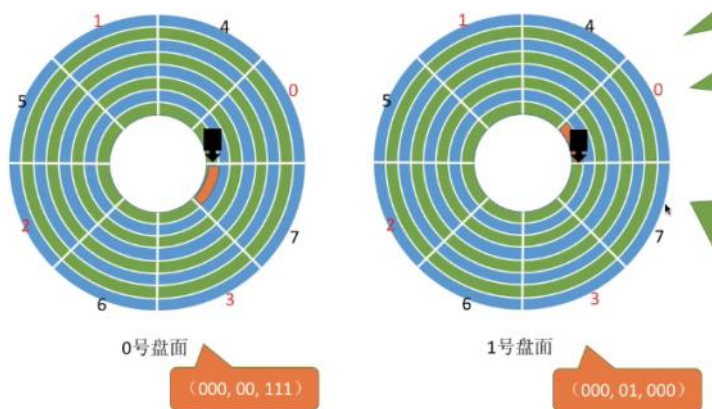


2、磁盘地址结构的设计

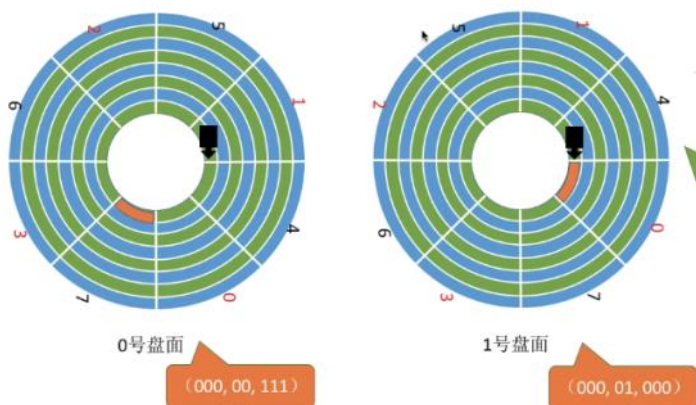
磁盘的物理地址是（柱面号、盘面号、扇区号）而不是（盘面号、柱面号、扇区号），如果是后者的话（00, 000, 000）到（00, 000, 111）只需要转2圈就读完了，之后再读（00, 001, 000）到（00, 001, 111）这个连续数据就需要启动磁头臂，将磁头移动到下一个磁道，物理移动的话需要耗费的时间多；如果是（柱面号，盘面号，扇区号）的话（000, 00, 000）到（000, 00, 001）读完后读取（000, 01, 000）到（000, 01, 111）这个区域，此时不需要移动磁头臂改变柱面，只需要激活下面盘面的磁头臂即可，因此节省了时间。



3、减少延迟时间的方法——错位命名



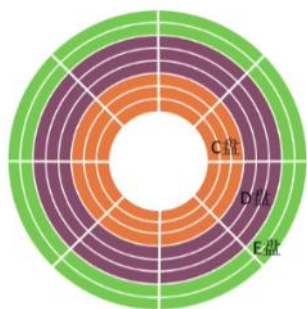
如上图0号盘面最内侧的扇区读完后接下来读1号扇区的最内侧扇区，0号盘面转2圈读取完，此时磁头臂的位置刚好是1号盘面数据的起始点，但是磁头臂需要花费一些时间处理数据，因此要转完一圈后再回来才能继续读取，这样增加了延迟时间。
解决方法也很简单，就是0号盘面和1号盘面的扇区号都错位命名即可，如下图



十三、磁盘的管理

1、磁盘初始化

磁盘在使用前需要进行低级格式化（物理格式化），将磁盘的各个磁道划分为扇区，一个扇区由扇区头、扇区尾、数据区域组成，管理扇区的各种数据结构都放在头和尾；然后将磁盘进行分区分成c盘d盘f盘这些；最后进行逻辑格式化，即创建文件系统包括根目录、初始化所用的数据结构（位示图、空闲分区表）等等。



2、引导块

计算机开机的时候需要进行一系列的初始化工作，这工作由初始化程序（自举程序）完成。初始化程序存放在ROM中，ROM是只读存储器，这里的数据在出厂的时候就写入且无法修改。这样就有问题了，初始化程序如果要更新的话，程序在ROM中无法更改怎么办？解决方案就是ROM里面放“自举装入程序”，开机运行的时候先运行该程序然后将自举程序装入内存完成初始化。

完整的自举程序存放在磁盘的启动块上，拥有启动分区的磁盘成为启动磁盘或者系统磁盘，电脑的c盘就是。

3、坏快的管理

坏快就是坏了的磁盘扇区，由于这是硬件上坏掉的，它无法通过系统来修复。

对于简单的磁盘，可以在逻辑格式化的时候检查坏块，说明哪个是坏扇区哪个是好扇区，在FAT上标注。

对于复杂的磁盘，可以设置一个磁盘控制器来维护一个坏块链并管理备用扇区，当访问到坏扇区的时候就跳转到备用扇区。

